

Quantitative Approaches in Modern Biology

BIO 165/265 — Lecture Script

Shaili Mathur Jonas Cremer

2026-10-06

Welcome

Welcome to BIO165/265

For beginners. This course is for biology students with little background in coding, mathematics, or statistics. We introduce every concept step by step and focus on the conceptual logic rather than on mathematical proofs. We apply each concept or method to real biological data in Python-based dry labs. If much of this is new to you, plan to invest time: getting comfortable with coding and quantitative reasoning takes practice, especially in the first weeks. We will guide you through it with these detailed notes, the dry labs, tutorials, and office hours. The potential reward is substantial: quantitative skills are increasingly central to modern, data-rich biology, and this course provides a foundation from which you can develop them further.

A guiding principle: from coin tosses to cells. Fundamental statistical ideas in biology can be understood through the simplest random experiment there is — tossing a coin. We will therefore return to the coin toss repeatedly throughout this course to illustrate why averages become reliable ([law of large numbers](#)), how uncertain an estimate is ([standard error](#), [bootstrapping](#)), and how to decide whether a difference is real ([is the coin fair?](#)). The same counting logic then describes sequencing reads and molecule numbers ([counting statistics](#)) and the stochastic gene expression of individual cells ([stochastic processes](#)).

AI as an opportunity to promote quantitative analysis. We encourage the use of AI tools in this course. Large language models can help substantially with coding, debugging, and exploring analysis approaches, and they make quantitative analysis more accessible than ever before — a real opportunity, especially if you are new to coding. At the same time, it is essential to maintain *conceptual control*: understand what each analysis step does and why, check AI-generated code and results critically, and judge whether an outcome makes biological and statistical sense. See [Use of AI tools](#) for our recommendations and the course policy.

Lecture Notes. This website contains detailed lecture notes that support the in-class lectures and provide additional context and detail. The notes are still under active development: we have newly put them together for this year's course and continue to extend and improve them. Your feedback is highly appreciated — please let us know about typos, unclear explanations, or suggestions for improvement. Contact [Jonas Cremer](#) or open an issue on [GitHub](#).

Jonas Cremer, March 2026

Part I

Week 1: Introduction and getting started

1 Introduction

1.1 The triangle of quantitative reasoning in modern biology

At its core, (modern) biology is also a quantitative data science. Advances in experimental methods to collect data, a detailed attention to numbers, and a broad range of clever quantitative approaches have promoted major breakthroughs in all fields of biology since more than hundred years. Over time the trend to quantification accelerated. Modern experimental methods, including sequencing, imaging, metabolomics, proteomics, CRISPR screens, and single-cell technologies promote unprecedented insights into the function of cells, organisms, and ecosystems, through massive datasets which need to be analyzed. Methods to collect and analyze data develop in a breathtaking speed. As a result, modern biology requires a some fundamental understanding of data handling and quantitative analysis.

Seen as quantitative data science, reasoning in biology consists of three fundamental and interconnected steps.

1. Data generation and handling: How biological data is generated, organized, stored, cleaned, and structured.
2. Modeling & Fitting: How quantitative relationships summarize patterns and reveal biological principles.
3. Evaluation & Testing: How we judge models, quantify uncertainty, and test hypotheses

We call this here the triangle of quantitative reasoning, illustrated in Figure 1.1. Across this triangle runs a natural workflow loop. Visualization connects data to models by revealing structure and patterns. Uncertainty analyses connects models to

evaluation by quantifying variability and confidence. And findings from the evaluation and testing informs new data generation and thus new analyses.

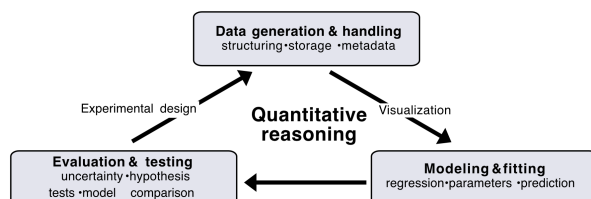


Figure 1.1: **The quantitative reasoning in biology.** Quantitative analyses consist of essential steps which are commonly performed repeatedly to generate better biological understanding.

Performing these steps efficiently and rigorously in a biological research project requires, besides a deep knowledge of the biological system one studies, an applicable knowledge in mathematics, statistics, and coding.

In this course, we introduce fundamentals of quantitative data analysis in modern biology. Starting with the basics of data handling, we will go through the triangle of quantitative reasoning, adding new analysis steps each week and applying those to real datasets through lectures and “dry labs”.

i Info: Popper revisited

The triangle of quantitative reasoning can be viewed as a modern, data-driven implementation of classical scientific reasoning as articulated by Karl Popper. In Popper’s framework, science advances through the formulation of hypotheses, the derivation of predictions, and attempts at falsification through experiment. Through the lens of data science, these statements can be made more rigorous. Hypotheses are encoded as mathematical or statistical models, predictions are expressed as quantitative expectations, and falsification is replaced by model evaluation, uncertainty quantification, and comparison across datasets. Rather than binary rejection, evidence accumulates continuously, and models are refined or replaced as new data become

1.2 Biological breakthroughs through quantitative reasoning

To start, let us step back and clarify why quantitative reasoning is so powerful and has become indispensable in modern biology.

Given the breathtaking progress in all areas of biology over the last century there are almost countless landmark discoveries that were possible only because of some quantitative reasoning, often as an elegant combination of careful experiments, mathematical models, and statistical analysis.

We here introduce three classical examples which changed neuroscience, biochemistry, and evolutionary biology.

1. **Hodgkin–Huxley and the foundation of neural signaling:** The work of Alan Hodgkin and Andrew Huxley began with precise voltage-clamp measurements of membrane currents in the squid giant axon. Using these data together with electrical circuit theory, they constructed a quantitative mathematical model in which voltage- and time-dependent sodium and potassium conductances determine membrane dynamics (Fig. 1.2A).

Notably, the kinetic and biophysical parameters of the model were constrained by voltage-clamp experiments and other independent measurements, rather than freely adjusted to reproduce action potentials. The model was then evaluated by showing that it could reproduce the timing, amplitude, and overall shape of experimentally observed action potentials (Fig. 1.2B,C).

By linking ion-channel kinetics to electrical signaling, this work established a mechanistic framework for neural excitability. The Hodgkin–Huxley model became the foundation of modern conductance-based modeling approaches in neurobiology, far beyond the original experimental system,

from single-neuron models to large-scale simulations of neural circuits.

! Quantitative takeaway

Quantitative analysis was essential to show how the collective behavior of many ion channels gives rise to predictable neural signals. This mechanistic link between microscopic kinetics and macroscopic electrical behavior could not be established from qualitative observations alone.

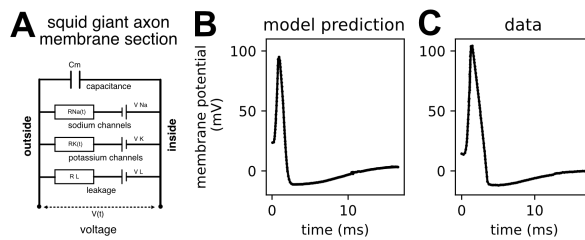


Figure 1.2: **Hodgkin–Huxley and the foundation of neural signaling.** (A) Schematic of the Hodgkin–Huxley modeling framework, in which membrane voltage dynamics are described using an equivalent electrical circuit. Voltage- and time-dependent sodium and potassium conductances represent the collective behavior of ion channels. (B) Action potential predicted by the Hodgkin–Huxley model using parameters determined independently from voltage-clamp experiments and biophysical measurements. (C) Experimentally measured action potential from the squid giant axon. Model predictions and experimental data adapted from Hodgkin and Huxley (1952). The correspondence between prediction and experiment illustrates how quantitatively constrained models can link ion-channel kinetics to emergent electrical behavior.

2. **Michaelis–Menten kinetics and the quantitative description of enzymatic catalysis:** Leonor Michaelis and Maud Menten investigated how enzymatic reaction rates depend on substrate concentration by systematically

measuring reaction velocities across a wide range of substrate levels. They observed a characteristic saturating relationship that could not be explained by linear reasoning alone. To account for this behavior, they introduced a simple kinetic model based on enzyme–substrate binding and catalytic turnover, yielding a nonlinear rate law characterized by two parameters: the maximum reaction rate $V_{\max}$ and the Michaelis constant K_m (Fig. 1.3).

The model was quantitatively evaluated by testing whether it reproduced measured rate–concentration curves. The introduction of $V_{\max}$ represented an important conceptual advance: it formalized the idea that enzymatic reactions are limited by a finite catalytic capacity. This insight later enabled a molecular interpretation of saturation in terms of an intrinsic turnover rate multiplied by enzyme abundance, $V_{\max} = k_{\text{cat}}[E]$, linking macroscopic rate measurements to underlying molecular processes.

This framework became foundational in molecular biology, well beyond the original invertase experiments. A prominent later example is the β -galactosidase assay used in bacterial gene regulation, which enabled precise and reproducible measurements of enzymatic activity across conditions (Fig. 1.3). Such quantitative assays made it possible to disentangle enzyme concentration from catalytic efficiency and helped establish how gene expression and metabolism can be described using predictive quantitative models. The same mathematical structure reappeared in microbial physiology, most notably in Monod’s description of how cellular growth rates saturate with nutrient availability, highlighting the broad applicability of the Michaelis–Menten framework across biological scales.

! Quantitative takeaway

Quantitative modeling of nonlinear rate–concentration data revealed enzymatic saturation and enabled the extraction of meaningful kinetic parameters. Concepts such as $V_{\max}$, K_m , and ultimately k_{cat} emerged through the quantitative

analysis.

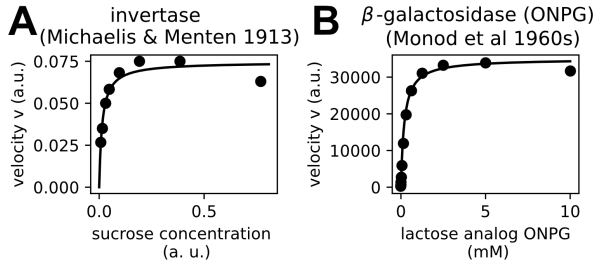


Figure 1.3: **Michaelis–Menten kinetics as a quantitative framework for enzymatic catalysis.** (A) Original measurements by Michaelis and Menten showing reaction velocity as a function of substrate concentration for the enzyme invertase, revealing a saturating nonlinear relationship. (B) Later measurements of β -galactosidase activity using the chromogenic substrate ONPG, illustrating the same characteristic saturation behavior in a widely adopted quantitative assay. In both cases, the Michaelis–Menten model captures the observed rate–concentration curves using two parameters, $V_{\max}$ and K_m , providing a mechanistic interpretation of enzymatic saturation. These quantitative assays laid the foundation for modern enzymology and for subsequent quantitative studies of gene regulation and metabolism.

3. **Luria–Delbrück and the quantitative foundations of molecular evolution:** The work of Salvador Luria and Max Delbrück provided a quantitative resolution to a central question in evolutionary biology: whether genetic changes arise randomly or are induced by environmental challenge. Using carefully designed experiments, they measured the number of resistant bacterial colonies across many parallel cultures grown under identical phage encountering conditions. They observed extreme fluctuations in colony counts between cultures. This pattern is incompatible with mutations arising only in response to phage exposure and thus supports randomly occurring mutations. We will discuss the quantitative analysis in more detail

in week 6 of this course when discussing probability and statistics. The work shape our understanding of evolution and continues to underpin modern studies of molecular evolution, population dynamics, and lineage tracing.

! Quantitative takeaway

Quantitative analysis of population-to-population variability was essential to distinguish random mutation from environmentally induced change—an inference that could not be drawn from qualitative observations or single experiments.

Modern biology: Quantitative analysis plays an equally central role in modern experimental biology. As experimental techniques now generate data at unprecedented scale and complexity, systematic quantitative analysis is essential for extracting meaningful biological insight. Consider, for example, the many modern biological approaches that can be understood as high-dimensional extensions of the classical examples discussed above. In neuroscience, large-scale recording and connectomics aim to link neural structure and activity to function across entire circuits, extending the Hodgkin–Huxley logic from single neurons to networks of thousands or millions of cells. In metabolism, omics technologies such as proteomics and metabolomics provide global measurements of enzyme abundances and metabolite concentrations, which are increasingly interpreted through quantitative frameworks such as metabolic and resource-allocation models that build directly on Michaelis–Menten kinetics. In evolutionary biology, lineage barcoding and sequencing-based tracking experiments allow researchers to follow the fate of thousands of lineages in parallel, bringing the probabilistic logic of the Luria–Delbrück experiment into a genomic, time-resolved setting.

Across these domains, experimental advances require quantitative analysis to identify structure in high-dimensional data, evaluate competing models, and extract meaningful biological insight.

1.3 This course, dry labs to learn quantitative analysis and a note for beginners

Before we start we want to emphasize important aspects of this course.

Scope and limitation of this course: In this class, we will only consider a small part of the remarkable breath of quantitative methods, focusing on the most essential and those important for modern omics analyses. Second, we will focus on the conceptual logic of analysis steps and their realization in Python and hands on computational notebooks, not mathematical proofs or rigor. The goal is a applicable understanding of essential steps, to utilize in research projects and to promote further studies of quantitative methods.

Dry labs: To learn quantitative analysis, we believe it is essential to apply concepts immediately to real datasets. In this course, we realize this hands-on approach through “dry labs” as problem sets. Each dry lab is built around a (Python based) computational notebook, an interactive document that combines narrative text, executable code, and visual or numerical output in a single workflow. Using this format, we introduce new analysis concepts, apply them directly to data, and interpret the results in context.

Different quantitative backgrounds: Students in this course arrive with a wide range of backgrounds in coding, mathematics, statistics, and biology. Some topics may feel familiar, while others may be entirely new. The dry labs and accompanying scripts are structured to support students with less prior experience by building foundational skills step by step. Students with more advanced backgrounds can revisit core ideas and deepen their understanding by analyzing datasets of their choice or engaging with optional problem sets.

For those of you who are new to coding or to many of the mathematical and statistical concepts introduced in this course, we want to emphasize that getting started is more feasible than ever before. Modern programming tools, extensive documentation, and AI have substantially lowered the barrier to entry. At the same time, becoming comfortable with quantitative analysis

still involves a learning curve and can feel challenging at first. It takes time to get used to the analysis workflow, become familiar with available tools, and develop a conceptual understanding of the essential analysis steps. To overcome this hurdle and make coding and quantitative analysis a routine part of your scientific workflow, we strongly encourage you to work collaboratively and learn together. We are also here to help.

i Info: Tutorials and office hours

To support students at all levels, the course offers optional introductory sessions during the first two Thursdays, an optional Thursday afternoon support session each week, and additional office hours throughout the quarter. We strongly encourage you to take advantage of these resources as you navigate the course.

1.4 Use of AI tools

Before starting, we want to briefly address the use of AI tools. Large language models and AI systems such as OpenAI ChatGPT, Google Gemini, or Anthropic Claude are developing at a remarkable pace. These tools are particularly effective for coding and data analysis and can substantially accelerate quantitative workflows in the life sciences.

AI can assist with writing and debugging code, exploring unfamiliar analysis approaches, and reducing the time required to implement computational methods. At the same time, it remains essential to maintain a solid conceptual understanding of the methods and analysis steps being used. We recommend doing introductory coding over the first few weeks of the with little to no use of AI for writing the code itself. However, AI tools (or forums such as stack overflow) are useful for debugging or helping with you're stuck. Also be sure to look at and engage with any code generated by AI; AI hallucinations are real, and uncritical or overly reliant use of AI tools can lead to misleading or even blatantly incorrect analysis results.

We also want to encourage you to use AI responsibly, with

consideration for the large environmental impact that AI has; as of now (and we expect for the foreseeable future), AI it requires an enormous amount of energy and water use. We link materials for a further discussion of this issue in Canvas.

1.4.1 Using AI tools effectively and responsibly

To use AI tools productively while avoiding common pitfalls, we recommend the following practices:

- 💡 **Best practice: Using AI effectively and responsibly**
 - **Break tasks into small steps.** Ask for help with specific tasks rather than entire analysis pipelines.
 - **Ensure you understand the code generated with AI assistance.** Verify that you follow the logic of the code. If unsure of specific steps, you can ask the AI to explain what the code does and why.
 - **Provide all the information needed when formulating an AI request.** E.g. provide explicit information about data structure, column names, units, analysis goals. Supplying a table header or a short data description often leads to much more useful suggestions.
 - **Test the code output** As with any code, AI-generated code must be tested and analysis results must be validated. E.g. test code on small datasets or in limiting cases where the expected outcome is known.
 - **Protect sensitive data.** Do not share confidential, identifying, or unpublished data with AI tools.

1.4.2 AI use in this course

In this course, we encourage the use of AI tools with a few exceptions.

AI when using problem sets. You may use AI tools to help set up analyses, explore code, and troubleshoot issues. Exceptions are problems or checkpoints explicitly marked as **no AI**, which must be completed without AI assistance to assess your own understanding.

AI during the project phase. You may also use AI tools during the final project phase for tasks such as coding, data wrangling, or plotting. Any such use must be disclosed in both the written report and the presentation (for example, a brief note describing what the AI was used for). You may not use AI tools to write the main text of the final project report; the scientific narrative and interpretation must be your own.

Table 1.1: Summary - AI use in this course

Course component	AI use allowed
Problem sets (problem marked no AI)	No
Problem sets (all other problems)	Yes
Final project: data analysis and coding	Yes
Final project: writing and scientific narrative	No

2 Getting Python and notebooks to Run

Let us next see how to run analysis code and the computational notebooks used in each dry lab. In this course, we will use the Python programming language together with Jupyter for data analysis and visualization. There are many ways to run Python and Jupyter, including Google Colab, JupyterLab, PyCharm, Anaconda Navigator, or directly from the terminal. All of these options are perfectly fine for this course. However, we highly recommend using Visual Studio Code (VS Code) together with a clean Python installation on your own computer. VS Code is free, works across operating systems, and integrates very well with Python scripts and Jupyter notebooks. Setting up your own local environment gives you the most flexibility to extend analyses and work on your own projects in the future.

i Info: Other editor preferences and R use

Feel free to use the Python editor and development environment you prefer. Similarly, you may also use **R**, another powerful language for data handling and analysis, to work on the problem sets and provide solutions. However, computational notebooks are provided only in Python, and using R would require translating the notebooks yourself.

2.1 Setup VS Code and Python

Installing VS Code and Python is straightforward. Basic instructions are provided here.

2.1.1 VS Code

1. Go to <https://code.visualstudio.com> and download the installer for your operating system.
2. Install VS Code using the default options.
3. Install the Python extension: Open VS Code and go to the Extensions view (left sidebar). Search for **Python** and install the Python extension by Microsoft. This extension is strongly recommended and assumed for the remainder of the course.
4. Install the Jupyter extension: In the Extensions view, search for **Jupyter** and install the Jupyter extension by Microsoft. This extension is required to open and run Jupyter notebooks (`.ipynb`) inside VS Code.
5. Set up a new project folder for this class in a designated location on your computer.

2.1.2 Python

Next, install Python. There are several ways to do this. We recommend installing Python via **Miniforge**, which provides the **conda** and **mamba** package managers. This approach helps avoid version conflicts that can arise when incompatible packages are installed globally.

1. Download Miniforge from <https://github.com/conda-forge/miniforge>. Choose the installer appropriate for your system (e.g. **arm64** for newer Apple Silicon Macs).
2. Install Miniforge using the default settings.
3. After installation, open a terminal and verify that Python and **mamba** are available:

```
python --version  
mamba --version
```

We recommend doing this in the terminal integrated into VS Code.

2.1.3 The terminal and command line

Many setup and debugging steps require entering commands in a terminal (also called a shell or command prompt). In VS Code, the terminal is typically located at the bottom of the editor. You can open it via **View→Terminal** or using the shortcut **Ctrl+`** (Windows/Linux) or **Cmd+`** (macOS).

2.2 Python environments and packages

Python becomes powerful when used together with external packages such as **numpy**, **pandas**, **matplotlib**, and **scipy**. However, installing many packages globally can easily lead to version conflicts and broken setups.

To avoid these issues, it is highly useful to work in a dedicated Python environment: a self-contained workspace with its own Python version and package set. This also supports reproducibility and makes it easier to share code. Here, we will use the environment tool **mamba**.

1. Recommendation: Before creating environments, make sure **conda-forge** is used as the primary package channel (this only needs to be done once per machine and reduces chances for package incompatibilities):

```
conda config --add channels conda-forge
conda config --set channel_priority strict
```

2. Create a new environment named **BI0165**:

```
mamba create -n BI0165 python=3.11
```

3. Activate the environment:

```
mamba activate BI0165
```

4. Install core packages needed for the first part of the course:

```
mamba install numpy pandas matplotlib scipy seaborn jupyter ipykernel
```

5. If everything works, you can already install packages that will be used later in the class:

```
mamba install pyDESeq2 umap-learn
```

You might get a request to downgrade numpy to an earlier version which you can accept.

6. The environment should appear as a selectable kernel in Jupyter (after restarting VS Code).

A kernel is the Python process that runs the notebook and determines which environment (Python version and packages) is used. In VS Code, the kernel can be selected via a button in the upper right corner of the notebook. See info box if the environment does not appear as a selectable kernel.

7. You can check the packages installed in the current environment:

```
mamba list
```

i Info: Troubleshooting: VS Code cannot find my environment / kernel

If VS Code does not show the BIO165 environment as a selectable kernel, there are a few options.

1. **Restart VS Code.** This often refreshes the environment detection and allows you to select the environment.
2. **Select the interpreter manually:** Open the Command Palette (Cmd+Shift+P on macOS, Ctrl+Shift+P on Windows/Linux), then run Python: Select Interpreter and choose BIO165.
3. **Install/register the kernel explicitly (fallback):**

```
mamba activate BIO165  
python -m ipykernel install --user --name BIO165
```

2.2.1 Using environments and kernels in VS Code

- Open a Jupyter notebook (`.ipynb`) in VS Code.
- In the upper-right corner select the Python interpreter or Jupyter kernel corresponding to the environment **BI0165**.

Registering or selecting one environment does not restrict you to it. You are free to create and use multiple environments and choose the appropriate kernel each time you run a notebook.

2.2.2 Running Jupyter notebooks

Jupyter notebooks are interactive documents that combine narrative text, executable code, and output (such as tables or plots) in a single file.

Rather than launching Jupyter separately, we will run notebooks directly inside VS Code.

To open and run a notebook:

1. Open a Jupyter notebook file in VS Code (e.g. the coding tutorial from week 1).
2. Select the Python kernel corresponding to your environment (e.g. **BI0165**) when prompted, or via the kernel selector in the upper-right corner.
3. Execute code cells using **Shift+Enter**.

Next steps: A short introduction to Python and Jupyter is provided in a tutorial notebook for this course, which we will work through together during the first Thursday class.

3 Biological data: Types and the Concept of Tidy Data

Life operates across molecular, cellular, organismal, and ecological scales. A wide range of experimental approaches has been developed to measure and observe processes operating at these different levels. As a result, biological data can take many different forms. A few examples are summarized here:

Example

Examples of biological data across molecular, cellular, and organismal scales.

- **Molecular scale:** gene expression measured by RNA sequencing, protein abundance measured by mass spectrometry, metabolite levels measured by chromatography-based methods.
- **Cellular scale:** cell growth measured by optical density or cell counts, gene expression or signaling activity measured by fluorescent reporters, single-cell states measured by flow cytometry or single-cell sequencing.
- **Organismal and tissue scale:** spatial organization measured by microscopy, physiological signals measured by sensors or recordings, behavior and development measured by tracking or observation.

As a consequence, the file formats we interact with also vary widely, including, for example, CSV, Excel, FASTA/FASTQ, TIFF, PNG, JSON, HDF5, and many others. While each data type and format requires specific handling steps, there are two fundamental points we want to emphasize here to get going:

metadata matters, and tables are fundamental for quantitative analyses.

3.1 Metadata

Every dataset comes with context: how and when an experiment was performed, what strain or sample was used, which instrument generated the signal, under which environmental conditions, what preprocessing was done, and who carried out which step. This contextual information is called **metadata**. Often included information are summarized here:

Example: Important metadata

- **Sample and biological context:** sample identifiers, strain or genotype names, organism, tissue or cell type, treatment conditions, experimental group labels
- **Measurement definition:** units, measurement type, time stamps, sampling intervals, spatial coordinates, replicate identifiers
- **Experimental and technical context:** instrument type and version, acquisition settings, calibration information, reagent or antibody batches, lot numbers
- **Processing and analysis history:** preprocessing steps already applied, normalization methods, filtering criteria, background subtraction, software versions
- **Experimental notes and deviations:** protocol deviations, known issues or failures, missing data explanations, quality flags

The quality of metadata determines whether a dataset is interpretable and reusable. As a result, thoughtfully designed metadata are an essential part of any good research project.

Equally important, when analyzing data, we must ensure that relevant metadata remain attached to the datasets we work with. One important concept to do that are tidy data formats which we will discuss below.

3.2 Tabular data

Regardless of the experimental method or the form of the raw data, derived data used for quantitative analysis are typically represented as **tabular data**, with numerical values organized into rows and columns.



Example: Tabular data

- Image-based analyses of cells often produce tables in which each row represents a cell and each column reports a measured property, such as cell size or the intensity of a fluorescent reporter.
- RNA-seq experiments transform short sequencing reads into tables where rows correspond to genes and columns correspond to samples or experimental conditions.
- Plate readers generate tables that record how cell abundance or activity changes over time.

As a result, tabular data form the foundation of nearly all quantitative analyses in biology, and the ability to efficiently work with tables is essential for rigorous data analysis.

In practice, however, tabular data of interest is often *messy data*. Common examples include Excel files with multiple unrelated tables on the same sheet, merged cells or color-coded labels instead of explicit variables, inconsistent column names or units, or separate sheets for different experimental conditions or replicates. Such structures create a substantial burden for quantitative analysis, and cleaning and reorganizing data often takes more time than the analysis itself. To promote efficient and rigorous data analysis, careful data organization is therefore crucial.

3.3 Tidy data as good default

To better organize data, the concept of **tidy data** has become increasingly influential in data science and quantitative biology. The core idea is to structure tabular data such that it is consistent, flexible, and well suited for computational analysis. Furthermore, tidy data makes the meaning of each data point explicit and allows metadata to be naturally integrated alongside measurements.

Historical context: Tidy data

The tidy data concept is relatively recent. It was popularized by Hadley Wickham in 2014, building on earlier ideas from statistics and database design, and supported by major advances in computational power and memory. Tidy data promotes flexible, computation-oriented representations that are well suited for the analysis of large and complex datasets. We here introduce an adjusted formulation highlighting its use in biological datasets.

For the analysis of biological data, we adopt in this course the following practical tidy data principles:

Definition: Tidy data for biology

Three major rules define tidy data:

1. Each row corresponds to a single, well-defined observation.
2. Each column corresponds to a clearly defined variable, identifier, or metadata information.
3. Needed identifiers and metadata are included as columns so that data points can be interpreted and compared meaningfully.

To illustrate these principles, consider measurements of the expression of β -galactosidase (LacZ), the enzyme responsible for lactose utilization. The lactose operon is a classical system that has been used to uncover fundamental principles of gene

regulation. Each observation corresponds to one measured LacZ expression value for a specific strain under a specific growth condition.

Following common table layouts in papers and textbooks, one might store these values in a *wide table*, with rows corresponding e.g. to a specific experimental condition and columns corresponding to cell types.

condition	WT	mut1	mut2
no lactose	12.4	10.8	11.6
with lactose	245.2	180.5	92.3

While this representation is compact and easy to read, it is more difficult to handle in code. Moreover, it makes it cumbersome to integrate metadata or extend the dataset with additional information.

In a tidy data representation, each row instead corresponds to a single expression measurement. Separate columns explicitly encode the cell type, experimental condition, and measured expression level.

gene	strain	condition	expression
lacZ	WT	no lactose	12.4
lacZ	mut1	no lactose	10.8
lacZ	mut2	no lactose	11.6
lacZ	WT	with lactose	245.2
lacZ	mut1	with lactose	180.5
lacZ	mut2	with lactose	92.3

This representation has several important advantages:

Advantage 1: Explicit meaning of observations. In this structure, the meaning of each row is unambiguous: it represents one measurement taken under a specific set of biological conditions.

Advantage 2: Natural integration of metadata. Additional information such as replicate identity, batch, measurement

date, or operator can be incorporated naturally by adding further columns, without changing the overall structure of the table.

gene	strain	condition	expression	date	experimenter	instrument	notes
lacZ	WT	no lactose	12.4	2025-10-08	Your Name	Tecan Spark	
lacZ	WT	with lactose	245.2	2025-10-08	Your Name	Tecan Spark	
lacZ	mut1	no lactose	10.8	2025-10-08	Your Name	Tecan Spark	pipetting error
lacZ	mut1	with lactose	180.5	2025-10-12	Your Name	Tecan Spark	
lacZ	mut2	no lactose	11.6	2025-10-12	Your Name	Tecan Spark	
lacZ	mut2	with lactose	92.3	2025-10-12	Your Name	Tecan Spark	

Advantage 3: Straightforward extension to more complex datasets. This structure extends naturally to more complex settings. For example, measurements of additional genes can be incorporated simply by adding a gene identifier as an additional column, without altering the overall organization of the table.

gene	strain	condition	expression
lacZ	WT	no lactose	12.4
lacZ	mut1	no lactose	10.8
lacZ	mut2	no lactose	11.6
lacZ	WT	with lactose	245.2
lacZ	mut1	with lactose	180.5

gene	strain	condition	expression
lacZ	mut2	with lactose	92.3
rplA	WT	no lactose	310.5
rplA	mut1	no lactose	305.1
rplA	mut2	no lactose	312.7
rplA	WT	with lactose	308.9
rplA	mut1	with lactose	306.8
rplA	mut2	with lactose	311.2

Advantage 4: Alignment with computational tools. Tidy data aligns well with how modern data analysis tools operate, including the pandas library in Python that we will use throughout this course. Tidy data enables efficient filtering and grouping, straightforward merging of datasets, flexible visualization, and reproducible statistical analysis.

In contrast to the original formulation of tidy data, the interpretation used here is intentionally less restrictive. Data from different cell types, strains, experimental conditions, replicates, or even measurement techniques can be stored in the same table, as long as their relationship to each observation is made explicit through appropriate variables and metadata. This flexibility is particularly useful for biological datasets, which often integrate multiple experimental modalities and conditions. Crucially, for this to work, sufficient information must be provided to distinguish experiments, conditions, and measurements unambiguously (rule three). For example, when integrating measurements from different genes, a gene identifier must be included as a column.

In practice, tidy datasets should therefore always be accompanied by a brief README or data dictionary file that documents the meaning, units, and interpretation of each column. This ensures that data remain interpretable not only by computational tools, but also by other researchers and by your future self.

i Info: Tidy data as good default not strict requirement

Tidy data is a powerful and flexible default for many analyses, but it is not the only data format in practice.

For very large datasets, tidy representations can become storage- and memory-intensive, making alternative formats more practical. In addition, many scientific fields have developed domain-specific data standards and file formats optimized for particular measurement techniques. Tidy data should therefore be viewed as a general organizing principle rather than a universal requirement.

Next steps: Explore Problem Sets 1 and 2 to become familiar with tidy data and its practical advantages.

Part II

Week 2: Biological data

4 Biological Data: Origin, Variability, and Rigor

Before analyzing a specific biological dataset, it is important to consider the origin of the data. Biological data do not simply exist; they are generated through biological processes, experimental design choices, measurement instruments, protocols, and human decisions. As a result, the biological signals of interest in a dataset are always entangled with other biological processes, as well as with technical and procedural effects (Fig. 4.1).

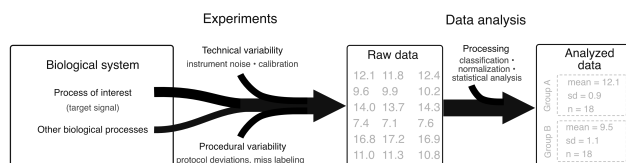


Figure 4.1: **Sources of variability across the experimental-analysis pipeline.** Numbers in biological datasets reflect the combined influence of the biological process of interest, other biological processes, as well as technical, procedural, and data processing steps. Quantitative analysis can characterize variability to partially entangle these signaling sources, but it cannot recover information that was never measured or correct biases introduced upstream.

Disentangling these contributions is a central task of quantitative data analysis and essential to gain meaningful biological insight.

To approach this disentanglement problem, it is useful to distinguish different sources of variability that jointly shape observed data:

! Definition: Different sources of variability shaping numbers in datasets

1. **Technical variability:** Variability introduced by measurement instruments and detection processes, such as detector noise, calibration uncertainty, or instrument drift.
2. **Procedural variability:** Variability arising from experimental execution and laboratory practice, including batch effects, reagent changes, or undocumented protocol deviations.
3. **Biological variability:** Intrinsic variability within and between cells, organisms, or populations. This includes variability arising from the biological process of interest as well as from other biological processes.

While technical, procedural, and unwanted biological variability are unavoidable, careful experimental design and measurement practices can limit their impact, increasing the extent to which observed data reflect the biological process of interest.

i Info: The semantic use of variability and noise.

Unwanted variability is often referred to as *noise*. However, it is important to recognize that this term is used in several contexts: to describe random measurement fluctuations, stochastic biological processes (e.g. gene expression noise), or more loosely anything that obscures a biological signal of interest.

Often, noise is implicitly treated as random and unbiased, in which case its impact can be reduced through replication and appropriate statistical analysis. However, not all unwanted variability is random. Some sources introduce *systematic bias*, which shifts measurements in a consistent direction and cannot be eliminated by increasing sample size alone. For example, calibration errors can lead to systematic misestimations that persist across replicates.

Finally, every biological measurement reflects the combined influence of multiple biological and technical processes,

not just the process of interest. For these reasons, we use *variability* as the more general term, encompassing both random fluctuations and systematic effects.

This leads to a simple but fundamental realization: **good data analysis requires good data**. Statistical methods can quantify uncertainty and variability in data, as we will discuss in later sections on statistics and uncertainty, but they cannot compensate for bad data generation and recover information that was not appropriately measured.

How to obtain high-quality biological data is beyond the scope of this class. However, it is important to recognize that a wide range of complementary strategies are commonly used in biological research to manage variability and improve data quality. These strategies have often been developed by groups and entire research communities over decades.

 **Example:** Common strategies to improve biological data generation

Obtaining high-quality data requires coordinated choices across experimental design, measurement, data processing, and data analysis.

- **Experimental design:** replication, randomization, appropriate controls
- **Measurement:** calibration procedures, spike-ins, reference samples
- **Data processing:** normalization, filtering, background correction
- **Data analysis:** batch correction; models that explicitly account for structured variability (e.g. mixed or hierarchical models)
- **Documentation:** metadata collection, protocol versioning, sample annotation

i Info: Analyzing data from others

Rigorous quantitative analysis requires a working understanding of how data were generated, processed, and documented. This can become challenging when analyzing data generated by other researchers, including colleagues, collaborators, or large consortia, where many experimental decisions are not directly visible in the dataset itself.

For published datasets, a careful reading of the methods section in a paper is often a crucial starting point. Key aspects to consider include how samples were collected and measured, what constitutes biological versus technical replicates, and whether randomization, or other sample selection strategies were used. Details on preprocessing steps, such as normalization, filtering, or background correction, are particularly important to correctly analyze data from others, as are clear definitions of units and detailed experimental conditions used. Methods sections also contain critical information about data availability, metadata, and prior statistical analyses.

4.1 The importance of rigor and reproducibility

To emphasize the importance and challenges of good data generation, let us briefly discuss more broadly rigor and reproducibility.

Rigor and reproducibility are the key elements of good scientific practice. They enable the evaluation of competing hypotheses, support the accumulation of reliable knowledge, and allow scientific results to be meaningfully compared, extended, and reused. In practice, however, high standards of rigor and reproducibility are not always met. While cases of deliberate scientific misconduct and fraud regularly ripple through the media and damage the reputation of science, they represent only a small fraction of a larger reproducibility problem. Far more common are unintentional failures of reproducibility arising from incomplete

documentation, missing metadata, unclear statistical reasoning, or unrecognized sources of systematic variability.

Example cell line contamination: A well-known example from cell biology is the widespread use of misidentified or contaminated cell lines, often referred to as the “HeLa crisis.” Beginning in the 1960s, it became clear that HeLa cells had contaminated many other human cell cultures. Subsequent systematic investigations revealed that a substantial fraction of widely used cell lines were misidentified or contaminated, leading to systematic errors that propagated through the literature. Although authentication practices have improved substantially since the 1960s, such issues continue to occur.

More generally, failures of reproducibility often arise from missing verification steps, analytic flexibility, or insufficient standardization.

Example

Recurring challenges for reproducibility in biological research

- **Cell line misidentification and contamination**
Insufficient verification of biological materials led to experiments being performed on unintended biological systems, with systematic errors propagating across many studies.
- **Reagent variability (the “antibody problem”)**
Differences in specificity or affinity between antibody lots, combined with poor documentation, have repeatedly compromised reproducibility across laboratories.
- **Unclear biological replication (pseudoreplication)** Treating multiple measurements from the same biological sample (e.g. cells from one animal, images from one culture) as independent replicates has inflated statistical support in many studies.
- **Undocumented analytic flexibility** In complex biological analyses, multiple reasonable preprocess-

ing and analysis choices exist. When these choices are not transparently reported, results may depend strongly on the chosen analysis path, limiting reproducibility and validation.

Such failures do not merely invalidate individual studies; they can mislead entire subfields and result in substantial loss of time, resources, and scientific opportunity. Addressing these challenges therefore requires systematic attention to rigor, transparency, and reproducibility throughout the research process.

Encouragingly, awareness of these issues has increased substantially.

4.2 The importance of dataset comparisons

As underscored by the cell line contamination example, even careful adherence to established standards of rigor and reproducibility does not guarantee that all relevant sources of variability or bias are known or under proper control.

This raises a fundamental question: how can we determine whether a dataset is good enough to analyze, and what can we reasonably learn from it? Detailed knowledge of experimental context, documentation, and known sources of variability is essential for assessing whether data were generated rigorously. However, this information might not be sufficient.

At first glance, this may sound discouraging. Yet these limitations also point toward a more constructive perspective on scientific progress. Robust biological insight rarely emerges from a single dataset in isolation. Instead, it arises when patterns recur across independent experiments, laboratories, time points, and experimental approaches. Replication at the level of entire studies—and convergence across diverse methodologies—provides strong evidence that an observed pattern reflects a genuine biological phenomenon rather than an artifact of a particular system or analysis.

From a quantitative perspective, comparing and integrating data across studies is key. Meta-analyses, cross-study comparisons,

and the integration of datasets generated at different times, in different laboratories, or using different experimental approaches allow reproducible patterns to emerge that are often invisible in individual experiments. When done carefully, such analyses turn variability itself into a source of information—helping to distinguish robust biological structure from context-dependent or technical effects.

5 Data Visualization

In the previous sections, we emphasized that a good quantitative analysis requires carefully generated data as a starting point. Visualization is commonly the next step in this process. Within the triangle of quantitative biology, visualization acts as a bridge. It allows us to explore structure in data, assess assumptions, generate hypotheses, and evaluate whether proposed models capture observed patterns. Visualization is also essential to communicate scientific results.

Importantly, visualization is not a passive presentation step. Choices made during visualization shape how data are perceived, which patterns stand out, and which sources of variability are emphasized or obscured. Well-designed visualizations reveal structure and guide interpretation, while poorly designed figures can hide important features or even mislead.

5.1 Visualization as a transformation

When we analyze data visually, we do not directly perceive numbers underlying a plot. Instead, values are encoded as graphical elements (such as positions, lengths, colors, or shapes) which are then interpreted by our visual system. Human vision is particularly effective at detecting patterns, trends, and relative differences, yet comparatively poor at precise quantitative comparison. As a result, visualization is not a neutral mirror of the data: the impression we form depends not only on the underlying biological and technical variability, but also on how the data are visually encoded.

Seen in this context, visualization adds an additional layer of transformation between biological processes and scientific interpretation (Fig. 5.1). Just as experimental design, measurement,

and data processing determine which sources of variability enter a dataset, visualization choices determine which aspects of that variability are emphasized, suppressed, or obscured. Some encodings highlight trends while downplaying dispersion; others can make noise appear as real trends or hide systematic bias.

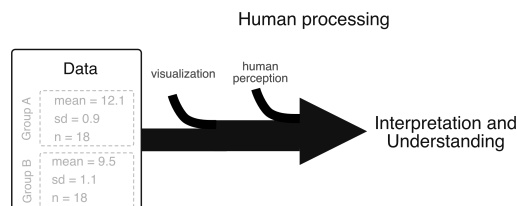


Figure 5.1: **Visualization shapes our perception of data and biological interpretation.** The numerical values in data underlying a plot are transformed through visualization choices and subsequently processed by the human visual system. As a result, scientific interpretation depends not only on the underlying data, but also on how data are visually encoded and perceived. Visualization therefore constitutes an additional transformation step that affects our perception of data.

5.2 Color perception and quantitative meaning

Color provides a clear example to illustrate how perception shapes quantitative interpretation. Human vision does not respond linearly to physical properties of light. Instead, color and brightness are reconstructed by the brain from the combined responses of different photoreceptor cells. As a consequence, equal numerical changes in color values do not necessarily correspond to equal perceptual changes.

In many visualizations, especially heat maps and image-based plots, numerical values are mapped to color values. Importantly, the resulting perception of plots can strongly depend on this mapping. Many color maps exaggerate small variations, intro-

duce artificial boundaries, or obscure meaningful gradients in the data, even when the underlying values change smoothly.

Modern computer displays further complicate this picture. Colors are typically represented as combinations of red, green, and blue intensities, which are convenient for computers but not designed to ensure perceptually uniform interpretation. A careful color choice is therefore essential when color is used to encode quantitative information.

As a practical guideline, perceptually uniform colormaps, such as `viridis`, `cividis`, `plasma`, or `inferno` for continuous data, are preferred. These maps preserve perceptual ordering and avoid artificial visual boundaries.

These perceptual differences become particularly apparent when visualizing the same dataset with different color maps. Identical data can appear smooth or segmented, uniform or highly structured, depending solely on the chosen color scale. Figure 5.2 illustrates this effect by showing the same data visualized with different colormaps.

5.3 Beyond color: other perceptual constructions

Color is only one way in which visualization interacts with perception. Other design choices can similarly shape interpretation. Three-dimensional plots are a prominent example. While visually appealing, 3D plots introduce depth cues, such as perspective, occlusion, and shading, that our brain automatically interprets. These cues often distort quantitative relationships, making it difficult to accurately compare values or assess distances along different axes.

In most cases, 3D plots obscure rather than clarify quantitative structure, particularly when used to display inherently two-dimensional data. Differences in viewing angle can change the apparent relationships between data points, and important features may be hidden behind others. For these reasons, 3D visualizations are rarely appropriate for quantitative analysis and

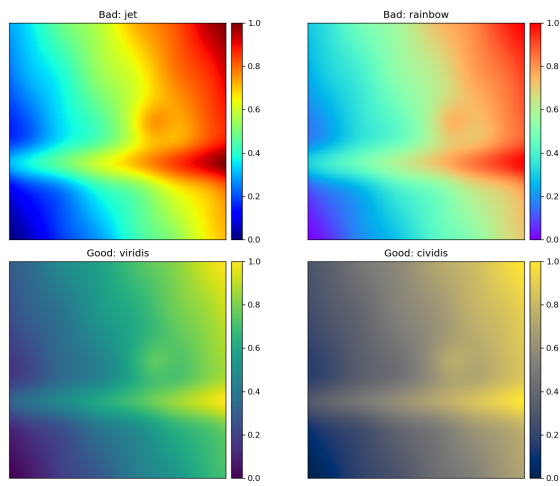


Figure 5.2: **Perceptual effects of colormap choice.** The same underlying data visualized with different colormaps can lead to very different visual impressions. Commonly used colormaps such as `jet` and `rainbow` introduce perceptual nonuniformities and artificial boundaries that can exaggerate or obscure structure. Perceptually uniform colormaps (e.g. `viridis` and `cividis`) preserve quantitative ordering and support more reliable interpretation of continuous data.

should generally be avoided in favor of simpler, two-dimensional representations.

Another common challenge arises in scatter plots when the number of data points is large. Overplotting can cause many points to overlap, making dense regions appear artificially sparse and hiding the true distribution of the data. In such cases, the apparent “noise” in a plot may reflect visualization choices rather than properties of the data themselves. Transparency, jittering, aggregation, or alternative representations are often necessary to accurately convey data density and variability.

More generally, visual parameters such as marker size, line width, smoothing, and plot dimensions influence which features of the data appear salient. While these tools can be used responsibly to improve readability, they can also unintentionally downplay variability or exaggerate structure. Thoughtful visualization design therefore requires awareness of how these choices affect perception.

5.4 Guiding principles for effective visualization

Taken together, these considerations lead to several general guidelines for good scientific visualization.

Best practice: Guidelines for effective plotting

Good plotting practices include:

- Show the data directly whenever possible, rather than relying solely on summaries or interpolations.
- Prefer clarity over decoration, and choose visual encodings (position, color, size, shape) intentionally.
- Provide clear axis labels, units, legends, and annotations to support quantitative interpretation.
- Use color schemes that preserve perceptual ordering and quantitative structure.

- Explicitly represent variability and uncertainty (e.g. distributions, confidence intervals, error bars), not just point estimates.
- Use consistent scales, colors, and encodings across related figures to facilitate direct comparison.
- Choose plot sizes and layouts appropriate for the intended medium (screen, print, presentation).

Conversely, several common practices should generally be avoided:

- Avoid truncated axes that exaggerate differences.
- Avoid non-perceptual or misleading color scales.
- Avoid bar plots for continuous data when the underlying distribution is of interest.
- Prevent overplotting that hides data density or structure.
- Avoid misleading smoothing, interpolation, or over-fitted trend lines.
- Avoid three-dimensional plots for quantitative data analysis.

Overall, effective visualizations are simple, familiar, and transparent in how they encode data, allowing the reader to focus on the underlying patterns rather than on the graphical design itself.

For further reading we recommend the good summary article by Rougier et al “Ten Simple Rules for Better Figures” available at <https://pmc.ncbi.nlm.nih.gov/articles/PMC4161295/>.

6 Datasets in this course

In this course, we will analyze several real datasets to illustrate common quantitative approaches in biology.

We will use a relatively compact “reference dataset” on bacterial growth to illustrate how to iterate through different analysis steps across the next five weeks, from data handling and initial data visualization to model fitting and hypothesis testing. We’ll also include some exercises in which you can choose to use other datasets.

Additional datasets will showcase different omics methods and allow data exploration across diverse biological systems.

6.1 Reference Dataset: Microbial growth analysis by Smith *et al.*

Our primary reference dataset is a collection of microbial growth curves from a recent high-throughput experiment by Smith *et al.* (2024). We chose this dataset because it is rich enough to support multiple quantitative analyses while remaining conceptually and computationally accessible early in the course.

6.1.1 Biological context

Human activity has introduced chemical pollutants into many ecosystems, for example through agriculture, industry, and medicine. These pollutants, alone or in combination, can strongly affect microbial populations, which in turn can alter entire ecosystems and even geochemical cycles.

To begin understanding the potential impact of pollutants on microbes, it is essential to investigate microbial growth. Growth

Table 6.1: **Experimental design of the Smith *et al.* bacterial growth study.** The experiments systematically combined eight chemical stressors representing common anthropogenic pollutants with a diverse set of bacterial isolates. Growth was measured across chemical combinations and strains.

Table 6.2: Chemical stressors

Chemical	Env.
Amoxicillin	Urban
Chlorothalonil	Rural
Diﬂufenican	Rural
Glyphosate	Urban
Imidacloprid	Urban
Metaldehyde	Rural
Oxytetracycline	Rural
Tebuconazole	Urban

Table 6.3: Bacterial strains

ID	Species	Phylum / Class
74	<i>Neobacillus soli</i>	Firmicutes
A001	<i>Pseudomonas baetica</i>	Gammaproteobacteria
B002	<i>Flavobacterium glaciei</i>	Bacteroidetes
H006	<i>Arthrobacter humicola</i>	Actinobacteria
H011	<i>Pseudomonas baetica</i>	Gammaproteobacteria
B71	<i>Rhizobium herbae</i>	Alphaproteobacteria
H19	<i>Sphingomonas faeni</i>	Alphaproteobacteria
A48	<i>Cornobacterium gallinarum</i>	Firmicutes
H87	<i>Acromonas popoffii</i>	Gammaproteobacteria
527	<i>Arthrobacter humicola</i>	Actinobacteria
—	<i>Escherichia coli</i>	Gammaproteobacteria
—	<i>Aliivibrio fischeri</i>	Gammaproteobacteria
Mix	Environmental mixture	Multiple phyla

is a hallmark of microbial life: how fast microbes grow determines how populations expand, compete, and persist in natural environments. Because microbes in real environments are typically exposed to multiple pollutants simultaneously, Smith *et al.* measured the growth of different bacterial species under a wide range of chemical conditions. In their study, different bacterial strains were exposed to eight single chemicals commonly encountered in freshwater environments (Table 6.1), as well as their combinations, resulting in 255 distinct chemical mixtures plus control conditions.

6.1.2 Measuring bacterial growth

To quantify how fast bacterial populations grow, changes in population biomass must be tracked over time. In practice, these growth curves are commonly measured using light absorbance or optical density (OD). Measured at a specific wavelength (often OD600), OD is a commonly used proxy for bacterial biomass in liquid culture. A few typical growth curves are shown in Figure 6.1.

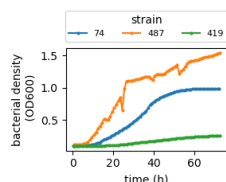


Figure 6.1: **Bacterial growth curves.** Three growth curve examples from Smith *et al.* showing change of bacterial abundance (OD600) over time.

Such curves exhibit characteristic phases: an initial phase of slow growth (lag phase), a phase of rapid growth (exponential or log phase), and a phase where optical density no longer increases (stationary phase). The growth rate quantifying the speed of growth can be extracted by analyzing how steep the growth curve is during exponential growth. We will return to this analysis in detail next week, when we discuss model fitting and uncertainty.

To obtain these growth curves for many culture conditions, Smith *et al.* used plate-reader experiments, where many culture conditions can be measured in parallel using 96-well plates and optical density can be recorded automatically over time.

6.1.3 Organization and structure of the dataset

Following good practices in transparent science, Smith *et al.* made their raw and processed data publicly available via a dedicated GitHub repository (<https://github.com/smithhttp/isolate-chem-mixtures>) .

In this course, we will focus on the raw growth curve data. We will extract growth rates, analyze uncertainty across replicates, and test whether growth rates differ significantly across chemical stressor conditions.

The growth curve data are provided in a single combined file. To work effectively with this dataset, it is important to understand its organization. Each growth curve is indexed by (i) the bacterial strain, (ii) the specific combination of chemical stressors, and (iii) the replicate number (alternatively strain, replication, plate, and well numbers also uniquely define growth curves). Together, these identifiers uniquely define each observation. See Dry Labs 1 and 2 for further discussion.

Info

This example from microbial ecology has broader relevance than one might first think. First, time series analysis and the quantification of variability across strains, conditions, and replicates arise in many biological contexts, making this dataset analysis broadly transferable. In addition, growth is fundamentally intertwined with gene expression and proteome composition, themes we will revisit later in the course.

6.2 Other datasets

In addition to the reference growth dataset, we will look in this course at different omics datasets from recent studies.

6.2.1 Change in proteome composition during synapse development

Summary:

As a first example, we include a large-scale proteomics dataset describing how the molecular composition of synapses changes during brain development. The dataset was published in a recent study by Wang et al (<https://www.nature.com/articles/s41564-024-01626-9>).

The study quantified the abundance of more than 1,000 synaptic proteins across developmental stages in humans and two other species. This dataset exemplifies the power of modern quantitative analyses in neurobiology.

Biological context:

Synapses are the fundamental units of communication between neurons, and their molecular composition changes substantially during brain development. To study these changes, the dataset provided here quantifies protein abundance in the *postsynaptic density* (PSD), a specialized protein complex at excitatory synapses.

The study used mass spectrometry to measure PSD protein abundance across developmental stages in human, macaque, and mouse. In humans, samples were collected from two cortical regions: the prefrontal cortex (PFC), an association cortex region that matures relatively late, and the primary visual cortex (V1), a sensory region that matures earlier. Macaque and mouse datasets provide complementary cross-species comparisons of synapse development.

Organization of the dataset:

For each sample group (human PFC, human V1, macaque, and mouse), the original study provides a table in which rows correspond to proteins (identified by UniProt ID, gene name, and Entrez ID) and columns show different biological samples. A separate metadata table links each sample to biological information such as developmental age or age group, cortical region, and sex.

To facilitate data exploration and analysis, we provide a reformatted tidy version of the data. In this tidy table, each row represents a single measurement: the abundance of one protein in one sample. Columns encode protein identifiers, sample identifiers, species, cortical region, developmental stage, and measured protein abundance. In addition, a separate sample-metadata table contains one row per sample and describes the biological context associated with each measurement.

6.2.2 Changes in cellular proteome composition across growth conditions

Summary:

As a second example, we provide a large-scale proteomics dataset describing how the protein composition of *Escherichia coli* changes across different growth conditions. The dataset, published in a recent study by Chure et al (<https://www.nature.com/articles/s41467-025-67553-3>) combines protein abundance measurements from many independent studies and labs into a unified, curated format. It illustrates why growth (and more generally the metabolic state of cells) is such a central control variable in cellular physiology and why it should be considered explicitly when studying cellular processes. The dataset also exemplifies the value of systematic cross-study comparisons to judge reproducibility.

Biological context:

Cell growth requires the coordinated allocation of cellular resources to make new biomass, including proteins involved in metabolism, biosynthesis, transport, and regulation. Changes in environmental conditions such as nutrient availability can strongly alter this coordination, affecting proteome composition and growth rate. Understanding how cells reorganize their proteome across conditions is therefore a central question in cell physiology and a prerequisite for systems-level insights into how cells function.

Proteome reorganization has been studied extensively in model organisms including *Escherichia coli*. Here, we provide an *E. coli* dataset derived from a recent study that quantified the condition-dependent partitioning of proteins across cellular compartments, including cytoplasmic, membrane-associated, and periplasmic proteins. Beyond compartmentalization, the dataset can be used more broadly to explore relationships between growth rate, protein allocation, and environmental variation. Because the dataset integrates proteome measurements from many different laboratories, it also enables systematic cross-laboratory comparisons to assess reproducibility.

Organization of the dataset:

The data focus on steady growth on different carbon sources. Measurements from different studies were converted into a joint tidy format, allowing direct comparison across experiments. The tidy data table contains one row per protein measurement in one experimental condition. Columns encode the bacterial strain, carbon source, replicate number, protein identifier (gene or protein name), study source, cellular localization, functional annotation, growth rate, and measured protein abundance (mass fraction). Each observation is uniquely identified by the combination of strain, carbon source, replicate, protein identity, and data source.

6.2.3 Transcriptional dynamics during the yeast metabolic cycle

Summary:

As a third example, we include an RNA-seq dataset describing transcriptional dynamics during the yeast metabolic cycle. The dataset collected by Gowans et al. ([https://www.cell.com/molecular-cell/abstract/S1097-2765\(19\)30734-8](https://www.cell.com/molecular-cell/abstract/S1097-2765(19)30734-8)) captures genome-wide gene expression changes over time in both wild-type budding yeast *Saccharomyces cerevisiae* and a targeted regulatory mutant. It provides a time-resolved view of how cellular metabolism, chromatin regulation, and gene expression are inherently coupled.

Biological context:

The yeast metabolic cycle is a well-studied oscillatory program in which cellular metabolism alternates between distinct phases characterized by different oxygen consumption rates, redox states, and biosynthetic activities. These metabolic oscillations are tightly coupled to transcriptional regulation: thousands of genes are periodically activated or repressed in synchrony with metabolic state.

Understanding how gene expression dynamics are coordinated with metabolic cycles is central to systems biology, as it reveals how cells temporally organize energy production, biosynthesis, and growth.

In this study, the authors used RNA sequencing to measure genome-wide transcriptional changes across successive phases of the metabolic cycle in wild-type yeast and in a mutant carrying a point mutation in the chromatin-associated transcription factor *TAF14* (W81A). Taf14 contains a YEATS domain that binds histone crotonylation, a chromatin modification linked to metabolic flux. The W81A mutation disrupts this interaction, thereby uncoupling metabolic state from chromatin-mediated transcriptional repression.

By comparing wild-type and *TAF14* W81A cells, the study tests whether transcriptional oscillations arise directly from metabolic state or whether they depend on chromatin-based regulatory mechanisms that sense metabolic flux.

Organization of the dataset:

The dataset is available as processed RNA-seq tables deposited in the Gene Expression Omnibus (GEO, accession GSE120019), containing gene expression measurements across multiple time points during the yeast metabolic cycle.

The tidy data table we provide contains one row per gene, per time point, and per genotype. Columns encode the systematic gene identifier (and name when available), time point within the metabolic cycle, genotype (wild-type or *TAF14* W81A), and normalized RNA abundance. Additional columns report statistical results from the original differential expression analysis, including log-fold changes and adjusted p-values.

Each observation is uniquely identified by the combination of gene identifier, time point, and genotype. This structure makes it easy to explore temporal expression patterns, compare wild-type and mutant dynamics, and link transcriptional changes to metabolic state.

6.2.4 Single-cell transcriptomics in yeast

Summary:

As a fourth example, we include a single-cell RNA-seq dataset from the budding yeast *Saccharomyces cerevisiae*, generated by Jackson *et al.* (eLife, 2020). This dataset contrasts bulk

RNA-seq measurements with single-cell transcriptomic profiles collected under different environmental conditions. It illustrates how high-dimensional molecular measurements at single-cell resolution reveal heterogeneity that is invisible in population-averaged data.

Biological context:

Bulk transcriptomic measurements report the average gene expression across many cells, implicitly assuming that the population is homogeneous. In reality, even genetically identical cells growing in the same environment can exhibit substantial cell-to-cell variability in gene expression. This variability can reflect stochastic gene expression, differences in metabolic state, cell-cycle position, or adaptive responses to environmental stress.

In this study, the authors used single-cell RNA sequencing to measure genome-wide gene expression in individual yeast cells grown under three conditions: rich medium (YPD), carbon starvation (CStarve), and growth on proline as the sole nitrogen source. These conditions induce distinct physiological states and levels of transcriptional heterogeneity. By comparing bulk and single-cell measurements, the dataset highlights how population averages can hide variability present at the single-cell level.

Organization of the dataset:

The dataset provided in this course consists of two complementary parts. First, bulk RNA-seq data were collected from actively growing yeast cultures in rich medium (YPD), with five biological replicates. In this table, each row corresponds to a replicate and each column corresponds to a gene, representing population-averaged expression levels.

Second, single-cell RNA-seq data were collected from individual yeast cells across the three growth conditions (YPD, CStarve, and Proline). In the single-cell table, each row represents one cell and each column represents a gene. Additional metadata columns encode experimental condition, batch information, and cell-level quality metrics.

We provide the original data tables published. Note that this is not a tidy data format as such a formatting would lead to unpleasantly large file sizes.

Part III

Week 3: Fitting and regression

7 Fitting and the importance of Regression

Once data have been visualized and explored, the next step is to move beyond descriptive plots and toward quantitative descriptions of patterns and relationships. This commonly involves fitting a function to data in order to summarize trends, extract parameters, or enable comparisons across conditions.

At a basic level, fitting refers to the process of relating observed data to a mathematical function. However, it is important to realize that the term fitting is often used very broadly, a bit of a semantic mess. An important conceptual distinction is between *interpolation*, *smoothing*, and *regression*.

7.1 Interpolation

Interpolation aims to describe the data between observed points by constructing a curve that passes exactly through all measurements. In this setting, the data points themselves are treated as if they were exact. Interpolation is therefore purely descriptive and is often the default behavior in plotting packages like Python's Matplotlib, where lines are drawn directly between data points to guide the eye.

While this can be visually appealing, it is rarely appropriate for biological data, which are inherently noisy. Interpolated curves can give a misleading impression of precision and may obscure the underlying variability in the measurements.

As an example, Fig. 7.1A shows the interpolation to the original enzyme activity data from Michaelis and Menten. Drawing an interpolated curve through all data points suggests a well-defined relationship between substrate concentration and activity, but

such a curve does not provide quantitative biological information, such as the Michaelis–Menten constant K_M .

7.2 Smoothing

Smoothing is also descriptive, but unlike interpolation it deliberately allows deviations from individual data points in order to reduce the influence of noise and reveal broader trends. Examples include moving averages or locally weighted smoothing methods.

In biological contexts, smoothing can be useful for visualizing overall behavior in noisy time-series data, such as growth curves, fluorescence traces, or sensor measurements. It can also be useful for identifying boundaries in image data. However, smoothing introduces subjective choices (such as window size or kernel shape) that affect the resulting curve. And like interpolation, smoothing does not by itself provide a framework for parameter estimation or evaluating a specific model or relationship between data.

An example of smoothing applied to the original Michaelis–Menten data and using moving averages is shown in Fig. 7.1B.

7.3 Regression

Regression, in contrast, treats observed data as noisy realizations of an underlying relationship and assumes a specific functional form. It explicitly acknowledges variability and seeks to estimate the parameters of that function such that it best describes the data.

Returning to the Michaelis–Menten example, a regression approach allows us to estimate the parameters K_M and $v_{\max}$ of the Michaelis–Menten relation

$$v = v_{\max} \frac{S}{S + K_M},$$

where $v_{\max}$ denotes the maximal reaction rate and K_M the substrate concentration at half-maximal rate, biologically important quantities. For the original data from Michaelis and Menten, the regression to this function is shown in Fig. 7.1C.

7.4 The importance of regression

Notably, regression provides not only point estimates of biologically meaningful parameters. Systematic regression also allows comparison across conditions and quantification of trends. For example, regression is the starting point to rigorously answer whether one enzyme operates faster than another, or whether cell growth across two conditions differ systematically. Regression is also crucial in many experimental workflows. Calibration curves, background correction, and baseline subtraction all rely on fitting specific functions to data in order to convert raw measurements into biologically meaningful quantities. In summary, regression is essential for quantitative biological data analysis. We next introduce the basic ideas underlying regression algorithms.

i Info: Semantic confusion!

The term originates from the historical concept of “regression to the mean,” but in modern data analysis it refers more generally to modeling relationships between variables in the presence of noise, as introduced above.

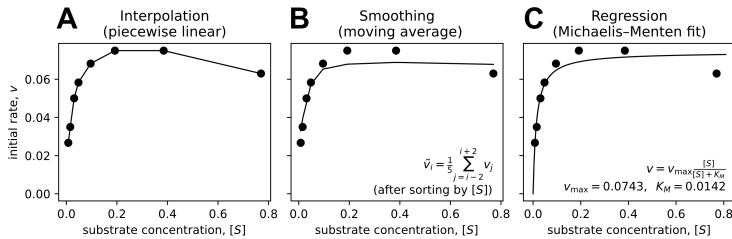


Figure 7.1: **Illustration of different fitting types using the original data by Michaelis and Menten on enzyme kinetics.** (A) Interpolation: a curve is forced to pass exactly through all data points by connecting neighboring measurements, treating measurements as exact. (B) Smoothing: a descriptive curve obtained by averaging neighboring data points (moving average after ordering by substrate concentration), allowing deviations from individual measurements to reduce noise and reveal broader trends. (C) Regression: a biologically motivated function (Michaelis–Menten relation) is fitted to the data by estimating parameters while explicitly accounting for variability.

8 Linear regression

While biological systems are generally nonlinear, linear regression is the most fundamental form of regression and underlies many methods in data analysis, including many machine-learning approaches. Linear regression appears frequently in biological data analysis as well, for example in calibration curves, growth-rate estimation, and dimensionality reduction of omics data.

Mathematically, linearity means that changes in input lead to proportional changes in output and that the effects of multiple inputs add together. In the simplest case (1D), the relationship between an input variable x and an output variable y is described by a line,

$$y = \beta_0 + \beta_1 x,$$

where β_0 is the intercept and β_1 is the slope (Fig. 8.1).

i Info: Input, output, predictors, and responses

Throughout this section, we refer to variables such as x as *predictors* (or input variables) and to y as the *response* (or output variable). Predictors are quantities used to explain or predict variation in the response. In biological contexts, predictors often represent experimentally controlled or observed inputs (e.g. substrate concentration, time, or treatment), while the response represents the measured outcome (e.g. reaction rate, growth rate, or expression level).

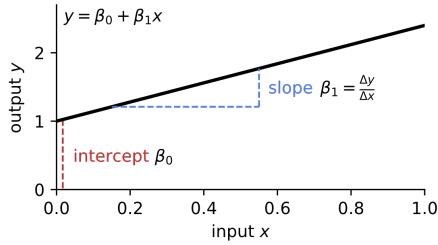


Figure 8.1: **A linear function and its parameters.** A one-dimensional linear model relates an input x to an output y through a line with an intercept β_0 and slope β_1 . The slope quantifies how much y changes per unit change in x ($\beta_1 = \Delta y / \Delta x$), and the intercept is the predicted value at $x = 0$.

8.1 Linear regression as an optimization problem

Assume we have a set of n data points, $\{(x_i, y_i)\}$, with i varying between 1 and n . Linear regression asks the following question: how do we choose the parameters β_0 and β_1 such that the line those parameters define best describes the data?

Regression turns this question into an optimization problem by introducing a *loss function* that quantifies the mismatch between the line predictions and the observed data. A commonly used choice is the ordinary least squares (OLS) loss, defined as

$$L(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2.$$

The goal of linear regression is to find the values of $\hat{\beta}_0$ and $\hat{\beta}_1$ that minimize this loss function. Intuitively, this corresponds to finding the line that minimizes the total squared vertical distance (δy) between the data points and the line.

Notably, this optimization problem can be solved analytically with basic concepts from differential calculus. We illustrate this in the following box for the intercept $\hat{\beta}_0$. The slope $\hat{\beta}_1$ can be derived similarly. For the following sections, it is most important to realize that the regression procedure has a well-defined

mathematical foundation. Furthermore, because analytical solutions to the optimization problem exist, linear regression can be readily applied to data.

i Info: Analytical solution for the intercept

To find the values β_0 and β_1 where the loss function is minimal, we need to identify the stationary point of L where the partial derivatives with respect to β_0 and β_1 vanish.

Here, we focus on β_0 .

We require

$$\left. \frac{\partial L}{\partial \beta_0} \right|_{\hat{\beta}_0} = 0.$$

Computing the derivative,

$$\frac{\partial L}{\partial \beta_0} = \frac{\partial}{\partial \beta_0} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 \quad (8.1)$$

$$= -2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i). \quad (8.2)$$

Setting this expression to zero at $\hat{\beta}_0$ yields

$$\hat{\beta}_0 = \bar{y} - \beta_1 \bar{x},$$

where $\bar{x}$ and $\bar{y}$ denote the sample means of x and y . Thus, we arrive at an equation stating how the optimal intercept depends on the provided data provided we know β_1 . With a similar approach for β_1 we get a full solution stating how both, optimal intercept and slope, depend on data points alone.

Linear regression is one of the few regression types in which closed-form analytical solutions exist. This property underlies its importance in statistics and data analysis.

8.2 Example: Michaelis–Menten data via a linearizing transformation

As a concrete biological example, we apply linear regression to the original enzyme kinetics data from Michaelis and Menten. As with many fundamental relations in biology, the relationship between substrate concentration and reaction rate is highly non-linear. However, it can be transformed into a linear relationship that can be fit with a straight line. Figure 8.2A shows the resulting linear fit, the inferred slope and intercept parameters, and the resulting parameters K_m and v_{max} of the Michaelis–Menten relation. Figure 8.2B shows the loss function for different slope and intercept parameters, confirming the identification of both parameters where the loss function is lowest.

i Info: Connection to Pearson correlation

Notably, a closely related quantity is the Pearson correlation coefficient, which measures the strength of linear association between two variables. If both x and y are standardized to have mean zero and variance one, then the slope $\hat{\beta}_1$ of the linear regression equals the Pearson correlation coefficient. Thus, correlation and linear regression capture closely related information, though they typically serve different analytical purposes.

To see this, let's consider the OLS slope of a linear regression of Y on X .

$$\begin{aligned}\hat{\beta}_1 &= \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} \\ &= \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}} \cdot \frac{\sqrt{\sum_i (y_i - \bar{y})^2}}{\sqrt{\sum_i (x_i - \bar{x})^2}} \\ &= p_{xy} \cdot \frac{s_y}{s_x},\end{aligned}$$

where p_{xy} is the Pearson correlation, and s_x and s_y are the standard deviation of x and y respectively. Therefore the slope can be rewritten as the Pearson correlation scaled

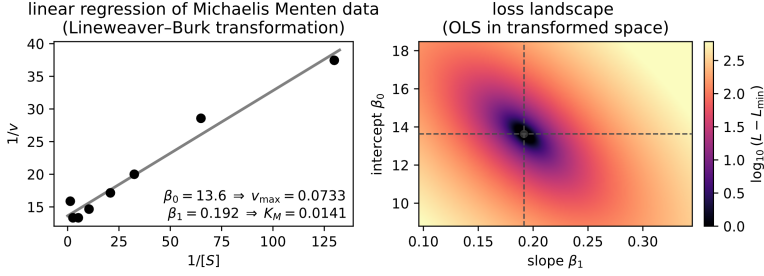


Figure 8.2: **Linear regression of the original Michaelis–Menten data using a Lineweaver–Burk transformation.** (A) Enzyme kinetics data are transformed by plotting the reciprocal reaction rate ($1/v$) versus the reciprocal substrate concentration ($1/[S]$). Under this transformation, the Michaelis–Menten relationship becomes linear and can be written as $\frac{1}{v} = \beta_0 + \beta_1 \frac{1}{[S]}$ with $\beta_0 = 1/v_{\max}$ and $\beta_1 = K_M/v_{\max}$. Applying ordinary least squares regression yields estimates for the intercept (β_0) and slope (β_1), which map to the original kinetic parameters of the Michaelis–Menten relation. (B) Loss landscape of the ordinary least squares objective in the transformed space as a function of β_0 and β_1 . The minimum identifies the regression parameters shown in (A), illustrating that linear regression corresponds to minimizing a well-defined loss function. This example demonstrates how nonlinear biological relationships can be analyzed using linear regression after transformation; in later sections, we revisit why such linearizations can distort error structure and motivate nonlinear regression on the original data.

by the standard deviation of the two variables.

8.3 Generalization to multiple predictors and multiple linear regression

The univariate linear regression discussed above, fitting a line to relate a single predictor variable to a response variable, is a special case of linear regression. Here, *univariate* refers to the fact that only one predictor is used. In practice, we often measure multiple predictor variables simultaneously and wish to relate them to a single response variable.

Linear regression naturally generalizes to this setting, commonly referred to as *multiple linear regression*, using vector and matrix notation.

i Info: Linear model with multiple predictors

Let

$$\mathbf{y} \in \mathbb{R}^n$$

denote the vector of observed responses,

$$\mathbf{X} \in \mathbb{R}^{n \times p}$$

the design matrix containing the predictor variables, and

$$\beta \in \mathbb{R}^p$$

the vector of regression coefficients. Each row of $\mathbf{X}$ corresponds to one observation, and each column corresponds to one predictor.

The linear regression model can then be written compactly as

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon,$$

where ε is a vector of residual errors capturing variability not explained by the linear relationship.

Despite involving multiple predictors, this model remains *linear in the parameters* β , which is the defining feature of linear regression. Univariate linear regression corresponds to the special case $p = 1$.

As in the univariate case, regression proceeds by minimizing the sum of squared residuals. The difference is that the prediction now combines contributions from multiple predictors:

$$L(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|^2.$$

Provided that the number of observations exceeds the number of parameters and that the columns of $\mathbf{X}$ are not linearly dependent, this optimization problem admits an analytical solution,

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

This expression highlights why linear algebra is central to linear regression. For high-dimensional biological datasets, such as omics data, advanced computational linear algebra methods are essential. In practice, tailored numerical algorithms are used rather than explicitly computing matrix inverses. These algorithms can be conveniently applied using different analysis packages, often designed for specific biological data types. We will return to these methods when discussing omics data analysis later in the course.

i Info: Different approaches to linear regression

Linear regression is one of the few regression settings in which closed-form analytical solutions exist.

This makes it computationally efficient and conceptually transparent. However, it is good to know that the ordinary least squares (OLS) loss used here is only one specific choice of loss function. Other loss functions are also used, for example to reduce sensitivity to outliers or to emphasize different aspects of the data. Furthermore, even for linear regression, practical implementations often rely on numerical linear algebra methods, especially for large or high-dimensional datasets.

9 Non-linear regression

9.1 Biology is inherently non-linear

Many biological processes involve **thresholds**, **saturation**, **cooperativity**, **feedback**, and **limited resources**. As a result, relationships in biological data are typically inherently non-linear, and it is important to go beyond linear regression.

Non-linearity means that changes in an input variable do not lead to proportional changes in the output. Combined effects do not simply add together, and small changes in conditions can produce large qualitative shifts in behavior. Mathematically, instead of straight lines (or planes), relationships are often described by curved, sigmoidal, or saturating functions. See the box for a few examples.

i Info: Examples of non-linear behavior in biology

1. **Enzyme kinetics.** Reaction rates saturate at high substrate concentrations because enzymes have finite catalytic capacity, as described by the non-linear Michaelis–Menten relation. Reaction rates also depend in complex, non-linear ways on temperature, pH, and intracellular salt concentrations.
2. **Microbial growth.** Growth is commonly exponential (not linear) under nutrient-replete conditions. Growth rates increase non-linearly with nutrient availability and saturate once uptake or metabolism becomes limiting.
3. **Gene regulation.** Transcriptional regulation and cellular signaling are highly non-linear processes, often exhibiting steep sigmoidal responses with strong

cooperativity, such that small changes in concentration can lead to large, almost switch-like changes in gene expression.

4. **Feedback-driven dynamics.** Positive and negative feedback loops can generate bistability or oscillations, as observed in metabolic regulation, circadian rhythms, and development. These behaviors have no linear analogue.

In summary, biological processes are often highly non-linear, with characteristics such as saturation, thresholds, and steep transitions that cannot be captured by linear models. Consequently, non-linear regression is an essential tool for quantitative biological analysis.

9.2 The idea of non-linear regression

How does non-linear regression work?

Conceptually, non-linear regression builds on the same logic as linear regression. Given a dataset (x_i, y_i) and a model

$$y = f(x; \phi),$$

we seek parameter values ϕ that best describe the data. Here, ϕ generally denotes a vector of multiple model parameters rather than a single coefficient.

As in the linear case, parameter estimation is formulated as an optimization problem. A common choice of loss function is the sum of squared residuals,

$$\mathcal{L}(\phi) = \sum_i (y_i - f(x_i; \phi))^2.$$

The crucial difference from linear regression is that f is now a genuinely non-linear function rather than a straight line. Notably, for non-linear functions there is typically no closed-form analytical solution for finding the parameter values that minimize the loss. Instead, non-linear regression relies on numerical optimization methods.

We do not discuss here in depth the many numerical algorithms that enable non-linear regression. Instead, we provide an intuitive picture of the optimization process using the concept of an *error landscape*. Each point in parameter space corresponds to a particular parametrization of the model, and the loss function assigns a height to each point, quantifying how well that parametrization describes the data. Performing regression then becomes the task of navigating this landscape to find a (potentially global) minimum.

Numerical optimization methods generally aim to find a minimum of a loss (or objective) function defined over a parameter landscape. A large and important class of these methods—iterative algorithms—operate by starting from an initial point in parameter space (often chosen heuristically or at random). At each iteration, the algorithm uses information about the local geometry of the loss function, such as its slope (the first derivative) and, in some cases, its curvature or concavity (the second derivative). Methods differ in how this information is obtained: when analytic expressions for derivatives are available, they may be used directly; otherwise, derivatives can be approximated numerically. Based on this local information, the algorithm updates the parameter values in a direction expected to reduce the loss and repeats this process until a convergence criterion is met (e.g., the change in parameters or loss becomes sufficiently small). Generally, if the slope is very large, a larger step is taken, and if the function is very curved a smaller step is taken.

Info: Gradient Descent

Gradient descent is one of the simplest and most widely used iterative optimization algorithms. At each iteration, the parameters are updated in the direction of the negative gradient of the loss function, which corresponds to the direction of steepest descent.

Let θ denote the parameter vector and $L(\theta)$ the loss function. The update rule is

$$\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k),$$

where $\eta > 0$ is the *learning rate*.

The learning rate determines the step size at each iteration. Large values may cause divergence, while small values can lead to slow convergence. Because gradient descent relies only on first-order derivative information, it is computationally efficient but may converge slowly in poorly scaled or highly curved landscapes.

Beyond basic gradient descent, many optimization algorithms modify how derivative information is used to improve convergence or robustness. Newton’s method and quasi-Newton methods (such as BFGS and L-BFGS) incorporate second-order curvature information, either exactly or approximately, to rescale parameter updates. “Conjugate gradient” methods are often used for large, sparse problems. In settings where derivatives are noisy or expensive, stochastic gradient descent and its variants (e.g., momentum, RMSProp, Adam) are common. Finally, derivative-free methods such as Nelder–Mead or evolutionary algorithms can be useful when gradients are unavailable or unreliable. Together, these methods form a toolbox whose appropriate choice depends on the structure of the problem, the cost of evaluating the loss, and the dimensionality of the parameter space.

9.3 Example: non-linear regression of enzyme kinetics

As an example of non-linear regression, we return to the enzyme kinetics data from Michaelis and Menten and now fit the original non-linear Michaelis–Menten model directly to the data. Unlike the linearized approach introduced earlier, this method preserves the original error structure of the measurements and yields parameter estimates that are less sensitive to distortions introduced by transformations.

Figure 9.1 shows the best-fit non-linear regression curve together with the experimental data.

Notably, the inferred parameters for K_m and $v_{\max}$ are here very similar to those obtained from the linearized regression approach. However, in general this agreement should not be

expected, and linearizing transformations can substantially bias parameter estimates.

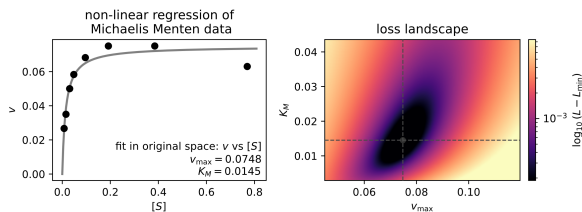


Figure 9.1: **Non-linear regression of Michaelis–Menten kinetics.** The original Michaelis–Menten model is fit directly to reaction rate data by numerical minimization of the squared residuals.

9.4 The challenges of non-linear regression

At first sight, finding a minimum in an error landscape may appear straightforward. However, error landscapes are often highly complex: they may be steep in some directions, flat in others, and can contain multiple local minima. As a result, identifying good solutions can be challenging.

Several practical difficulties commonly arise:

- The algorithm may converge to a local rather than a global minimum.
- Results can depend strongly on the initial parameter guesses.
- Step sizes matter: overly large steps may cause divergence, while very small steps lead to slow convergence.
- Flat regions of the landscape can substantially hinder progress.

These challenges become more pronounced as the number of fitted parameters increases. An intuitive picture based on a two-dimensional landscape can strongly underestimate the difficulty of optimization in higher-dimensional parameter spaces.

While modern numerical libraries implement sophisticated strategies to mitigate these issues, non-linear regression remains inherently more fragile than linear regression and requires careful evaluation of results. Choosing appropriate initial parameter values, checking convergence, and validating fits are essential steps in obtaining reliable results and often require substantial manual intervention and judgment.

See Problem Set 3 for hands-on examples of performing and evaluating non-linear regression using standard computational tools.

9.5 The importance of evaluation

Importantly, while non-linear regression is an essential and powerful tool, it also requires substantially more care than linear regression. Because non-linear regression uses numerical optimization, successful running of an algorithm alone does not guarantee that the resulting parameters are meaningful or reliable.

In practice, non-linear fitting should always be viewed as an *interpretive* step rather than a purely mechanical one alone. A fitted curve may visually appear in good agreement with the data but still reflect an inappropriate model structure, poorly constrained parameters, or numerical artifacts of the optimization procedure. For this reason, every non-linear fit must be evaluated critically.

At a minimum, one should assess whether the fitted model captures the major trends of interest in the data without introducing systematic deviations, whether inferred parameter values are biologically plausible, and whether the data actually constrain the parameters of interest (see box).

i Info: Practical guidance for evaluating non-linear fits

When interpreting a non-linear regression, it is good practice to ask:

- Does the fitted model systematically miss parts of

the data, or does it capture the overall trend?

- Do residuals appear random, or do they show structure such as curvature or changing variance?
- Are fitted parameters robust to changes in initial parameter guesses or fitting settings?
- Are parameter values biologically reasonable?
- Do many different parameter combinations explain the data nearly equally well?

In Problem Set 3, we will work through concrete examples of linear and non-linear regression in Python, including fitting biologically motivated models and visualizing error landscapes.

Part IV

Week 4: Probability and uncertainty

10 A probabilistic view and the law of large numbers

Following data visualization and fitting (regression), a common next step is to evaluate results and assess their reliability. This requires a quantitative analysis of uncertainty, which in turn requires statistical considerations.

10.1 Measurements as random variables

As discussed previously, biological data are inherently variable, reflecting a combination of biological heterogeneity, technical noise, and procedural effects (see Section 4). As a consequence, repeated measurements under identical experimental conditions do not produce identical numerical outcomes. We therefore need to adopt a *probabilistic perspective*.

Key to a probabilistic description is treating measurement results not as fixed values, but as *random variables* drawn from an underlying probability distribution. A single measurement provides one realization from this distribution, not a direct measurement of the true underlying biological quantity. Repeating the same experiment under identical conditions produces not a single number, but a distribution of values. It is this distribution that contains the information we seek, and statistical quantities such as means or standard deviations summarize properties of this distribution and our uncertainty about it.

! Definition: Random variables and probability distributions

A **random variable** is a quantity whose value can vary between repeated measurements, even when experimental

conditions are kept the same. We denote a random variable by a symbol such as X .

A **probability distribution** describes how likely different values of X are to occur. For a discrete variable, this is specified by probabilities $P(X = x)$; for a continuous variable, by a probability density $p(x)$. Probability distributions are normalized such that the total probability equals one.

To illustrate this probabilistic perspective, consider a coin-toss experiment. Suppose a coin lands heads with probability p and tails with probability $1 - p$. A single coin toss produces one outcome—heads or tails—but this single observation does not tell us much about the underlying process (for example, how large p is).

In technical terms, a single toss is one sample from the underlying probability distribution, but it is insufficient to characterize that distribution. By repeating the coin toss many times, we obtain a collection of outcomes whose relative frequencies allow us to estimate the underlying probability p . This illustrates a general principle: repeated measurements allow us to learn about the properties of an underlying probability distribution.

i Info: Coin toss and Bernoulli process

The coin-toss experiment is one of the few cases where the probability distribution and its statistical properties can be treated fully analytically. Each toss follows a Bernoulli process. Most biological experiments are more complex: the underlying distributions are rarely known exactly and are often analytically intractable. Accordingly, numerical approaches play a central role in biological data analysis, including for the coin-toss example discussed here.

10.2 The law of large numbers

A particularly important property of a probability distribution is its mean, also called the expected value. The mean provides

a central tendency of the distribution and is often the quantity of biological interest.

! Definition: Mean

The **mean** (or average) of a random variable describes the central value of its underlying probability distribution.

For a *discrete* random variable X that takes values x with probabilities $P(X = x)$, the mean is defined as

$$\mu = \sum_x x P(X = x).$$

For repeated measurements $X_1, X_2, \dots, X_n$, the corresponding **sample mean** is

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i.$$

For the coin-toss example, we can assign the value $X = 1$ to heads and $X = 0$ to tails. The sample mean of many coin tosses then corresponds to the fraction of heads observed (Fig. 10.1A). Importantly, as the number of coin tosses increases, the sample mean stabilizes and approaches the true probability p of obtaining heads. This behavior is illustrated in Fig. 10.1B using computational simulations of repeated coin tosses. Notably, the convergence occurs despite each individual outcome remaining random. This fundamental behavior is known as the *law of large numbers*.

! Definition: Law of large numbers

For independent and identically distributed random variables with a finite expected value, the sample mean converges to the true expected value as the number of observations increases.

The importance of the law of large numbers is hard to overstate. It provides the theoretical foundation for estimating average biological quantities through repeated measurements and explains why increasing sample size improves the reliability of experimental results.

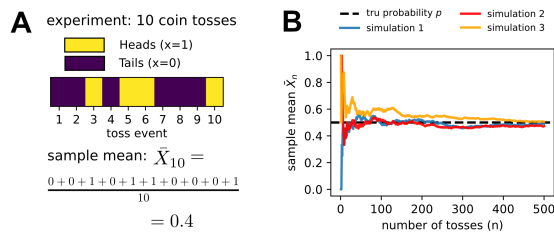


Figure 10.1: **The law of large numbers during repeated tossing of a fair coin.** (A) A sequence of coin tosses and the corresponding sample mean. (B) The running sample mean converges toward the true probability $p = 0.5$ as the number of trials increases. Despite individual outcomes remaining random, the sample mean becomes increasingly stable. Different lines represent independent simulations of repeated coin tosses. Assumes a fair coin with $p = 0.5$.

Crucially, the law of large numbers relies on assumptions. Measurements must be independent and drawn from the same underlying distribution. In biological experiments, these assumptions are often—but not always—satisfied as outlined more in the box.

i Info: The law of large numbers in biology

The assumptions underlying the law of large numbers can be violated in biological experiments. Particularly, independence might not be ensured. Batch effects can introduce systematic shifts between measurements, time correlations can arise in longitudinal studies, and shared reagents or instruments can induce dependencies between samples. Recognizing these effects and designing experiments to minimize them is essential for correct statistical interpretation.

Together, while variability is unavoidable in biology, repeated measurements allow us to extract robust quantitative insights about expectation values. In the following sections, we build on this probabilistic foundation to introduce uncertainty estimates,

beginning with the standard error of the mean.

11 Sampling variability and the standard error of the mean

The law of large numbers states that sample averages converge toward true underlying values as more and more samples are taken. In practice, however, we always work with a finite number of samples. As a result, even when the law of large numbers applies, sample means retain residual variability due to finite sampling.

For example, consider again the coin-toss experiment. If we analyze multiple independent series of coin tosses, each consisting of, for example, 10 tosses, we find that the sample mean differs from series to series (Fig. 11.1A).

To quantify this uncertainty in the mean, it is helpful to formalize the problem a bit more.

For each series of n coin tosses, we compute the sample mean

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i,$$

where the X_i are independent random variables drawn from the same underlying distribution.

Because the individual measurements (or tosses) X_i are random variables, the sample mean $\bar{X}_n$ is itself a random variable. Its value fluctuates from one series of coin tosses to another. This variability of the sample mean across repeated experiments quantifies the uncertainty in our estimate of the true mean. The key question is therefore: how large are these fluctuations of $\bar{X}_n$, and how do they depend on the number of samples n ?

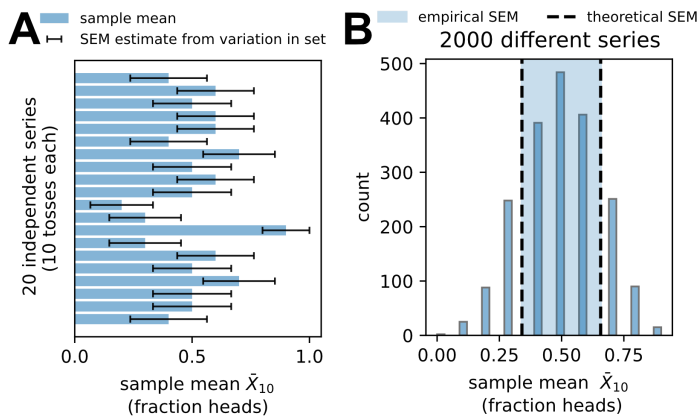


Figure 11.1: **Sampling variability and the standard error of the mean when simulating coin tosses.** (A) Sample means obtained from repeated independent sets of 10 coin tosses. Each horizontal bar represents the mean fraction of heads from one experiment. Different experimental sets yield different mean values due to random sampling. (B) Distribution of sample means across 2000 experiments. The width of this distribution reflects uncertainty in the estimated mean.

11.1 The standard error of the mean

To quantify the fluctuations of the sample mean $\bar{X}_n$, and thus the uncertainty in our estimate of the true mean, we consider the standard deviation of the mean, also called the **standard error of the mean (SEM)**.

The standard deviation and the related variance are fundamental characteristics of a random variable, describing the spread of values (see box).

! Definition: Variance and standard deviation

The **variance** and **standard deviation** quantify the spread of a random variable around its mean.

The variance measures the average *squared* deviation from the mean, which ensures that positive and negative deviations do not cancel and that larger deviations are weighted more strongly. The standard deviation is the square root of the variance and has the same units as the original measurement, making it easier to interpret.

For a *discrete* random variable X with mean μ and probabilities $P(X = x)$, the **variance** is defined as

$$\text{Var}(X) = \sum_x (x - \mu)^2 P(X = x).$$

The standard deviation is

$$\sigma = SD(X) = \sqrt{\text{Var}(X)}.$$

Notably, the SEM (the standard deviation of the mean) follows a very simple relation: If individual measurements fluctuate around the true mean with a typical spread characterized by a standard deviation σ , then the standard error of the mean is given by

$$\text{SEM} = \frac{\sigma}{\sqrt{n}}.$$

Importantly, this formula follows directly from basic probability considerations. We show the derivation in the following box.

i Info: Derivation of the standard error of the mean (SEM)

The standard error of the mean is defined as the standard deviation of the sample mean $\bar{X}$:

$$\text{SEM} \equiv \text{SD}(\bar{X}) = \sqrt{\text{Var}(\bar{X})}.$$

We start by computing the variance of the sample mean:

$$\begin{aligned}\text{Var}(\bar{X}) &= \text{Var}\left(\frac{1}{n} \sum_{i=1}^n X_i\right) \\ &= \left(\frac{1}{n}\right)^2 \text{Var}\left(\sum_{i=1}^n X_i\right) \quad (\text{scaling rule}) \\ &= \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) \quad (\text{additivity rule}) \\ &= \frac{1}{n^2} \sum_{i=1}^n \sigma^2 \quad (\text{identical distributions}) \\ &= \frac{\sigma^2}{n}.\end{aligned}$$

Taking the square root yields the standard deviation of the mean:

$$\text{SEM} = \sqrt{\text{Var}(\bar{X})} = \frac{\sigma}{\sqrt{n}}.$$

Note that this derivation made use of two basic rules:

- Scaling: $\text{Var}(aZ) = a^2 \text{Var}(Z)$
- Additivity: for independent variables, $\text{Var}(\sum Z_i) = \sum \text{Var}(Z_i)$

The simple SEM formula highlights two important points. First, the standard error of the mean decreases with the number of measurements (e.g. coin tosses), reflecting the increasing precision of the sample mean as sample size grows. Second, the magnitude of the uncertainty depends on the variability of the underlying distribution. Measurements drawn from a broader distribution lead to larger uncertainty in the estimated mean.

In practice, the standard deviation σ characterizing this underlying variability is usually unknown. Instead, it is estimated from the observed spread of the data. As a consequence, the SEM formula provides a simple and practical way to estimate uncertainty in the mean directly from experimental measurements.

To build intuition for this uncertainty under realistic conditions, consider again the coin-toss example. If we repeat the coin-toss experiment many times—each time computing the sample mean—the distribution of these means becomes smooth and stable (Fig. 11.1B). The width of this distribution reflects the uncertainty in the mean, indicated by the blue shaded region. This width is well captured by the SEM predicted by the formula above (dashed black lines).

In real experiments, we rarely perform thousands of independent replicates. Instead, we estimate the uncertainty of the mean using the sample standard deviation and the SEM formula. This estimated uncertainty is commonly visualized using error bars, as shown for each coin-toss series in Fig. 11.1A.

Let us further clarify at this point the relation of the SEM to confidence intervals and standard deviations.

First, **confidence intervals**: The standard error of the mean quantifies how much the sample mean is expected to fluctuate due to finite sampling. Confidence intervals build on this idea by converting the SEM into a range of plausible values for the true mean. See box.

i Info: Confidence intervals: from SEM to uncertainty ranges

Under common assumptions—independent measurements and sufficiently large sample size—the distribution of the sample mean is approximately normal. In this regime, uncertainty in the mean can be summarized by

$$\bar{X} \pm z \times \text{SEM},$$

where z is a factor determined by the desired confidence level.

For a 95% confidence interval,

$$\bar{X} \pm 1.96 \times \text{SEM},$$

reflecting the fact that approximately 95% of values drawn from a normal distribution lie within 1.96 standard deviations of the mean.

Importantly, a 95% confidence interval does not assign a probability to a specific interval containing the true mean. Instead, it means that if the experiment were repeated many times, 95% of the resulting confidence intervals would contain the true mean.

Confidence intervals therefore provide a practical and interpretable way to communicate uncertainty and to compare estimates across conditions.

Second, it is important to clearly distinguish between the standard deviation and the standard error of the mean. These are related but conceptually distinct quantities and in the literature error bars are used to describe one or the other. See box.

i Info: Standard deviation vs. standard error

Standard deviation (SD) quantifies the variability of individual measurements. It describes how spread out the data are and reflects biological heterogeneity, technical noise, or both.

For measurements $X_1, \dots, X_n$ with sample mean $\bar{X}$, the sample standard deviation is

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2}.$$

Standard error of the mean (SEM) quantifies uncertainty in the estimated mean due to finite sampling. It describes how much the sample mean would vary if the experiment were repeated many times under identical conditions:

$$\text{SEM} = \frac{s}{\sqrt{n}}.$$

Key distinction:

- SD answers: *How variable are individual measurements?*
- SEM answers: *How precisely has the mean been estimated?*

Error bars may represent SD, SEM, confidence intervals, or other quantities. Figures therefore always need to state explicitly what error bars represent.

11.2 Why normal distributions appear so often

Before we move on to bootstrapping, an easily applicable method for estimating uncertainty in quantities beyond the mean, we briefly comment on the central limit theorem (CLT). Alongside the law of large numbers, the CLT can be seen as a second major theorem in statistics that greatly simplifies statistical reasoning.

The CLT states that, under fairly general conditions, the distribution of sample means approaches a normal distribution as the sample size increases, regardless of the shape of the underlying measurement distribution. For example, in the coin-toss simulations considered above, the resulting distribution of sample means follows the familiar bell-shaped form of the normal distribution (Fig. 11.1B).

The theorem explains why quantities such as the standard error of the mean and confidence intervals are often expressed using normal distributions.

More broadly, the CLT rationalizes why many classical statistical methods are built around normal distributions. Methods such as Student's t-tests for example build on normal distributions to provide simple analytical formulas for uncertainty estimation and hypothesis testing.

Importantly, however, the CLT relies on stronger assumptions than the law of large numbers. Moreover, similar to the law of

large numbers it is an asymptotic result: it describes behavior only in the limit of large sample sizes. In most biological experiments, sample sizes are small and normal approximations as well as the statistical methods that rely on them, can be inaccurate or misleading.

For this reason, we emphasize computational approaches in this class, such as bootstrapping and permutation tests. These rely less on distribution assumptions and instead build uncertainty estimates directly from data.

11.3 Maximum likelihood estimation: fitting probability distributions

So far, we have estimated the mean by averaging repeated measurements. More generally, however, we often want to estimate parameters of an explicit probabilistic model from data. That is, we want to fit a probability distribution to data. Maximum likelihood estimation (MLE) provides a unifying framework for doing exactly this.

11.3.1 The key idea

The key idea of MLE is simple: we choose parameters of the probabilistic model such that the observed data are as likely as possible under the assumed probability model.

Concretely, suppose we observe data points $X_1, \dots, X_n$ that are modeled as independent realizations of a random variable with probability distribution $p(x | \theta)$, where θ denotes an unknown parameter (or set of parameters). The *likelihood* is defined as the probability of the observed data, viewed as a function of θ ,

$$\mathcal{L}(\theta) = \prod_{i=1}^n p(X_i | \theta).$$

The maximum likelihood estimate $\hat{\theta}$ is the parameter value that maximizes this likelihood.

! Definition: Maximum likelihood estimation (MLE)

Given observed data $X_1, \dots, X_n$ and a probabilistic model $p(x | \theta)$, the **maximum likelihood estimate** $\hat{\theta}$ is defined as

$$\hat{\theta} = \arg \max_{\theta} \mathcal{L}(\theta), \quad \mathcal{L}(\theta) = \prod_{i=1}^n p(X_i | \theta).$$

i Info: Why does the likelihood factorize?

The likelihood is written as a product because the data points are assumed to be independent. Independence implies that the probability of observing all data is the product of the probabilities of the individual observations.

11.4 Example: coin tossing as maximum likelihood estimation

Consider again the coin-toss experiment. We model each toss as a Bernoulli random variable,

$$X_i = \begin{cases} 1 & \text{heads,} \\ 0 & \text{tails,} \end{cases}$$

with unknown probability p of obtaining heads.

Under this model, the probability of observing a particular sequence $X_1, \dots, X_n$ is

$$\mathcal{L}(p) = \prod_{i=1}^n p^{X_i} (1-p)^{1-X_i}.$$

This expression is the likelihood of the parameter p given the observed data, that is, it quantifies how plausible different values of p are. Maximizing $\mathcal{L}(p)$ with respect to p yields a remarkably simple result:

$$\hat{p} = \bar{X}_n,$$

the sample mean of the observed coin-toss outcomes.

i Info: Maximum likelihood estimation for a Bernoulli process

Note: We include this derivation to show that the result can be obtained explicitly. You do not need to follow every algebraic step; the key point is that the sample mean emerges as the maximum likelihood estimate.

Consider n independent coin tosses with outcomes

$$X_1, X_2, \dots, X_n \in \{0, 1\},$$

where $X_i = 1$ denotes heads and $X_i = 0$ tails. Assume each toss follows a Bernoulli distribution with unknown probability p of heads.

The likelihood of observing the data is

$$\mathcal{L}(p) = \prod_{i=1}^n p^{X_i} (1-p)^{1-X_i} = p^{\sum_i X_i} (1-p)^{n-\sum_i X_i}.$$

Rather than maximizing $\mathcal{L}(p)$ directly, we maximize the log-likelihood:

$$\ell(p) = \log \mathcal{L}(p) = \left(\sum_i X_i \right) \log p + \left(n - \sum_i X_i \right) \log(1-p).$$

Taking the derivative with respect to p and setting it to zero yields

$$\frac{d\ell}{dp} = \frac{\sum_i X_i}{p} - \frac{n - \sum_i X_i}{1-p} = 0.$$

Solving for p gives

$$\hat{p} = \frac{1}{n} \sum_{i=1}^n X_i = \bar{X}_n.$$

Thus, the maximum likelihood estimate of the probability of heads is simply the sample mean of the observed outcomes.

11.4.1 Least squares as maximum likelihood estimation

The maximum likelihood framework also explains why least squares regression appears so ubiquitously in data analysis.

Assume we model measurements by a straight line with additive noise,

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i,$$

where the noise terms ε_i are independent and normally distributed with mean zero and variance σ^2 .

Under this assumption, each observation y_i is itself normally distributed around the model prediction,

$$p(y_i | \beta_0, \beta_1) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y_i - (\beta_0 + \beta_1 x_i))^2}{2\sigma^2} \right].$$

Assuming independent measurements, the likelihood of the full dataset is the product of these Gaussian probabilities,

$$\mathcal{L}(\beta_0, \beta_1) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y_i - (\beta_0 + \beta_1 x_i))^2}{2\sigma^2} \right].$$

Maximizing this likelihood is equivalent to maximizing its logarithm. Notably, as we discuss further in an optional problem set, this is equivalent to minimizing the sum of squared residuals.

This is why ordinary least squares is typically used as the loss function in regression: least squares regression is the maximum likelihood estimator for the parameters of a straight-line model $\beta_0 + \beta_1 x$ under the assumption of independent Gaussian measurement noise.

In the next section, we will introduce *bootstrapping*, a complementary approach that allows uncertainty estimation without relying on an explicit probabilistic model.

12 Bootstrapping

To quantify uncertainties, we have so far focused on the sample mean, with the standard error of the mean (SEM) quantifying the uncertainty of an average. The SEM is powerful and theoretically well grounded in the law of large numbers, which applies broadly in biological settings and well-designed experiments. However, what if we want to quantify uncertainty in quantities beyond the mean, such as regression parameters, medians, or nonlinear model fits, for which simple analytical error formulas often do not exist? And what if we want to understand not just the size of uncertainty, but the shape of the uncertainty distribution itself?

One convenient and increasingly used approach for estimating uncertainty in such cases is **bootstrapping**. Unlike many statistical methods, bootstrapping does not require assuming a specific functional form for the underlying distribution of the data. Instead, it asks a simple empirical question: how much variability is already present in the data we have measured?

12.1 Resampling with replacement generate virtual realizations

The core idea of bootstrapping is the systematic resampling of the observed dataset. In practice, we let the computer mimic repeated experiments by repeatedly drawing from the available data. To illustrate this idea, consider again the coin toss example. Suppose we perform a coin toss experiment with $n = 10$ tosses and record the outcomes (Fig. 12.1A).

To simulate repeated experiments, we generate new datasets by sampling *with replacement* from these observed outcomes. Each resampled dataset has the same size as the original experiment

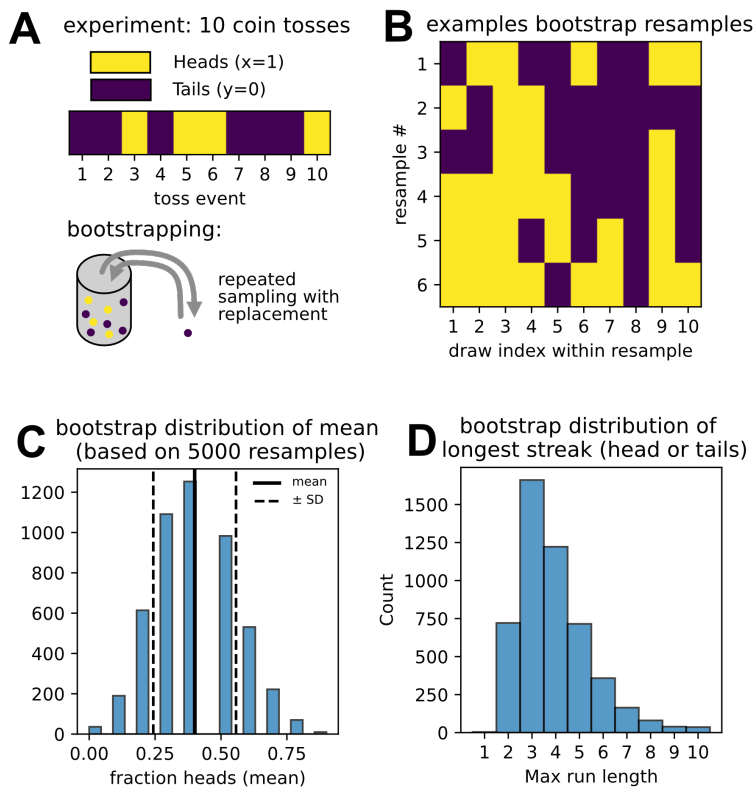


Figure 12.1: **Bootstrapping illustrated with a small coin-toss dataset.** (A) One observed experiment with $n = 10$ tosses (0=tails, 1=heads). (B) Example bootstrap resamples (each resample draws $n = 10$ outcomes *with replacement*) and the corresponding estimated fraction of heads. (C) Bootstrap distribution of the estimated fraction of heads across many resamples, approximating the sampling distribution of the estimator.

but may contain repeated values and omit others (Fig. 12.1B). By repeating this resampling procedure many times, we generate many “virtual” realizations of the experiment.

12.2 Generating and analyzing bootstrap distribution for the quantity of interest

For each bootstrap sample, we then compute the statistic of interest, for example the mean fraction of heads. Repeating this procedure yields a distribution of bootstrap estimates for this quantity. This **bootstrap distribution** approximates the sampling distribution of the estimator: a wider distribution indicates greater sampling uncertainty, while a narrower distribution indicates higher precision. Fig. 12.1C shows an example for the mean of coin-tossing.

Bootstrapping is powerful because it approximates the full sampling distribution of a quantity of interest directly from the available data. This is particularly useful when we care about statistics beyond the mean, or when the shape of the uncertainty distribution itself is informative.

For example, once the bootstrap distribution has been obtained, it can be used to define confidence intervals. E.g. the 2.5th and 97.5th percentiles of the bootstrap mean distribution can be used to define a 95% bootstrap confidence interval. Another example for coin tossing is the statistics of longest streaks (either heads or tails only). The distribution resulting from bootstrapping is shown in Fig. 12.1D.

12.3 Practical implications

Because bootstrapping works for **any statistic**, not just means, it is particularly useful in biology, where data are often skewed and where meaningful summaries frequently go beyond averages. Quantities such as medians, regression slopes, and parameters of nonlinear models often lack simple analytical formulas for their uncertainty. Bootstrapping provides a general way to estimate uncertainty for these quantities directly from the data.

Furthermore, bootstrapping is straightforward to implement with modern computational tools. In the accompanying Jupyter notebook, we demonstrate how to implement bootstrapping in Python and apply it to examples where analytical uncertainty estimates are unavailable or inappropriate.

i Historical context

The bootstrap is a fairly new approach, introduced by Bradley Efron in 1979. While conceptually simple, its widespread use became practical only with fast computing. In the life sciences, bootstrapping has become increasingly common as datasets have grown larger and computational tools more accessible, enabling routine uncertainty estimation for complex statistics and models.

12.4 Bootstrapping to evaluate fitted parameters

So far, we have used bootstrapping to quantify uncertainty in simple summary statistics such as the mean. The same idea can be applied to evaluate uncertainty in fitted model parameters, even when analytical error estimates are unavailable.

As a concrete example, consider fitting an exponential growth model

$$N(t) = N_0 e^{rt}$$

to time-series data, where r denotes the growth rate. A standard regression fit yields a single best-fit estimate $\hat{r}$ (Fig. 12.2A). However, due to measurement noise and finite sampling, this estimate is uncertain.

Bootstrapping provides a direct way to quantify this uncertainty. Starting from the observed growth curve, we generate many bootstrap datasets by resampling the measured data points (t_i, N_i) with replacement (Fig. 12.2B). For each bootstrap dataset, we refit the exponential model and record the resulting growth rate. Repeating this procedure produces a **bootstrap distribution** of fitted growth rates (Fig. 12.2C).

The spread and shape of this bootstrap distribution reflect how sensitive the fitted parameter is to sampling variability in the data. A narrow distribution indicates that the growth rate is well constrained by the measurements, whereas a broad or skewed distribution signals substantial uncertainty or limited information content in the data.

Notably, this approach requires no analytical error formulas and applies equally to linear and nonlinear models. Bootstrapping is therefore a good general method for estimating uncertainty in fitted parameters.

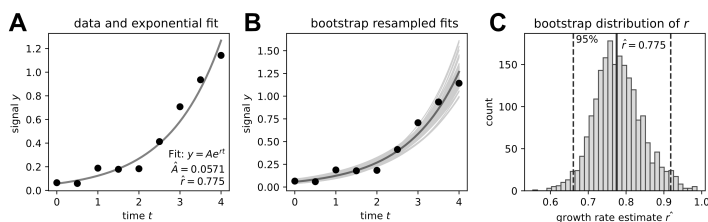


Figure 12.2: **Bootstrapping to estimate uncertainty in fitted growth parameters.** (A) Exponential growth data with measurement noise (points) and a best-fit exponential model (solid line), yielding a single point estimate of the growth rate $\hat{r}$. (B) Example bootstrap resamples of the original dataset, generated by resampling the observed data points (t_i, N_i) with replacement. Each resampled dataset is refit independently. (C) Bootstrap distribution of the fitted growth rate r obtained from many resampled datasets. The spread of this distribution quantifies the uncertainty in the growth-rate estimate due to finite sampling and measurement noise.

12.5 Caveats and assumptions

While bootstrapping is very general, it does not eliminate the need for careful experimental design. Like analytical error estimates, bootstrap results rely on the assumption that the available data are representative of the underlying process. In partic-

ular, standard bootstrapping assumes statistically independent observations, analogously to the assumptions underlying the SEM introduced above (e.g., not time-correlated measurements or pseudo-replicates). Bootstrapping is therefore a powerful complement to analytical approaches, not a substitute for well-defined experiments or clearly identified biological replicates.

Part V

Week 5: Hypothesis testing

13 Hypothesis testing

In the previous section, we focused on estimating uncertainty: how precisely can we determine a quantity given a finite sample size and noisy data? In many biological settings, however, we want to go further and ask whether observed data are consistent with a proposed explanation or hypothesis. *Hypothesis testing* provides a statistical framework for addressing this question.

Crucially, hypothesis testing does not aim to directly assess whether a specific hypothesis is true, something that is generally very difficult, if not impossible, to establish conclusively. Instead, hypothesis testing evaluates whether the observed data are *consistent with a baseline explanation*, called the *null hypothesis*. If the data are highly inconsistent with the null hypothesis, this provides evidence against it and in favor of an alternative explanation.

13.1 Example: testing whether a coin is fair

To introduce the core idea, consider again the coin-toss example introduced earlier. Suppose we toss a coin $n = 10$ times and record heads as 1 and tails as 0. After performing the experiment, we compute the observed fraction of heads and obtain $\bar{X}_n = 0.7$ as shown in Fig. 13.1A. As this fraction deviates substantially from 0.5, we may wonder whether the coin is not fair. That is, the coin is biased, not producing heads and tails with equal probability.

To evaluate this suspicion or hypothesis, we first formulate a null hypothesis: the coin is fair, with the probability of heads being $p_0 = 0.5$. Assuming this hypothesis is true, we can ask what outcomes we would expect from repeated coin tossing experiments with $n=10$ tosses each. Using a computer, we can

simulate many independent series of 10 coin tosses under the assumption of a fair coin and record the resulting fractions of heads. The distribution of these simulated outcomes is shown in Fig. 13.1B.

We can now evaluate our observation ($\bar{X}_n = 0.7$) relative to this reference distribution. Specifically, we ask: what fraction of simulated experiments would produce a result at least as extreme as the observed one? This fraction corresponds to the shaded region in Fig. 13.1B and is commonly called a *p-value*.

For 10 tosses, observing a fraction of heads larger as 0.7 (or 0.3 or smaller) is highly likely (about 37%) and compatible with the sampling variability expected under the assumption of a fair coin. That is, the *p-value* is about 0.35. Accordingly, with this experiment alone we cannot conclude that the coin is unfair. In other words, the observation do not provide strong evidence against the null hypothesis.

13.2 General steps of hypothesis testing

The coin-toss example already contains all essential ingredients of a hypothesis test. We now explicitly state the steps involved.

- First, we need to formulate a *null hypothesis*. The null hypothesis represents a baseline explanation for the data. In the coin-toss example, the null hypothesis was that the coin is fair, meaning that the probability of heads is $p_0 = 0.5$.
- Second, we define a *test statistic* that summarizes the observed data in a single number. In the coin-toss experiment, a natural test statistic is the sample mean

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i,$$

which corresponds to the observed fraction of heads.

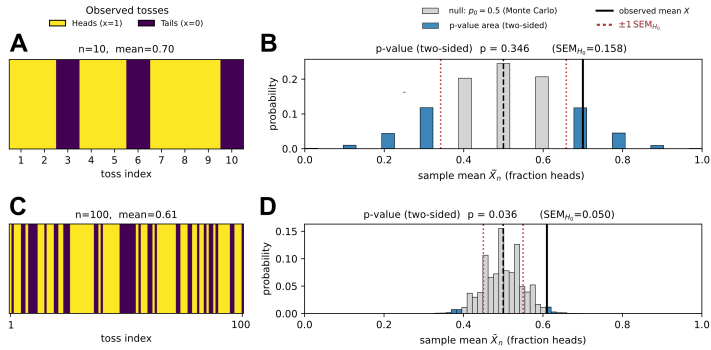


Figure 13.1: **Hypothesis testing to evaluate if a coin is not fair.** (A) Outcomes of a single coin-toss experiment with $n = 10$ tosses. Each vertical line represents one toss (heads = 1, tails = 0). The mean $\bar{X}_n$ is the obtained fraction of heads. (B) *Null distribution* of the sample mean generated by Monte Carlo simulation under the null hypothesis of a fair coin ($p_0 = 0.5$). The vertical dashed line marks the null mean, and the solid line marks the observed mean. The shaded region corresponds to outcomes at least as extreme as the observation, defining the two-sided p-value. Dotted lines indicate $\pm 1 \text{ SEM}$ under the null hypothesis. (C,D) Same as in (A,B) but for $n = 100$ tosses. In this case, the null distribution is much narrower, and the the observed deviation from 0.5 leads to a much smaller p-value.

- Third, we construct a reference distribution for this test statistic under the null hypothesis. This distribution describes the variability we would expect if the null hypothesis were true and the experiment were repeated under identical conditions. In the coin-toss example, the reference distribution was obtained by repeatedly simulating coin-toss experiments under the assumption of a fair coin.
- Finally, we compute a *p-value*. The p-value is defined as the probability of observing a test statistic at least as extreme as the one measured, assuming that the null hypothesis is true. In the coin-toss example, the p-value corresponds to the shaded area in Fig. 13.1B, representing outcomes with a deviation from p_0 that is equal to or larger than the observed deviation.

i Info: One-sided and two-sided p-values

Hypothesis tests can be either one-sided or two-sided, depending on the scientific question.

One-sided test: A one-sided test considers deviations in only one direction (either larger or smaller than expected under the null hypothesis). The p-value is the probability of observing a test statistic at least as extreme as the observed one in that specified direction.

Example: Testing whether a drug *increases* growth rate.

Two-sided test: A two-sided test considers deviations in both directions (larger *or* smaller than expected). The p-value is the probability of observing a test statistic at least as extreme as the observed one in either direction.

Example: Testing whether a drug *changes* growth rate (increase or decrease).

When to use which:

- Use a one-sided test only when deviations in the opposite direction are scientifically irrelevant.
- Use a two-sided test when both increases and decreases are meaningful.
- In most biological applications, two-sided tests are preferred by default.

Importantly, the p-value is *not* the probability that the null hypothesis is true. Rather, it quantifies how compatible the observed data are with the null hypothesis. Small p-values indicate that the observation would be unlikely if the null hypothesis were correct, providing evidence against it.

Furthermore, large p-values do not establish the null hypothesis. A p-value may for example be large simply because the sample size is too small to exclude the null hypothesis. This is illustrated by the coin-toss example. As shown in Fig. 13.1C and D, increasing the number of observations to $n = 100$ narrows the reference distribution and leads to a much smaller p-value, making it unlikely that the coin is fair. In our example, to generate the observations we actually assumed a slightly biased coin with a probability of 0.6 for heads. The p-value was initially large only because of the small sample size.

13.3 From coin tosses to gene expression: comparing biological conditions

The coin-toss example is unusually simple because the null hypothesis specifies a complete probabilistic model. We know exactly how data should behave under the assumption of a fair coin, and we can generate the corresponding reference distribution directly.

In biology, we most often do not know underlying probabilistic models and hypothesis testing is commonly used in a different setting. Rather than asking whether data match a fully specified probability model, we typically compare measurements across experimental conditions and ask whether observations are significantly different.

A canonical example is gene expression analysis. Suppose we measure the expression level of a gene across multiple biological replicates in two conditions, for example untreated cells and cells exposed to a drug. Each condition yields a set of expression measurements, and we observe that the average expression differs between groups (Fig. 13.2A).

At this stage, two distinct questions arise:

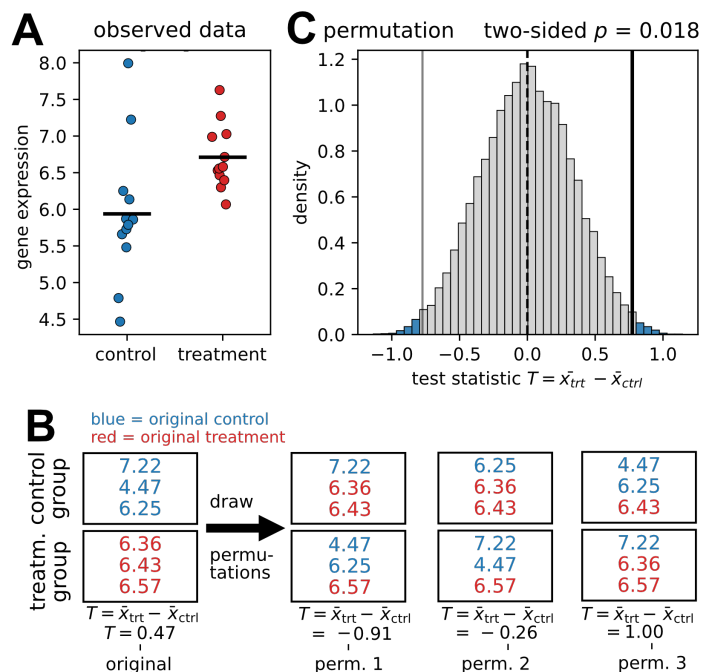


Figure 13.2: **Permutation test example: Probing for differences in gene expression.** (A) Expression measurements of a single gene across two experimental conditions (e.g. control and treatment), with each point representing one biological replicate. The observed difference in mean expression between conditions defines the test statistic $T_{\text{obs}} = \bar{x}_{\text{trt}} - \bar{x}_{\text{ctrl}}$. (B) Illustration of how permutation samples are generated under the null hypothesis for a simplified example with 6 measurement points only. The numerical values correspond to the same observed measurements shown in (A). Colors indicate the *original* group membership, while their reassignment into control and treatment groups is shuffled. Each column shows one possible reassignment of labels consistent with the null hypothesis that group labels are interchangeable. For each reassignment, the test statistic T is recomputed. (C) Null distribution of the test statistic obtained by systematically drawing many such permutations of the labels while keeping the observed values fixed (this includes again all observed datapoints shown in A). The vertical line marks the observed test statistic T_{obs} . The shaded region indicates permutation outcomes at least as extreme as the observed value and defines the two-sided p-value.

1. How large is the observed difference?
2. Is the difference significant or could a difference of this size plausibly arise if the drug had no effect?

The first question concerns effect size and uncertainty, which we can quantify using e.g. SEMs or bootstrapping. The second question requires a hypothesis test.

The null hypothesis in this setting is: **Expression measurements in the two conditions are drawn from the same underlying distribution.**

13.4 Permutation tests to evaluate differences

Importantly, this null hypothesis does not specify a particular probabilistic model or probability distribution for expression values. Instead, it states that the experimental condition does not influence expression. If this is true, then the labels “control” and “treatment” are arbitrary and carry no information. In other words, under the null-hypothesis, the assignment of measurements to conditions is interchangeable. But if that is true, then randomly reassigning condition labels should not systematically change the observed difference in expression. To evaluate the null-hypothesis we thus run a **permutation test**. That is, we randomly assign measurements to different conditions and examine the consequences for a statistic of interest. Specifically, to perform a permutation test, we proceed as follows:

1. Compute the observed test statistic of interest, for example the difference in mean expression between conditions.
2. Randomly assign the measured expression levels to the condition labels. Figure 13.2B shows, as simplified example, a few permutations for 6 datapoints only.
3. Recompute the test statistic for the shuffled data.
4. Repeat this procedure many times to generate a reference distribution under the null hypothesis.

5. Compare the observed test statistic to this distribution to obtain a p-value.

For the data in Figure 13.2A the result of the permutation test is shown in Figure 13.2C. Technically, it shows the null distribution obtained by permuting condition labels many times. The blue shaded region indicates permutation outcomes at least as extreme as the observed difference and defines the p-value. For this example, the resulting p-value is 0.018, meaning that under the typical threshold of 0.05 we do consider the gene expression levels to be significantly different.

13.5 Relation to classical parametric tests

Notably, permutation tests like the one shown construct the null distribution directly from the data. Permutation tests therefore provide a powerful and general framework for hypothesis testing in biological data, especially when analytical distributions are unknown or unreliable. Historically, however, parametric tests, most commonly the t-test, have been widely used in biological data analysis. These tests are elegant and powerful when their assumptions are satisfied, but they rely on key assumptions such as independent observations, approximately normally distributed noise, and similar variances between groups.

Biological data frequently violate these assumptions, in which case parametric p-values can become misleading. In many biological settings, permutation tests are therefore preferable, as they make fewer assumptions and rely directly on the structure of the observed data.

i Info: The t-test: historical context and modern perspective

The t-test is one of the most widely used statistical tests in the life sciences and has been a cornerstone of data analysis for more than a century. It was introduced by William Sealy Gosset in 1908, working under the pseudonym Student, to address inference from small-sample experiments when the population variance is unknown.

At the time, computation was done by hand and analytical solutions were essential. The t-test provided a simple, closed-form way to assess whether two group means differ more than expected from sampling variability alone. For much of the 20th century, it became the default tool for hypothesis testing in biology, medicine, and the social sciences.

From a modern perspective, the t-test can be understood as a special case of a more general idea: it approximates the sampling distribution of a difference in means under the assumptions of independent observations, approximately Gaussian noise, and similar variances between groups. When these assumptions hold, the t-test is powerful.

However, it is now feasible with modern computers to construct null distributions directly from the data using permutation tests. Permutation tests require fewer assumptions, make the null hypothesis explicit, and work naturally for arbitrary test statistics and small sample sizes. Thus, while the t-test remains useful and interpretable under its assumptions, permutation tests provide a more general and transparent framework for hypothesis testing in contemporary biological research.

Feature	t-test	Permutation test
Distribution assumptions	Yes	No
Small sample sizes	Risky	Robust
Arbitrary test statistics	No	Yes

13.6 Practical implementation and important caveats

With modern computational tools, permutation tests are straightforward to implement. We show how this is done in Python in Problem Set 5. However, it is important to recognize that hypothesis testing and p-values come with important limitations and potential pitfalls.

i Info: Common pitfalls when interpreting p-values

- **A p-value is not the probability that the null hypothesis is true.** It is the probability of observing data at least as extreme as those measured, assuming the null hypothesis is true.
- **A small p-value does not measure effect size or biological relevance.** Statistical significance does not imply biological importance.
- **Significance thresholds are arbitrary.** Common cutoffs such as $p < 0.05$ are conventions, not fundamental scientific boundaries.
- **P-values alone do not establish biological hypotheses.** Robust conclusions require independent validation, additional data, and complementary experiments. Hypothesis testing does not replace the need for clever experimental design and rigorous research.
- **Multiple hypothesis testing must be accounted for.** When many tests are performed, false positives accumulate unless appropriate corrections are applied.

We discuss multiple hypothesis testing and correction methods in detail in the next section.

14 Multiple hypothesis testing

Hypothesis testing becomes particularly challenging when many hypotheses are tested simultaneously. This situation is common in modern biological research, especially in high-throughput and exploratory studies such as transcriptomics, proteomics, genome-wide association studies, and large-scale screens, where thousands to millions of tests may be performed in parallel.

In this setting, naive interpretation of p-values can be highly misleading. For example, if we test 10,000 independent hypotheses, then even if all null hypotheses are true, a common significance threshold of $\alpha = 0.05$ implies that we expect on average about 500 false positives purely due to random variation.

The core challenge is that p-values are defined for a *single* hypothesis test. When many tests are performed, the probability of observing small p-values increases. This is not a flaw of statistics, but a mathematical consequence of repeated testing.

In modern biological settings, this problem is especially acute because:

- datasets are large and high-dimensional,
- many hypotheses are tested, often generated post hoc,
- measurements are noisy and heterogeneous,
- follow-up experiments are costly.

How can we deal with this challenge and prevent large numbers of false discoveries that misdirect resources and obscure true biological signals?

14.1 Error rates and false positives

To understand how multiple testing corrections work, it is useful to introduce formally false positives and error rates.

[TABLE]

A case in which a test is declared significant when the null hypothesis is in fact true is called a *false positive*, or a *Type I error*. The probability of making such an error in a single test is controlled by the significance level α . For a single hypothesis test, using $\alpha = 0.05$ means that we accept a 5% chance of a false positive.

When m hypotheses are tested simultaneously, the expected number of false positives is approximately

$$m\alpha.$$

Thus, even moderate values of α can lead to large numbers of false discoveries in large-scale studies.

Two major error rates are commonly used to control this problem:

Family-wise error rate (FWER).

The family-wise error rate is the probability of making *at least one* false positive among all tests. A classical method to control the FWER is the **Bonferroni correction**, which replaces α with

$$\alpha_{\text{Bonf}} = \frac{\alpha}{m}.$$

Only p-values smaller than α/m are declared significant. This approach is simple and rigorous, but often overly conservative in large datasets.

i Info: Why the Bonferroni correction works

Suppose we perform m independent hypothesis tests, each at significance level α' . Let's assume the worst case: each hypothesis test is a true null, and let's call each hypothesis test $\mathcal{H}_0^{(1)}, \mathcal{H}_0^{(2)}, \dots, \mathcal{H}_0^{(n)}$.

For a single test, the probability of a false positive is at

most α' .

The probability of at least one false positive across all tests satisfies

$$\mathbb{P}(\text{at least one false positive}) \leq m\alpha'.$$

To see this we use the union bound:

$$\begin{aligned} & \mathbb{P}(\text{at least one false positive}) \\ &= \mathbb{P}(\{\text{reject } \mathcal{H}_0^{(1)}\} \cup \{\text{reject } \mathcal{H}_0^{(2)}\} \cup \dots \cup \{\text{reject } \mathcal{H}_0^{(n)}\}) \\ &\leq \mathbb{P}(\{\text{reject } \mathcal{H}_0^{(1)}\}) + \dots + \mathbb{P}(\{\text{reject } \mathcal{H}_0^{(n)}\}) = m\alpha' \end{aligned}$$

To ensure that this probability is at most α , we require

$$m\alpha' \leq \alpha.$$

We can see that if $\alpha' = \frac{\alpha}{m}$, the above condition is met.

This leads to the Bonferroni correction: each individual test is evaluated at level α/m , guaranteeing control of the family-wise error rate.

False discovery rate (FDR).

The false discovery rate is the expected fraction of false positives among all rejected hypotheses. Controlling the FDR allows some false positives, but limits their proportion. Referring to the table above, $FDR = V/R$; the number of false discovery per rejected null hypothesis. Procedures such as the **Benjamini–Hochberg** method adaptively adjust p-value thresholds and are widely used in omics and screening studies.

FDR control typically provides a better balance between discovery and reliability in exploratory biological research. This is ideal for finding candidate effects, where tolerance for false positives is higher.

The Benjamini–Hochberg (BH) method controls the false discovery rate (FDR) by comparing each p-value to an adaptive threshold.

Let $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$ be the sorted p-values.

For a desired FDR level q , BH finds the largest index k such that

$$p_{(r)} \leq \frac{r}{m}q.$$

This is visualized below, on a plot showing p-values arranged by rank.

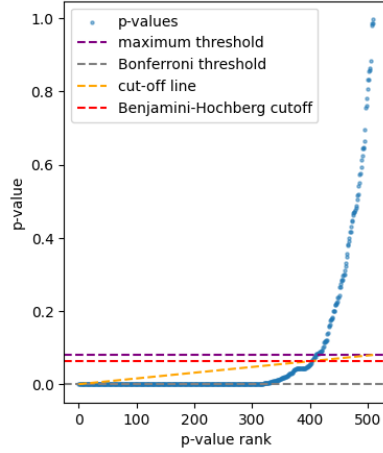


Figure 14.1: Benjamini-Hochberg procedure illustrated on a plot of ranked p-values. P-values are sorted in ascending order and shown on the y-axis, with their rank on the x-axis. Thresholds for the Bonferroni correction (grey), maximum threshold at which we want to bound the FDR (purple) and the Benjamini-Hochberg (BH) correction (red) are shown as horizontal lines. The BH correction line is shown in orange; the BH threshold is drawn where this line crosses the empirical p-values.

The procedure is as follows:

1. Sort p-values into ascending order.
2. Find the largest p-value ($p_{(k)}$) that meets the condition $p_{(r)} \leq \frac{rq}{m}$
3. Reject the null hypothesis for all values lower than $p_{(r)}$

See the box below for the proof for why this controls FDR at level q .

i Info: Proof that Benjamini-Hochberg procedure bounds expected FDR

The goal of the procedure above, which we will show here that it meets, is to bound the expected FDR at a level q (commonly set to 0.05). Referring to the table showing different types of error rates above, this is equivalent to wanting

$$\text{FDR} = V/R \leq q.$$

The orange line in Figure 14.1 represents the maximum value p-value which would satisfy keeping the expected FDR bounded by q . This is a linear relationship because the FDR is the ratio between total hypotheses rejected and allowable false positives; as we reject more hypothesis, we're comfortable with proportionally more false discoveries.

Let's remember that p-values arising from true null hypotheses are uniformly distributed between 0 and 1; $p \sim \text{Uniform}(0, 1)$, or equivalently $\mathbb{P}(p \leq t) = t$ for $t \in [0, 1]$.

For the Benjamini-Hochberg procedure, all rejected p-values satisfy $p_{(i)} < \frac{rq}{m}$, where r is the total number of rejected hypotheses, q is the level at which FDR is controlled, and m is the total number of tests. To see this visually, remember that we reject all p-values up until rank r that are under the line $y = x \frac{q}{m}$; $\frac{q}{m}$ is the slope (you can see this as this is the maximum distance in the orange line divided by the maximum x-distance). Let's call this bound on p-values of rejected hypotheses $t_r := \frac{rq}{m}$.

From following the procedure on the data, we now have a fixed value for; $R = r$. Let's call the number of true nulls in the data (which we cannot measure) m_0 . The probability of a true null having a p-value below t_r is simply t_r , so the expected number of true nulls (i.e $E(V)$) below t_r is $t_r m_0$. by substituting this into the definition of FDR, we can see that the expected value of FDR is indeed bounded by q :

$$E\left[\frac{V}{R}\right] = \frac{t_r m_0}{r} = \frac{rqm_0}{mr} = q \frac{m_0}{m} \leq q.$$

14.2 Final notes

Multiple-testing correction does not turn hypothesis testing into a definitive decision rule. Even after correction, a small p-value only indicates that the data are unlikely under a specified null model; it does not establish biological truth.

In biology, hypothesis testing should therefore be viewed as a tool for prioritization rather than confirmation. Statistical significance helps identify candidates for follow-up, but robust conclusions require independent data, replication, and mechanistic understanding.

This point will be emphasized when we specifically consider omics data and their biological interpretation.

Part VI

Week 6: Counting statistics

15 Counting Statistics and Discrete Biological Measurements

Following our discussion of fitting, uncertainty, and hypothesis testing, we now turn to counting statistics, a final major ingredient required to rigorously analyze biological data.

Many biological measurements are fundamentally based on counting. Classical examples include the number of colonies that appear on a plate after plating, or the counting of radioactive decay events in tracer experiments. Counting also lies at the core of essentially all modern omics technologies. This includes sequencing reads, detected transcripts in cells, spectral counts in proteomics, and barcodes in lineage-tracking experiments.

Understanding how such counts arise, how they fluctuate, and how their variability depends on experimental conditions is therefore essential for interpreting modern biological data. In this section, we introduce the basic statistical principles of counting before turning to omics analyses next week.

15.1 Counting as a stochastic process

At first glance, counting may appear to be a simple deterministic procedure: one counts objects, and repeating the same experiment under identical conditions should yield the same result. In practice, however, this is rarely the case. Even under carefully controlled conditions, repeated counting experiments produce different outcomes.

This variability is not due to experimental mistakes. Instead, it reflects two distinct sources of randomness. First, even when the

underlying biological system is unchanged, the act of observing a finite number of discrete events introduces sampling noise. This sampling noise arises purely from probabilistic sampling and is present in every counting experiment. Second, biological and technical factors can introduce additional variability between cells, samples, and experiments. Here, we focus on sampling noise and its statistical consequences.

The coin-toss model introduced earlier illustrates this behavior. Each toss represents a single random event with two possible outcomes, occurring with probabilities p and $1 - p$. When many repeated coin-toss experiments are performed, the total number of heads obtained is not fixed. Instead, it fluctuates from experiment to experiment, following a probability distribution. Mathematically, each toss is described by a Bernoulli distribution, and the total number of heads follows a binomial distribution (Fig. 15.1A).

Strikingly, many biological counting experiments follow the same logic. Each molecule, cell, or genetic element represents a potential trial analogous to a coin toss: it is either detected or not, mutated or not, sequenced or not.

As a concrete example, consider a typical RNA-seq experiment in which thousands or millions of sequencing reads are generated from a pool of transcripts encoding a lot of different genes. Each read can be viewed as a random draw from this pool. Focusing on a single gene, each draw has some probability of originating from that gene. The observed read count therefore reflects the outcome of many such random trials, a Bernoulli process (Fig. 15.1B).

i Info: Sampling with and without replacement in sequencing experiments

In principle, many biological counting experiments (such as RNA sequencing) sample molecules from a finite pool *without replacement*. However, in most omics experiments, the total number of molecules greatly exceeds the number of reads. Removing one molecule has a negligible effect on the remaining pool. In this limit, sampling without replacement is well approximated by independent Bernoulli

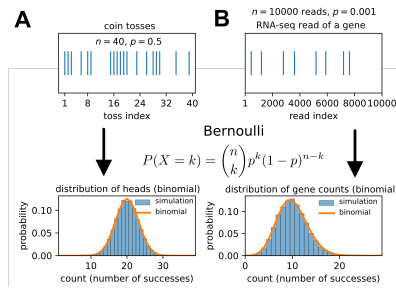


Figure 15.1: **Bernoulli trials in coin tossing and RNA-seq.**

(A) Repeated coin-toss experiments consist of independent Bernoulli trials with success probability $p \approx 0.5$, producing binomially distributed numbers of heads. (B) In RNA-seq, each sequencing read represents a Bernoulli trial with small probability p of originating from a given gene, again producing binomially distributed counts. Although the contexts differ, both experiments follow the same underlying statistical logic, differing only in their parameters.

trials. Note that this can be different in single-cell methods like single-cell RNA-Seq where available reads be very low.

15.2 From binomial to Poisson: rare events and large numbers

In most biological systems, individual events occur with very small probabilities (in contrast to a coin flip) but have many opportunities to occur. Consider again RNA-seq as an example. In a pool of transcripts from thousands of genes, the probability that a given read originates from one specific gene is typically very small. However, many millions of reads are analyzed to assemble a useful picture of gene expression. This logic applies similarly to proteomics spectral counts, barcode sequencing, and other high-throughput assays (see box below).

i Info: Low probabilities and high counts in biological measurements

Many modern high-throughput counting experiments share the same basic structure: individual events are rare, but the number of opportunities for these events is very large. Common examples include:

- each DNA snippet has a small probability of being sequenced, but millions of molecules are sampled,
- each transcript has a small probability of being detected in single-cell assays, but thousands of cells are measured,
- each peptide has a small probability of being identified in mass spectrometry, but large peptide pools are analyzed,
- each barcode has a small probability of being sampled in lineage-tracking experiments, but many lineages are tracked,
- each cell division has a small probability of mutation, but many divisions are observed.

In all of these cases, measurements arise from many independent trials with a small success probability p and a large number of trials n , leading to a finite expected count $\lambda = np$.

This combination of rare events and many opportunities naturally gives rise to Poisson statistics.

In such scenarios, large numbers of trials with rare successes, the number of counts obtained for a single element is well described by Poisson statistics:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

where λ is the expected number of events. For example, when analyzing 10^6 sequencing reads, if a gene contributes on average about 1000 reads, then $\lambda \approx 1000$.

Importantly, this formula naturally arises as a limit of the binomial distribution (see box). To consider counting statistics we thus specifically focus on the Poisson distribution in the following.

i Info: The Poisson limit of the binomial distribution

The Poisson distribution arises as a limiting case of the binomial distribution.

For a binomial process with n trials and success probability p ,

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}.$$

Now consider the limit where

$$n \rightarrow \infty, \quad p \rightarrow 0, \quad \lambda = np \text{ is fixed.}$$

In this regime, we can approximate:

$$\binom{n}{k} \approx \frac{n^k}{k!}, \quad p^k = \left(\frac{\lambda}{n}\right)^k, \quad (1 - p)^n \approx e^{-\lambda}.$$

Substituting,

$$P(X = k) \approx \frac{n^k}{k!} \left(\frac{\lambda}{n}\right)^k e^{-\lambda} = \frac{\lambda^k e^{-\lambda}}{k!}.$$

This is exactly the Poisson distribution.

15.3 Mean–variance relationships in counting data

A defining feature of the Poisson distribution is that it is fully determined by a single parameter, the expected number of events λ . Once λ is specified, the entire distribution is fixed.

What then is the standard deviation or variance of the Poisson distribution? It is given by the same parameter: For a Poisson-distributed random variable X , the mean and variance are

equal:

$$\mathbb{E}[X] = \text{Var}(X) = \lambda.$$

This relation means that the variation of this distribution is substantial. Specifically:

- Low-count measurements are dominated by statistical noise,
- relative uncertainty is largest for rare events (small λ ,
- higher counts (larger λ) are more precise but still noisy,
- distributions become approximately Gaussian at large λ .

These properties are illustrated in Fig. 15.2.

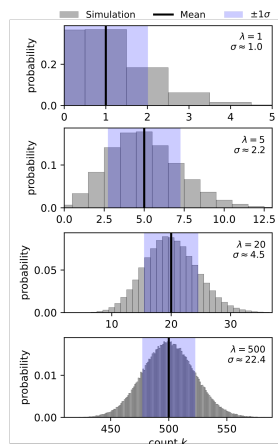


Figure 15.2: **Poisson counting noise is substantial and depends on the mean.** Shown are Poisson probability mass functions for four different expected counts λ . As λ increases, the distribution becomes broader in absolute terms but narrower relative to the mean. A defining property of the Poisson model is $\mathbb{E}[X] = \text{Var}(X) = \lambda$, so the standard deviation scales as $\sqrt{\lambda}$ and the fractional uncertainty as $1/\sqrt{\lambda}$. Low-abundance features are therefore dominated by sampling noise, whereas highly expressed features are measured much more precisely.

A useful RNA-seq intuition is to compare two genes with different expression levels. Suppose one gene has $\lambda = 5$ reads per sample, while another has $\lambda = 500$. For the lowly expressed gene, the standard deviation is $\sqrt{5} \approx 2.2$, comparable to the mean. For the highly expressed gene, the standard deviation is $\sqrt{500} \approx 22$, which is small relative to the mean. Thus, highly expressed genes are intrinsically easier to quantify than lowly expressed genes.

15.4 Biological noise, overdispersion, and negative binomial models

In contrast to many continuous measurements, counting noise cannot be separated from the signal itself. The Poisson distribution provides a natural baseline model for this sampling noise. However, real biological data often exhibit substantially more variability than predicted by Poisson statistics. That is, the observed variance is commonly higher than the mean:

$$\text{Var}(X) > \mathbb{E}[X].$$

This phenomenon is known as *overdispersion*. Overdispersion reflects additional sources of variation such as cell-to-cell heterogeneity, bursty gene expression, unobserved regulatory fluctuations, and technical variability. As a result, Poisson models provide a useful lower bound on measurement noise, but it is often necessary to go beyond the Poisson model and account for additional variability.

A common way to account for overdispersion is the negative binomial distribution. It arises naturally when a Poisson process is combined with a fluctuating (Gamma-distributed) expectation value λ . This mechanism is illustrated in Fig. 15.3. For a fixed mean λ , counting noise is described by a Poisson process (Fig. 15.3A), whereas for a fluctuating mean, the resulting count distribution follows a negative binomial model (Fig. 15.3B).

For the negative binomial distribution, the variance is given by

$$\text{Var}(X) = \mu + \frac{\mu^2}{\theta},$$

where θ is the dispersion parameter that quantifies the additional variability relative to a Poisson process. In the limit $\theta \rightarrow \infty$, the negative binomial distribution reduces to the Poisson distribution.

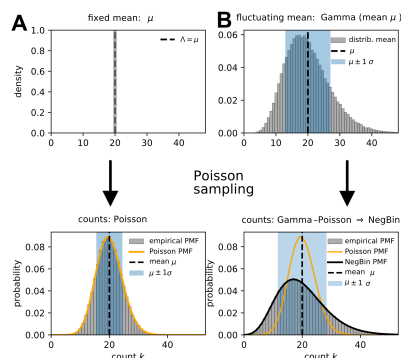


Figure 15.3: Overdispersion from a fluctuating rate. (A) Fixed-rate Poisson sampling: The expectation value is fixed at μ (upper panel). Counts follow $X \sim \text{Poisson}(\mu)$ with $\text{Var}(X) = \mu$ (lower panel). (B) Fluctuating-rate sampling: The mean fluctuates around μ according to a Gamma distribution (upper panel). Counts generated from a Poisson process with such a fluctuating rate follow a negative binomial distribution with mean μ and variance $\mu + \mu^2/\theta$ (lower panel). Histograms show simulations; solid curves show the analytical Poisson and negative binomial probability mass functions.

i Info: Why the name “negative binomial” distribution?

Historically, the negative binomial distribution was introduced to describe how many failures occur before a fixed number of successes in repeated Bernoulli trials. Mathematically, the same distribution also arises from a binomial expansion with a negative exponent (Taylor series expansion of $(1 - p)^{-r}$), hence the name.

State-of-the-art methods for analyzing omics data build directly on these basic principles of counting statistics, with the negative binomial distribution as a starting point, as we will discuss in

Section ??.

In the following section, we will also examine the classic fluctuation test of Luria and Delbrück, a highly influential experiment that introduced rigorous statistical reasoning into molecular biology.

Part VII

Week 7: Omics and high-dimensional data

16 Omics Data: From Counting to Cellular State

Over the past two decades, advances in sequencing, mass spectrometry, and biochemistry have transformed biology through “omics” measurements. These approaches have shifted biological research from focusing on individual components to studying coordinated, system-wide responses. Exciting! However, proper statistical analysis is crucial for these methods, both to reveal their full potential and to prevent misleading conclusions.

In this section, we discuss the basic analysis steps of omics data. We focus on RNA sequencing to illustrate major analysis principles and highlight key interpretational challenges.

i Historical context: Omics methods are relatively new

Early large-scale molecular measurements emerged in the late 1990s with DNA microarrays. In the mid-2000s, next-generation sequencing enabled RNA sequencing (RNA-seq), providing higher resolution and dynamic range. In parallel, advances in mass spectrometry drove modern proteomics and metabolomics. Since the 2010s, continued improvements in throughput, cost, and computation have made omics approaches routine tools in systems and biomedical research. Today, multi-omics studies integrating transcriptomic, proteomic, and metabolic data are increasingly common. In parallel, data analysis pipelines have evolved substantially alongside measurement technologies.

16.1 RNA-Seq in a Nutshell

RNA sequencing (RNA-seq) has become one of the most widely used omics technologies, providing insights into the transcriptome and gene expression of cells. RNA-seq measures gene expression by converting cellular mRNA into DNA fragments and counting how often each fragment is sequenced. In a typical experiment, RNA is extracted from cells, converted into complementary DNA (cDNA), fragmented, and prepared as a sequencing library. Millions of short reads are then generated and computationally aligned to a reference genome. The number of reads mapping to each gene provides an estimate of its expression level. In the following we focus on the analysis of RNA-seq data with the counts as a starting point.

16.2 Gene Length, Sequencing Depth, and Normalization of Read Counts

Raw read counts cannot be interpreted directly as expression levels. Two technical factors strongly influence the number of reads assigned to a gene:

- **Library size (sequencing depth):** Samples with more total reads have larger counts for all genes. Sequencing depth varies depending on the experimental setup and commonly differs by up to an order of magnitude between samples, see Fig. 16.1A.
- **Gene length:** Longer transcripts tend to generate more reads because there is more sequence to sample. For example, in *E. coli*, the lengths of different genes vary by almost two orders of magnitude, as shown in Fig. 16.1B.

As a result, raw counts must be normalized before expression levels can be meaningfully compared. A widely used normalization that accounts for both gene length and sequencing depth is *Transcripts Per Million (TPM)*.

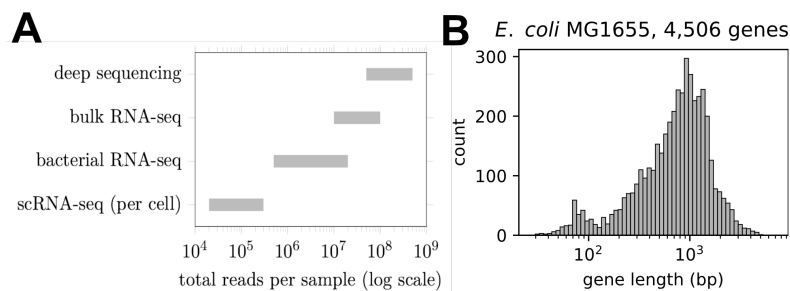


Figure 16.1: **Two major technical sources of variation in RNA-seq read counts.** (A) Total sequencing depth varies substantially across samples. (B) Gene length varies. In *E. coli* it spans nearly two orders of magnitude. Both effects must be corrected before gene expression can be compared across samples.

First, read counts are divided by transcript length,

$$\frac{X_g}{L_g},$$

where X_g is the read count and L_g is the length of gene g .

These length-corrected values are then rescaled so that they sum to one million within each sample:

$$\text{TPM}_g = \frac{X_g/L_g}{\sum_j X_j/L_j} \times 10^6.$$

By construction, TPM values correct for both gene length and sequencing depth.

i Info: Historical note: RPKM and its limitations

An earlier and still widely used normalization scheme is *Reads Per Kilobase per Million mapped reads (RPKM)*, which corrects read counts for both gene length and sequencing depth:

$$\text{RPKM}_g = \frac{10^9 X_g}{L_g N_s},$$

where X_g is the read count for gene g , L_g is its length (in bases), and N_s is the total number of mapped reads in sample s . At first glance, this normalization appears to make expression levels directly comparable across samples. However, RPKM normalizes by the total number of mapped reads, which reflects both transcript abundance and gene length. As such, long, highly expressed genes contribute disproportionately to this denominator. It is thus typically better to use TPMs.

16.3 Comparing Transcription Levels Within and Between Samples

Normalization schemes such as TPM provide a convenient way to summarize gene expression levels and allow for the relative comparison of expression levels. Consider as example a culture of *E. coli* growing steadily on glucose. The TPM of ribosomal genes is very high, while the TPM of beta-galactosidase (LacZ) is very low and barely expressed for growth on glucose, see Fig. 16.2A. Notably, expression levels vary strongly between genes, spanning several orders of magnitude (Fig. 16.2B). This spread is very common in transcriptomics data and indicates the strong variation with which cells express different genes.

Importantly, however, TPMs alone do not retain information about the underlying read counts. As such they do not directly quantify counting noise and statistical uncertainty. For example, two genes with similar TPM values may be supported by very different numbers of reads and therefore differ strongly in statistical precision.

16.4 Noise Floors from Counting Statistics

To evaluate measurement uncertainty and make statements about statistical significance, we must consider the statistical properties of count data. Let us first examine the read counts

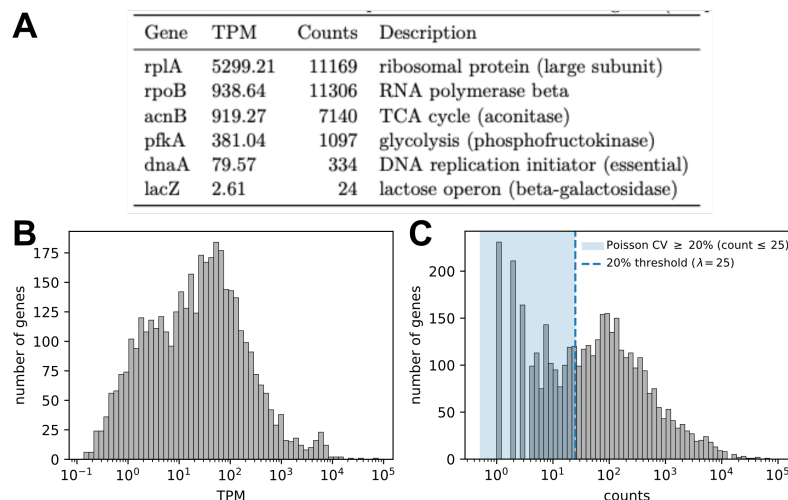


Figure 16.2: TPM values and raw counts for *E. coli* growing steadily on glucose. (A) TPM values for selected example genes. (B) Distribution of TPM values across all detected genes. (C) Distribution of raw read counts across all detected genes.

for our example condition, growth on glucose. The distribution of counts for the discussed example is shown in Fig. 16.2C.

From the previous section on counting noise, we expect uncertainty to be very high when counts are low and to decrease as the number of counts increases. For example, for a pure Poisson process the coefficient of variation (standard deviation compared to mean) scales as $\frac{1}{\sqrt{\langle X \rangle}}$, implying an uncertainty of about 20% when $\langle X \rangle \approx 25$.

Realizing this scaling, we can define a simple noise threshold. For example, expression levels supported by 25 or fewer counts are particularly noisy due to counting noise alone and we should not consider those. While such a threshold provides a useful first guideline it is somewhat arbitrary. For a more meaningful analysis of measurement noise and statistical uncertainty, we must go beyond such simple thresholds.

Towards a better analysis, let us first consider the variation between two biological replicates. For growth on glucose, the

expression levels of both replicates are shown in Fig. 16.3A. Because raw counts differ due to differences in sequencing depth, we normalize the data to counts per million reads (CPM) here before comparison. The scatter plot reveals strong overall agreement, but also noticeable variation for some genes.

Let's analyze this noise a bit more systematically. If counting noise were the only source of variability, we would expect Poisson statistics, with variance proportional to the mean. In real RNA-seq data, however, the variance is typically larger than predicted by a Poisson model, a phenomenon known as *overdispersion* (see previous section on counting noise).

Importantly, overdispersion is not just a mathematical fit parameter: it reflects both (i) genuine biological variability between replicates (cells are not identical), and (ii) additional technical variability beyond pure counting statistics (e.g. library prep and mapping variability).

To quantify this effect, we plot the observed variance against the mean expression level (Fig. 16.3B). Despite substantial scatter, a clear trend emerges: the variance increases with the mean and is well described by a negative binomial model with fitted overdispersion parameter α .

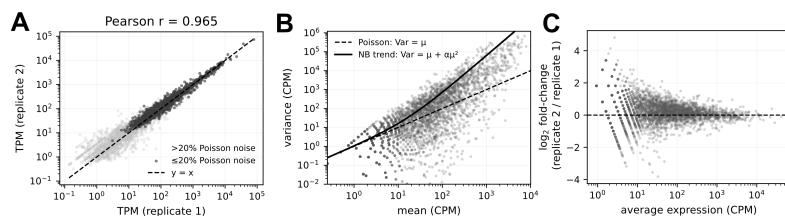


Figure 16.3: Noise analysis of RNA-seq replicates. (A) Scatter plot of CPM-normalized expression levels for two biological replicates grown on glucose. (B) Mean–variance relationship across genes. The dashed line indicates the Poisson expectation $\text{Var} = \mu$, and the solid line shows a fitted negative binomial trend including overdispersion.

In summary, there is substantial variation between replicates. Nevertheless, this variation follows systematic statistical patterns that can be described using counting noise and negative

binomial models. This provides the foundation for rigorous statistical tests of differential gene expression.

16.5 Differential Expression

How can we build on this noise consideration to analyze for possible differences in gene expression across experimental conditions. One way to perform this differential expression analysis is the use of the package **DESeq2**.

Conceptually, the differential expression analysis is simple. The key idea is to extract from the data an estimate of counting noise and biological variability, and then ask: under these noise assumptions, are observed differences statistically significant?

Specifically, DESeq2 tests the null hypothesis

$$H_0 : \beta_{1g} = 0,$$

that is, that there is no systematic difference in expression of gene g between two conditions.

To probe this null hypothesis, DESeq2 proceeds through several steps:

1. First, the noise level in the data is analyzed by assuming a negative binomial distribution, with parameters estimated from the data.
2. Given this noise model and the observed counts, DESeq2 fits a gene-wise generalized linear model and evaluates whether the condition effect (the log fold change) is distinguishable from noise.
3. The resulting p-values are adjusted to account for multiple-hypothesis testing when probing thousands of genes in parallel. This adjustment follows the Benjamini–Hochberg procedure to control the false discovery rate.

See the following box for more technical details.

i Info: DESeq2 under the hood: statistical model and hypothesis testing

DESeq2 models RNA-seq counts using a negative binomial distribution. That is, for gene g in sample s , the observed count X_{gs} is assumed to follow

$$X_{gs} \sim \text{NB}(\mu_{gs}, \alpha_g),$$

where

- μ_{gs} is the expected mean count,
- α_g is the gene-specific dispersion (overdispersion) parameter.

This implies

$$\text{Var}(X_{gs}) = \mu_{gs} + \alpha_g \mu_{gs}^2,$$

that is, the variance exceeds the Poisson variance when $\alpha_g > 0$. Conceptually, this overdispersion captures both biological variability between replicates and additional technical noise beyond pure counting statistics.

To estimate the dispersion from data, DESeq2 first estimates $\hat{\alpha}_g$ separately for each gene. Because these estimates are noisy, especially for low-count genes, DESeq2 applies shrinkage toward a global mean–dispersion trend, yielding stabilized dispersion values.

With the overdispersion estimated, DESeq2 then performs hypothesis testing, specifically probing the null hypothesis that the expression level of a gene does not differ across conditions.

Formally, the mean count is modeled using a generalized linear model (GLM),

$$\log(\mu_{gs}) = \beta_{0g} + \beta_{1g} \text{Condition}_s + \dots,$$

where β_{1g} represents the log fold-change between conditions.

A generalized linear model extends ordinary linear regression by allowing

- non-normal noise distributions (here: negative binomial),
- and a nonlinear link function (here: the logarithm).

This makes it possible to model count data while retaining a linear structure in the parameters.

DESeq2 tests the null hypothesis

$$H_0 : \beta_{1g} = 0,$$

which corresponds to no differential expression between conditions.

This is done using a Wald statistic, which compares the estimated coefficient to its standard error and yields a p-value. Intuitively, this is analogous to a t-test on the estimated log fold change: it asks whether the observed fold change is large compared to the noise estimated from the data.

Finally, p-values are adjusted using the Benjamini–Hochberg procedure to control the false discovery rate.

16.6 DESeq2 example

As example, let us compare the gene expression of wild type *E. coli* with a mutant strain lacking the transporter `ptsG`. As such, the mutant is not able to grow as fast on glucose as the wild type strain. The dataset contains replicate measurements for both strains, and Python (or R) conveniently provides DESeq2 results with adjusted p-values and estimated fold-changes in expression for all detected genes.

These results are commonly illustrated with a volcano plot, a scatter plot showing p-values versus fold change. For the discussed example, the volcano plot is shown in Fig. 16.4. This is a very convenient way to identify significantly changing genes which then can be probed further. For the shown example, there are for example 744 of 4506 genes identified which change significantly with a fold change of 2 or more. If we want to

understand more how the mutant affects expression we can start with these genes.

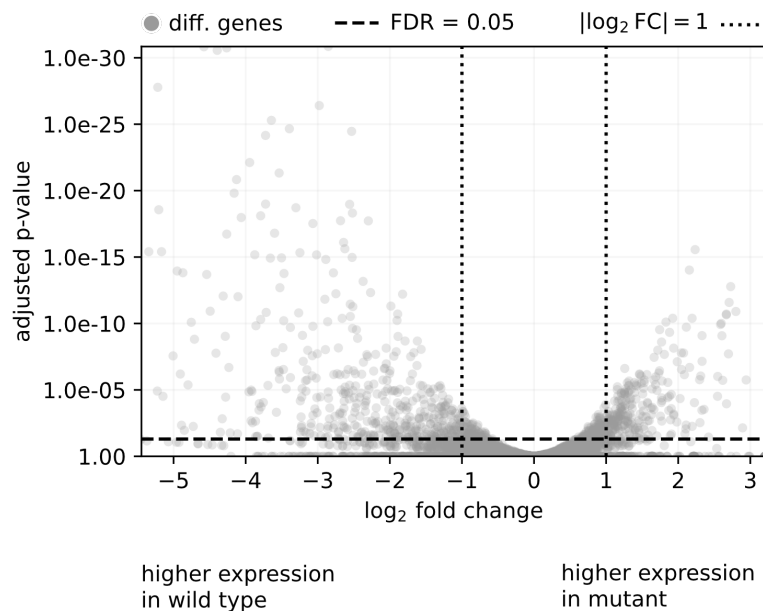


Figure 16.4: Differential gene expression between wild-type and glucose-uptake mutant strains. Each point represents one gene, plotted by $\log_2$ fold change and adjusted p-value. Genes with large and significant expression changes appear in the upper left and upper right regions.

16.7 From Differential Expression to Cellular Physiology

DESeq2 is an example of a modern analysis pipeline that integrates state-of-the-art statistical methods and is easy to apply using Python or R. However, as with all statistical tests, it is important to recognize that DESeq2 only probes for statistical significance within the specific modeling framework implemented. Statistical significance alone is neither sufficient nor necessarily indicative of biological relevance. Relying too strictly on a single analysis framework can even obscure important biological

insights.

To emphasize this challenge, we discuss particularly two points important when considering DESeq2 results.

- Statistically significant changes in expression do not necessarily reflect gene-specific regulatory mechanisms, highlighting the importance of careful interpretation.
- Biologically important changes may fail to reach statistical significance, for example due to limited statistical power or high variability.

To illustrate these points, we discussed two examples in the lecture. First, constitutively expressed genes, which lack active regulation of RNA polymerase recruitment, can nevertheless show strong expression changes across growth conditions. Second, during diauxic growth and the subsequent transition to acetate utilization, key metabolic genes required for growth on acetate do not always appear as significantly differentially expressed, even though their activity is essential for shaping the observed growth phenotype.

Overall, DESeq2 and differential expression analysis in general are powerful and essential tools for analyzing transcriptomic data. However, meaningful biological interpretation requires integrating these results with physiological knowledge, metabolic context, and complementary measurements.

16.8 The Challenge of Holistic Models

Connecting the Transcriptome to Cell States

With these caveats of differential expression analysis in mind, how can we go further and derive biological insights beyond the identification of significant changes? To explore this question, let us first consider an idealized scenario.

Suppose we have measurements of gene expression across different conditions and cellular states. Ideally, we would like to

formulate a holistic stochastic model that captures the relationship between transcriptional activity and cellular state so that we can predict one given the other.

To be more explicit, consider a dataset with expression measurements for M genes across N samples. As a first and simplest model, we might attempt to describe the relationship between transcription and a cellular state variable for different samples (e.g. growth rate) using a linear model (Fig. 16.5).

$$\begin{array}{c}
 \text{Gene 1} \quad \text{Gene 2} \quad \dots \quad \text{Gene M} \\
 \text{sample 1} \quad \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1M} \end{bmatrix} \\
 \text{sample 2} \quad \begin{bmatrix} x_{21} & x_{22} & \dots & x_{2M} \end{bmatrix} \\
 \vdots \\
 \text{sample N} \quad \begin{bmatrix} x_{N1} & x_{N2} & \dots & x_{NM} \end{bmatrix} \\
 \text{e.g., 20 samples} \quad \text{matrix X (N x M)}
 \end{array}
 \begin{array}{c}
 \text{effects} \\
 \begin{bmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_M \end{bmatrix} \\
 \text{vector } \beta
 \end{array}
 =
 \begin{array}{c}
 \text{cellular state} \\
 \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \\
 \text{vector y}
 \end{array}$$

x_{ij} = number of transcripts (TMP) in sample i , gene j
 β_j = effect of transcript j on disease state
 y_i = disease state of sample i

Figure 16.5: Schematic illustration of a linear regression model relating gene expression to cellular state. Each sample is represented by a vector of gene expression values, and regression coefficients quantify the contribution of each gene to the cellular phenotype.

Let $\mathbf{y} \in \mathbb{R}^N$ denote a vector describing the cellular state of each sample (e.g. growth rates), and let $\mathbf{X} \in \mathbb{R}^{N \times M}$ denote the matrix of gene expression values, where each row corresponds to a sample and each column to a gene. We then seek a coefficient vector β (of length M) such that

$$\mathbf{y} \approx \mathbf{X}\beta.$$

Here, the entries of β quantify how strongly the expression of each gene contributes to the cellular state. If we would know β we could predict the cell state for a given transcriptomic state.

In principle, the vector β could be estimated using standard linear regression by minimizing a suitable loss function (as we discussed before). However, a fundamental difficulty arises from the mismatch between the number of genes and the number of samples. While transcriptomes typically contain thousands of genes ($M \gg 1$), the number of samples is often much smaller, for example 10, 20, or 100 ($N \ll M$).

The problem is that in this setting, the regression problem is highly under determined with far more unknown parameters than information on the cell state. As a result, many different coefficient vectors β can fit the data equally well, and no unique solution exists (at least without any additional assumptions or constraints).

16.9 Dimensionality reduction

So what can we do? One option would be to return to differential expression analysis alone, as discussed before. However, there is another powerful approach, dimensionality reduction. The central question is this: Can we find a lower-dimensional description of the data that helps us overcome the underdetermination problem, while at the same time providing interpretable biological insights?

Apriori, it is not obvious that such an approach should work. The transcriptome contains thousands of genes, and in principle each gene could vary independently. However, in many biological systems this is not what happens. Instead, gene expression changes occur in highly coordinated ways.

As a powerful example, consider steady bacterial growth not only in a single condition (e.g. glucose), but across multiple growth conditions (e.g. different carbon sources). In principle, a transcriptome with $M = 4500$ genes could explore an enormous M -dimensional space of possible expression states. However, empirical data reveal that expression changes are structured and coordinated, effectively constraining the system to a much lower-dimensional manifold.

The dimensionality reduction is massive! To illustrate this idea, we here reconsider the proteomics dataset on bacterial growth introduced earlier, which measures the relative abundance of proteins during steady growth under different conditions. In principle, one could do a similar analysis with transcriptomics data but we used proteomics data here to use the dataset we already introduced.

16.10 Principal Component Analysis

There are several ways to systematically identify and construct lower-dimensional representations of high-dimensional data. One of the most widely used and mathematically straightforward methods is principal component analysis (PCA).

PCA identifies orthogonal directions in data space, called *principal components*, along which the variance of the data is maximal. The first principal component (PC1) captures the largest possible variance, the second component (PC2) captures the largest remaining variance subject to being orthogonal to PC1, and so on. See the box below for a more mathematical treatment. Geometrically, PCA corresponds to finding the best-fitting low-dimensional subspace that approximates the data cloud (see lecture).

We now apply PCA to the proteomics dataset. The dataset we used contains measurements from 7 different experimental conditions. Each condition is represented as a point in the space of principal components, as shown in Fig. 16.6A.

Remarkably, the first two principal components already explain about 53% and 22% of the total variance in the data, respectively. That is about 75% of the total variance, highlighting that a low-dimensional structure is present capable of describing describe change in proteome composition with much of the variability in proteome composition being captured by only two degrees of freedom.

To interpret this low-dimensional representation biologically, we next examine how the principal components relate to growth rate as physiological variables. There is a remarkable strong association between PC1 and cellular growth rate (Fig. 16.6B), suggesting that growth rate is a major organizing variable of the proteome.

To further understand this relationship, we analyze groups of proteins using functional annotations based on Clusters of Orthologous Groups (COGs), a protein classification scheme commonly used. Two major protein fractions emerge which change with principal component 1 (Fig. 16.6C). First, proteins involved in translation (COG J), including ribosomal proteins

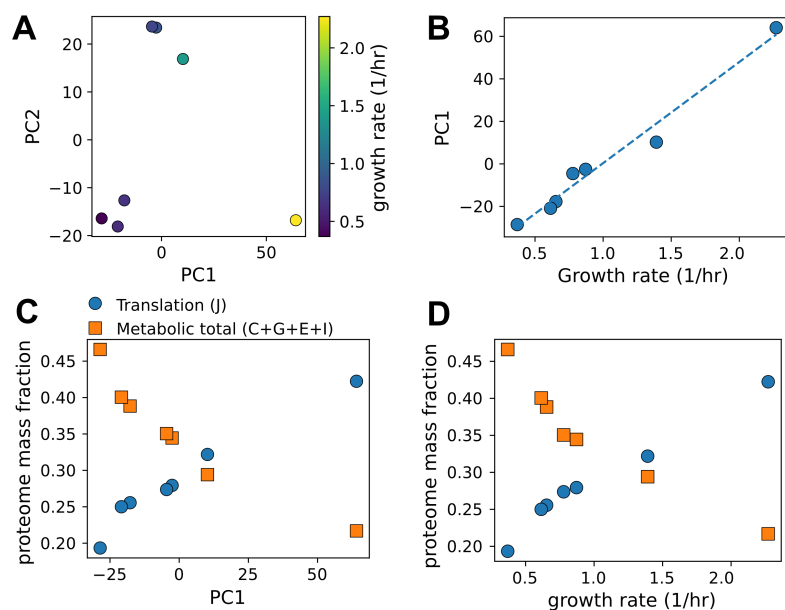


Figure 16.6: Principal component analysis of proteome composition across growth conditions. (A) Projection of experimental conditions onto the first two principal components. (B) Relationship between PC1 and cellular growth rate. (C) Relationship between PC1 and summed mass fractions of all proteins involved in translation and metabolism (defined by COG categories J and C,G, E, I respectively). (D) Same protein fraction plotted versus growth rate.

and associated factors, show strong systematic variation. Second, the combined abundance of metabolic proteins (COG C, G, E, and I) also varies prominently. Accordingly, these two proteome fractions change antagonistically with growth rate: fast-growing cells invest heavily in translation machinery, while slower-growing cells allocate a larger fraction of their proteome to metabolic functions (Fig. 16.6D).

In summary, this analysis demonstrates that PCA reveals a low-dimensional structure in proteome composition. A very small number of principal components (here two) captures a large fraction of the observed variability and these components correspond to biologically meaningful physiological programs. The realization of these protein sectors, hystorically first realized by a systematic change of ribosome fractions with growth rate lead to the formulation of low-dimensional resource allocation models. These models build on low-dimensional protein sector models and can describe many growth phenotypes with surprising accuracy.

i Info: The Mathematical Idea Behind Principal Component Analysis

Although PCA is often introduced geometrically, it is based on an elegant linear-algebraic framework. We describe here the mechanics of PCA, but we do not provide a proof for why this works. We recommend the tutorial by Jonathon Shlens¹

Step 1 - Organizing the data as a matrix: We begin by arranging the data in a matrix

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1M} \\ x_{21} & x_{22} & \cdots & x_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{NM} \end{pmatrix},$$

where each row represents one experimental condition (sample), and each column represents one gene or protein. Thus, N is the number of samples and M the number of measured variables. Before applying PCA, the columns of X are usually centered by subtracting their mean values.

Step 2 - Measuring how variables co-vary: From X , we construct the covariance matrix

$$C = \frac{1}{N-1} X^\top X,$$

which summarizes how strongly different genes or proteins vary together across conditions. Large positive entries in C indicate coordinated changes, while small values indicate weak coupling.

Step 3 - Finding principal directions: PCA consists of finding the eigenvectors $\mathbf{v}_k$ of the covariance matrix:

$$C\mathbf{v}_k = \lambda_k \mathbf{v}_k.$$

Each eigenvector defines a direction in gene-expression space. The corresponding eigenvalue λ_k measures how much variance lies along that direction.

Step 4 - Changing coordinates: The original data matrix X is then projected onto these directions:

$$Z = XV,$$

where V is the matrix whose columns are the eigenvectors in descending order of corresponding eigenvalues; thus eigenvectors are ordered by the amount of variance that direction explains in the data.

The new matrix Z contains the principal components. Its first column is PC1, the second is PC2, and so on.

Interpretation: PCA is therefore a coordinate transformation: it rotates the original gene-expression axes into new axes that are ordered by how much biological variability they capture. If only a few eigenvalues are large, the data effectively live in a low-dimensional subspace. This is the mathematical origin of dimensionality reduction.

¹Jonathon Shlens (2005), [A Tutorial on Principal Component Analysis](#)

17 Nonlinear dimensionality reduction and UMAP

In many datasets, variation is structured along curved or branching manifolds that cannot be well approximated by a linear subspace. In such cases, nonlinear methods can be useful. A widely used example is UMAP (Uniform Manifold Approximation and Projection), which is based on ideas from topology and manifold learning.

1. **Constructing a neighborhood graph:** UMAP starts by identifying, for each data point, its k nearest neighbors in the original high-dimensional space (typically using Euclidean or cosine distance). From these neighbors, UMAP builds a weighted graph in which:
 - nodes represent samples,
 - edges connect neighboring samples,
 - edge weights reflect how close two samples are.

These weights are chosen so that nearby points are connected with high probability, while distant points are weakly connected. This graph represents the local geometry of the data.

2. **Interpreting neighborhoods as fuzzy sets:** UMAP interprets each local neighborhood as a *fuzzy set*, meaning that membership is gradual rather than binary. Two points can be connected strongly, weakly, or not at all. All local fuzzy neighborhoods are then combined into a single global fuzzy graph that approximates the manifold structure of the data. This step encodes the assumption that the data lie on a low-dimensional manifold embedded in high-dimensional space.

3. **Low-dimensional embedding:** UMAP then seeks a low-dimensional representation (typically 2D or 3D) in which the distances between points reproduce the structure of the fuzzy graph as closely as possible. Formally, UMAP constructs a similar fuzzy graph in the low-dimensional space and optimizes the embedding by minimizing a cross-entropy loss between the high- and low-dimensional graphs. This optimization balances two competing goals:

- nearby points in high dimensions should remain close,
- unrelated points should be separated.

The result is an embedding that preserves local neighborhoods while arranging them coherently in low dimensions. For further details, discussion and examples for how to implement UMAP we recommend the documentation and accompanying tutorials at <https://umap-learn.readthedocs.io>.

Interpretation and limitations: Unlike PCA, UMAP does not produce linear axes or interpretable coefficients. The embedding coordinates have no direct physical meaning. UMAP is also stochastic: different runs may yield slightly different layouts unless random seeds are fixed. Because UMAP emphasizes local structure, distances between far-apart clusters are not necessarily meaningful. Apparent cluster separation or trajectories should therefore be interpreted cautiously and ideally validated by independent biological information.

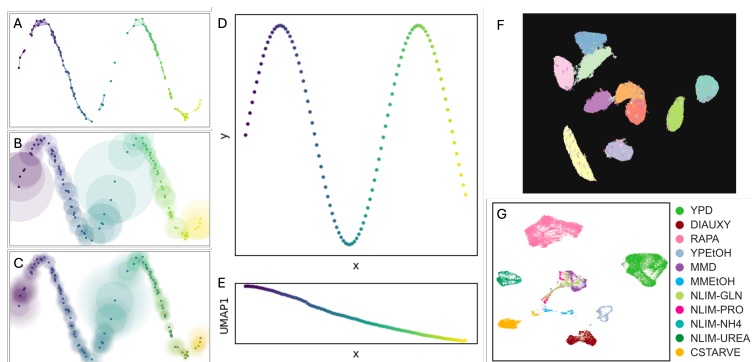


Figure 17.1: **UMAP as a dimensionality reduction tool.**

(A-C) demonstration of the logic of UMAP: a graph (as in A) is created based on neighboring points. A “fuzzy” ball is made around each data-point, and the fuzziness of the intersection between any two points translates to weights in a distance matrix. Data is then embedded into a new space based on the distance matrix, preserving short distances between nearby points in the original space. Figures are adapted from [this tutorial](#) (D-E) Implementation of UMAP on data samples from a sinusoidal curve; UMAP is able to capture this variation with just one dimension. (F) Example of UMAP clustering on images of numbers written in different fonts. UMAP is able to cluster similar numbers together. (G) UMAP on scRNA-Seq data of yeast grown in different media, from Jackson et al (*eLife*, 2020). Each point represents a single cell, and are colored by the media they were grown in.

18 Clustering Algorithms

As we’ve seen so far, high-dimensional (or other) data can often form sub-groups or clusters. Common instances of these include groups based on environmental condition, developmental state, cell type or state, samples from geographic locations, batch effects, etc. These show up intuitively in visualizations, but can actually be non-trivial to automatically group into clusters. There is a very large family of algorithms, which each have different use cases. Here we discuss the canonical examples of k-means in some depth, and provide a brief overview of other examples. Most standard clustering algorithms are easily implemented with standardized and efficient packages across common coding languages.

K-means clustering

The K-means algorithm is as follows:

1. Initialize centroids randomly. The number of centroids is pre-determined and corresponds to the number of clusters.
2. Assign each point to the nearest centroid.
3. Move each centroid to the mean location of the points in the cluster
4. Repeat until the algorithm converges and points no longer change assignments

Note that it is possible for this algorithm to get stuck in a “pathological” state, where centroids end up not at the center of intuitive (or optimal) clusters, but rather at points in between clusters of datapoints. This is conceptually similar to getting stuck at a local minimum in optimization methods we have discussed so far. To avoid this, it is useful to visualize the clusters to verify that they make sense intuitively, to repeat

the algorithm with a few iterations of initial locations for the cluster centroids.

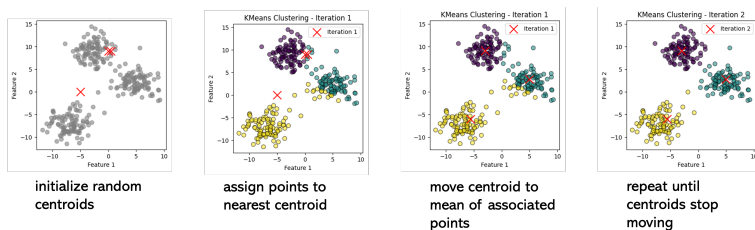


Figure 18.1: **K-means clustering:** The images above (left to right) visualize data that intuitively fall into three clusters, and show the progression across iterations, where centroids are randomly initialized, each point is assigned to the nearest centroid, the centroid is moved to the mean of the points, and so on until the algorithm converges and points do not change assignment.

Beyond k-means, there are many other clustering methods that can handle situations where its assumptions are too restrictive. Hierarchical clustering, for example, builds a tree of clusters by either repeatedly merging small groups into larger ones or splitting large groups into smaller ones, allowing us to view the data at different levels of detail. Density-based methods such as DBSCAN group together points that lie in crowded regions and mark isolated points as noise, which makes them useful for finding irregularly shaped clusters. Another approach models the data as coming from a mixture of distributions and uses the Expectation–Maximization algorithm to estimate which points likely belong to which group, allowing for partial membership rather than forcing each point into a single cluster. There are also graph-based and spectral methods that rely on measuring how similar points are to one another to uncover more complex patterns. The figure below adapted from [this blog](#) shows how different algorithms perform in different contexts.

Finally, take care when interpreting the results of clustering. Many clustering algorithms involve some degree of randomness—for example, different initial starting points can lead to different solutions—so repeated runs may not produce identical group-

ings. In addition, results can change substantially depending on choices such as the number of clusters, distance measure, or tuning parameters (for instance, the neighborhood size in DBSCAN). For this reason, it is good practice to check the stability of clusters under different settings and to consider whether the solution is consistent with domain knowledge.

It is also important to remember that most clustering algorithms will produce clusters even if the data do not contain any meaningful grouping. The presence of clusters in the output does not by itself prove that a strong or “real” cluster structure exists; apparent groupings may arise from noise, scaling choices, or high dimensionality. Whenever possible, clustering results should be supported with additional evidence, such as external validation, replication on new data, or quantitative measures of stability.

Finally, be cautious when clustering points based on the results of non-linear dimensionality reduction methods such as UMAP or t-SNE. These methods are designed primarily for visualization and can distort distances or exaggerate separation between groups in two or three dimensions. Clusters that appear well separated in a low-dimensional plot may not be as distinct in the original feature space. Whenever feasible, clustering should be performed in the original space (or a carefully chosen representation), with reduced-dimensional embeddings used mainly as an aid to interpretation rather than as definitive evidence of discrete groups.

Part VIII

Week 8: Dynamical systems and stochastic processes (optional)

19 Differential equation and the modeling of dynamical systems

i **Optional topic.** Weeks 8 and 9 (dynamical systems and nonlinear dynamics) are an optional part of the course.

Omics measurements provide detailed descriptions of cellular states, but they capture only momentary snapshots. Yet cells are not static systems, molecular abundances continuously change through production, degradation, and growth. To understand what we measure, we must therefore adopt a dynamical perspective and describe biological processes explicitly in time. The mathematical tool for this is differential equations, which we introduce in this section.

19.1 A flux perspective to explore biology

To build dynamical models, a useful and unifying perspective is to view biology in terms of *fluxes*, that is, rates at which material, energy, and information flow through biological systems. At every level of organization, living systems are governed by such fluxes. Within cells, metabolic fluxes convert nutrients into biomass; transcriptional fluxes produce RNA; translational fluxes generate proteins. At the level of populations and ecosystems, fluxes determine composition through birth, death, competition, and trophic transfer. At even larger scales, biological activity drives global biogeochemical cycles, with elements such as carbon, nitrogen, and phosphorus continuously circulating between organisms and the environment.

In each case, observed abundances are not fixed properties but the outcome of competing processes unfolding in time. Molecules, cells, and organisms are constantly produced and removed. As a result, biological quantities change according to a simple and remarkably general principle:

$$\text{change} = \text{production} - \text{removal}.$$

19.2 Differential Equations to model fluxes

To describe biological fluxes mathematically, we use differential equations. For a time-dependent quantity $x(t)$, the derivative

$$\frac{dx}{dt}$$

denotes how fast x is changing at a given moment in time t . Differential equations state how this rate of change is determined by the state of the system. For example, production rates may depend on substrate availability, enzyme levels, or regulatory signals, while removal rates may depend on degradation machinery or growth.

As a result, biological models are often written in the form

$$\frac{dX}{dt} = P(X, t) - R(X, t),$$

where $P(X, t)$ represents the total production rate and $R(X, t)$ the total removal rate, both of which are functions of the system state X (and possibly time). This is a differential equation. For concrete applications, these functions need to be specified and parametrized based on the specific biology studied. But once set, these differential equations predict how the system evolves over time.

19.3 Exponential Growth

We illustrate these ideas with a simple but biologically very important example: exponential growth. Consider as example

the bacterial growth scenario studied in our reference dataset. If each cell or organism divides at a constant rate per unit time, then the rate of increase of the population is proportional to its current size. This leads directly to the differential equation for the population size $N(t)$:

$$\frac{dN}{dt} = \lambda N,$$

Here, λ is commonly called the growth rate.

This differential equation has a simple analytical solution,

$$N(t) = N_0 e^{\lambda t},$$

where N_0 is the population size at time $t = 0$, specifying the initial condition of the system. This initial condition and the growth rate uniquely determine the future trajectory of $N(t)$, see Figure 19.1A for an example.

The relevance of exponential growth in biology is hard to overstate. Many fundamental biological processes are autocatalytic: cells and organisms make more of themselves, so larger populations grow faster. The same logic applies at the molecular level: more ribosomes enable faster translation, which in turn produces more ribosomes. The same equation with negative λ also describes exponential decay, making it relevant for mRNA degradation, cell death, or dilution through cell division.

19.4 Logistic Growth and Steady States

In real biological environments, exponential growth cannot continue indefinitely. Nutrients become limiting, waste products accumulate, space is finite, and physical and physiological constraints slow reproduction. As populations grow, they increasingly compete with themselves for shared resources. A minimal model that incorporates such effects is logistic growth defined by the following differential equation:

$$\frac{dN}{dt} = \mu N - \mu \frac{N^2}{K}.$$

Here K is called the carrying capacity of the environment. When $N \ll K$, the first term dominates and growth is nearly exponential. As N increases and approaches K , the second term becomes more important, growth slows, and the population stabilizes. Figure 19.1B illustrates this dynamics.

We logistic equation nicely illustrates an important general concept: the dynamics leads to a stable state. We can identify this behavior directly from the equation without solving it explicitly. At a steady state, the population no longer changes, so we require $dN/dt = 0$, which gives

$$\mu N^* - \mu \frac{(N^*)^2}{K} = 0.$$

This equation has two solutions: $N^* = 0$ (extinction) and $N^* = K$ (the carrying capacity). For growing populations, $N^* = K$ represents a stable steady state toward which the system evolves. Steady states emphasize the dynamic balances that often govern biological systems. Many biological variables, from metabolite concentrations, to mRNA, protein levels, and populations, remain approximately constant over time not because they are static, but because production and removal processes are finely balanced.

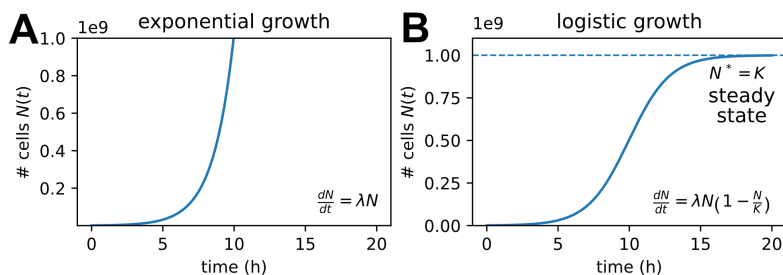


Figure 19.1: **Canonical growth models illustrating differential equations.** (A) Exponential growth trajectory. (B) Logistic growth trajectory. The system nicely illustrates the approach of a steady state $N = K$ (dashed line).

19.5 More complex differential equations and numerical solutions

In practice, most biologically relevant differential equation models are substantially more complex than the simple growth equations discussed so far. Realistic models often involve many interacting variables, nonlinear feedback, regulatory mechanisms, and multiple timescales. For such systems, explicit analytical solutions are rarely available. However, we can compute their solutions numerically. Numerical methods approximate the continuous dynamics by stepping forward in small time increments and updating the system state iteratively. With modern computational tools, this approach is efficient, reliable, and easy to implement, even for large and highly nonlinear models.

Info: Obtaining numerical solutions

The basic idea to numerically solve a differential equation and predict trajectories is to discretize time into small steps Δt and approximate the dynamics as a dynamics in discrete timesteps,

$$x(t + \Delta t) \approx x(t) + \frac{dx}{dt} \Delta t.$$

Starting from an initial condition, this update rule is applied repeatedly to generate an approximate trajectory. Different algorithms, such as Runge–Kutta methods, follow this idea and refine it to minimize the accumulation of numerical errors. In practice, numerical solvers are conveniently implemented in scientific libraries such as `scipy.integrate` in Python, making simulation of complex biological systems quite easy. However, it is important to keep in mind that numerical errors can occur and it is important to check results for consistency.

The examples of exponential and logistic growth discussed above describe the time evolution of a single variable. In realistic biological systems, however, we often need to track many interacting quantities simultaneously, leading to systems of coupled differential equations. In addition, many processes vary not

only in time but also across space. Nutrient concentrations diffuse through tissues, morphogen gradients form during development, and populations spread across landscapes. Describing such spatial dynamics requires extending our models to partial differential equations, which connect derivatives in space and time.

As an important biological example of a multi-variable dynamical system, we now turn to the coupled dynamics of transcription and translation, linking mRNA and protein levels.

20 The Dynamical Relation Between mRNA and Proteins

i **Optional topic.** Weeks 8 and 9 (dynamical systems and nonlinear dynamics) are an optional part of the course.

To demonstrate the power of differential equations, we now apply this framework to one of the most fundamental biological processes: the dynamical relationship between gene expression, mRNA, and protein levels. Understanding this relationship better is essential to interpretate transcriptomic and proteomic data.

As a concrete example, consider the previously discussed steady growth of *E. coli* on glucose. Across genes and expression levels, protein abundances are strongly tied to mRNA abundance, as illustrated by the relationship between relative transcript and protein fractions derived from omics data (Fig. 20.1). Importantly, the shown correlation represents only a static snapshot of a highly dynamic process. To understand how such correlation patterns arise and to move toward a dynamical description, we turn to the central dogma of molecular biology:

$$\text{DNA} \rightarrow \text{RNA} \rightarrow \text{protein}.$$

Rather than viewing this as a simple flow chart, we now setup a dynamical model of the central dogma. Consider a specific gene and let $m(t)$ denote the number of mRNA molecules and $p(t)$ the number of protein molecules in a cell for that specific gene. The fundamental processes of transcription, translation, and degradation can then be described by two coupled differential equations.

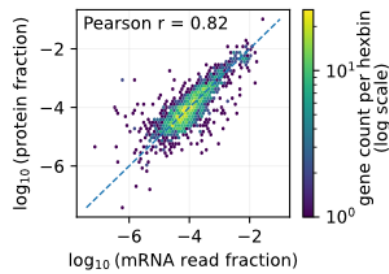


Figure 20.1: **Relationship between mRNA and protein abundance in *E. coli*.** Hexbin plot showing relative protein abundance versus relative mRNA abundance across genes during steady growth. Color indicates the number of genes per bin. The overall correlation reflects the coupling between transcription and translation predicted by the central-dogma model. Deviations from the identity line (blue) arise from additional regulatory processes, including gene-specific translation efficiencies, differential protein stability, post-transcriptional regulation, and measurement noise.

We begin with mRNA dynamics. The change in mRNA abundance due to transcription and degradation is given by

$$\frac{dm}{dt} = G\kappa - \delta_{\text{mRNA}} m.$$

Here, G denotes the copy number of the gene on the chromosome, and κ is an effective transcription rate (mRNA per unit time) describing how rapidly RNA polymerases produce transcripts from this gene. The parameter δ_{mRNA} is the mRNA degradation rate. The degradation term is proportional to m because each individual mRNA molecule has a certain probability per unit time to be degraded. When more mRNA molecules are present, more degradation events occur per unit time.

We can similarly describe the dynamics of protein molecules p :

$$\frac{dp}{dt} = \gamma m - (\delta_p + \lambda) p.$$

Here, γ is the translation rate (proteins produced per mRNA per unit time), and δ_p is the protein degradation rate. In growing cell cultures, protein numbers are also reduced by cell division. This process, known as dilution, scales with the growth rate λ and is often more important than active degradation in fast-growing cells.

Together, these differential equations provide a simple model for the coupled dynamics of transcription and protein synthesis. Notice their hierarchical structure: protein dynamics depend on mRNA, while mRNA dynamics are independent of protein. This reflects the directionality of the central dogma. When we later consider gene regulation, this strict one-way dependence will be relaxed, giving rise to feedback loops and more complex dynamical behavior.

20.1 Estimating Gene Expression Parameters in Cells

As with every differential equation model, connecting the central-dogma model to real biology requires specifying realistic values

for all model parameters. Importantly, these parameters are not abstract fitting constants, but correspond to measurable biological quantities, made possible by more than a century of methodological advances and elegant experiments in molecular biology. In the following we focus on *E. coli* numbers and specifically the gene *lacZ*, providing short order of magnitude estimations for each parameter.

Transcription rate κ .

In *E. coli*, RNA polymerases operate with an elongation speed of approximately 60 nt/s when synthesizing mRNA. The time required to complete a transcript therefore depends on gene length. For *LacZ*, for example, transcription takes about

$$\tau_{\text{trs}} \approx 0.85 \text{ min.}$$

The maximal transcription rate is given by the inverse of this,

$$\kappa_{\text{max}} \approx \frac{1}{\tau_{\text{trs}}} \approx 70 \text{ h}^{-1}.$$

In real cells, however, transcription rates are typically much lower. Initiation is limited by promoter strength and by the availability of RNA polymerase. As a very rough argument, consider that about 1000 RNA polymerases are shared to transcribe roughly 4000 genes. The effective transcription capacity per gene is therefore expected to be several-fold lower than κ_{max} . We therefore use in our estimation and effective value

$$\kappa \sim 17 \text{ h}^{-1}$$

as an order-of-magnitude estimate.

Gene copy number G .

The factor G accounts for gene copy number, which can exceed one in fast-growing cells due to multi-fork replication (cells maintain several copies of the chromosome which they replicate in parallel to make sure chromosome replication does not become a major bottleneck in growth). Here, we consider a single-copy situation only with $G \approx 1$.

mRNA degradation rate δ_{mRNA} .

Radiolabeling and transcriptional shutoff experiments have reported mRNA half-lives in *E. coli* of approximately

$$\tau_{1/2} \approx 5 \text{ min.}$$

For order-of-magnitude estimates, the degradation rate follows as a reciprocal of this timescale,

$$\delta_{\text{mRNA}} \approx \frac{1}{\tau_{1/2}} \approx 12 \text{ h}^{-1}.$$

Translation rate γ .

Ribosomes synthesize proteins at a rate of about 20 AA/s. For LacZ, this corresponds to a time to synthesize the protein of roughly

$$\tau_{\text{tsl}} \approx 1.3 \text{ min.}$$

The throughput of a single translating ribosome is therefore approximately

$$\gamma \approx \frac{1}{\tau_{\text{tsl}}} \approx 46 \text{ h}^{-1}.$$

Protein degradation rate δ_p .

Most proteins in steadily growing *E. coli* are relatively stable and have long half-lives, often on the order of

$$\tau_{1/2, \text{prot}} \approx 30 \text{ h}$$

or longer. The corresponding degradation rate is therefore small,

$$\delta_p \approx \frac{1}{\tau_{1/2, \text{prot}}} \approx 0.03 \text{ h}^{-1}.$$

Growth rate λ (dilution).

For growth on glucose, a typical doubling time is

$$T_d \approx 1 \text{ h.}$$

This corresponds to a growth rate of order

$$\lambda \approx \frac{1}{T_d} \approx 0.45 \text{ h}^{-1}.$$

In this regime, dilution by growth dominates protein removal,

$$\lambda \gg \delta_p,$$

so active protein degradation can often be neglected to first order.

Table 20.1: **Order-of-magnitude parameter estimates for fast-growing *E. coli*.**

Parameter	Meaning	Estimate (h ⁻¹)	How estimated
λ	growth rate (dilution)	~ 0.5	doubling time $T_d \approx 1$ h
δ_{mRNA}	mRNA degradation	~ 12	mRNA half-life $t_{1/2} \approx 5$ min
δ_p	protein degradation	~ 0.03	protein half-life $t_{1/2} \approx 30$ h
γ	translation (effective)	~ 46	LacZ synthesis time $t_{\text{synth}} \approx 1.3$ min
κ_{max}	max transcription throughput	~ 70	RNAP speed ~ 60 nt/s, LacZ length
κ	effective transcription rate	~ 17	RNAP limitation + initiation efficiency
G	gene copy number	1 or larger	replication state / locus position

20.2 Steady-State Expression Levels

With all parameters specified, we can now analyze how transcription, translation, and degradation determine protein abundance. We begin by considering steady-state conditions, in

which production and removal are balanced for both mRNA and protein.

Mathematically, steady state corresponds to $dm/dt = dp/dt = 0$. Setting $dm/dt = 0$ gives

$$m^* = \frac{G\kappa}{\delta_{\text{mRNA}}},$$

and setting $dp/dt = 0$ with $m = m^*$ yields

$$p^* = \frac{\gamma m^*}{\delta_p + \lambda} = \frac{\gamma G\kappa}{\delta_{\text{mRNA}}(\delta_p + \lambda)}.$$

This expression has a clear biological interpretation: under steady-state conditions, protein abundance is determined by total synthesis divided by total removal. Transcription, translation, degradation, and growth together set the mRNA and protein levels through these balances.

Using the order-of-magnitude parameter values estimated above, we estimate for a steady state

$$m^* \sim 1 \quad \text{and} \quad p^* \sim 10^2,$$

Roughly one mRNA molecule and on the order of one hundred protein molecules per gene are present in a cell. While the exact values vary substantially across genes and conditions, the order of magnitudes have been confirmed in cells and highlight an important general feature of gene expression: mRNA copy numbers are often very small, whereas protein copy numbers are typically much larger.

This separation of scales has important consequences. Low mRNA copy numbers make transcription intrinsically noisy, while higher protein copy numbers provide partial buffering and stability at the protein level. We will come back to this point when analyzing stochastic processes.

Notably, for the steady growth of *E. coli* on glucose discussed earlier, many parameters in this expression, including γ , δ_{mRNA} , δ_p , and λ , are approximately constant and shared across genes. In this regime, most gene-to-gene variation in protein abundance arises primarily from differences in transcriptional activity, encoded in $G\kappa$. As a result, the model predicts the

near-proportional relationship between mRNA and protein levels observed in the data (Fig. 20.1, blue line). Deviations from this simple relationship indicate additional layers of regulation, such as gene-specific translation efficiencies, differential protein stability, or post-transcriptional control.

Taken together, these steady-state relations define an important baseline or null model for gene expression. When most kinetic parameters are shared across genes, variation in protein abundance is dominated by transcriptional activity, and mRNA and protein levels are expected to scale proportionally. This provides a simple reference point against which more complex regulatory effects can be identified.

20.3 Response Dynamics and Timescales

The same framework also naturally extends beyond steady-state conditions to describe how expression levels change in time. We here use the model to illustrate how mRNA and protein levels respond when gene expression is switched abruptly on or off at time $t = 0$. A numerical simulation of such a scenario is shown in Fig. 20.2.

When transcription is activated, mRNA levels rise rapidly, followed more slowly by protein accumulation. Conversely, when transcription is shut off, mRNA decays quickly, while protein levels decline more gradually. These distinct responses reflect the different characteristic timescales of RNA and protein turnover. In many cells, mRNA half-lives are typically on the order of minutes or tens of minutes, whereas many proteins are removed primarily by dilution through growth. As a result, transcript levels track regulatory changes much more rapidly than protein levels, which integrate cellular history over longer times.

These differential timescales are not merely a modeling detail, they impose fundamental physiological constraints. Because proteins accumulate and decay slowly, cells cannot instantaneously reconfigure their proteome in response to environmental change. This limitation helps explain phenomena such as apparent overcapacity in certain protein types and anticipatory synthesis of proteins before they are strictly required. In this

way, simple dynamical considerations already provide insight into broader bacterial growth strategies and resource allocation principles.

So far, we have considered deterministic and linear models of gene expression. In reality, regulatory feedback and molecular noise introduce nonlinearity and variability, leading to richer dynamical behaviors. These effects will be explored in the following lectures.

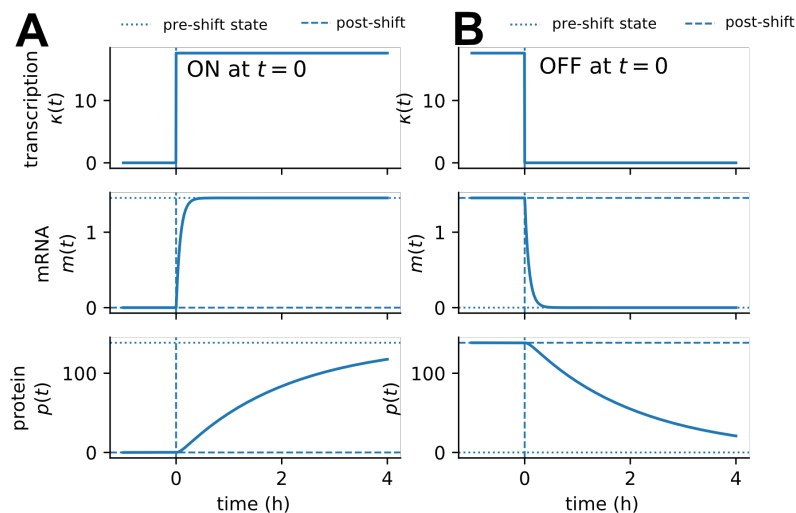


Figure 20.2: **Gene expression ON/OFF dynamics.** Step-wise change in transcription to on (A) or off (B) at $t = 0$. Vertical dashed lines mark the switch time ($t = 0$). Horizontal dashed and dotted lines indicate the pre-shift and post-shift steady states, respectively. The simulations highlight the separation of timescales: mRNA responds rapidly on the timescale $1/\delta_{\text{mRNA}}$, whereas protein changes more slowly on the effective removal timescale $1/(\delta_p + \lambda)$, often dominated by dilution in fast growth.

21 Stochastic Processes

i Optional topic. Weeks 8 and 9 (dynamical systems and nonlinear dynamics) are an optional part of the course.

Differential equations, such as those introduced earlier to model the central dogma, are powerful tools for analyzing dynamical systems. However, by construction, models based on differential equations are deterministic. That is, once the initial condition is specified, the trajectory is entirely determined by the governing equation. For a fixed initial condition, there is no variability in the predicted dynamics.

Real biological processes, however, are noisy and fluctuate in time. This naturally raises the question of how appropriate a purely deterministic description is for modeling biological systems, and whether a probabilistic framework might sometimes be more suitable. This is not merely a technical complication; stochasticity is a fundamental feature of biology, and in many cases it plays a central role in how cells function and make decisions.

As an example, consider the transcription and translation dynamics of a gene in a bacterial cell, as discussed in the previous section. We estimated typical mRNA copy numbers to be very low, on the order of one molecule per cell. In this regime, dynamics are not smooth and continuous but instead consist of discrete stochastic events. Relative fluctuations are large, as a change of just one or two molecules represents a substantial fraction of the mRNA copy number. Consequently, gene expression in bacteria is inherently stochastic, and this stochasticity propagates through translation, leading to variability in protein levels.

To analyze such situations more systematically, we now introduce the basic concepts of stochastic processes.

21.1 The Poisson Process

We start with a simple stochastic process. We study a random variable $N(t)$ that increases by one whenever a new event occurs, with the key characteristic that events occur with a constant probability per unit time, denoted by the rate k . Mathematically, the probability that an event occurs during a short time interval Δt is

$$\mathbb{P}(\text{one event in } \Delta t) = k \Delta t,$$

while the probability of two or more events during Δt is negligible for sufficiently small Δt . This assumption defines a Poisson process, and we will see below why it is called that way.

To describe the occurring dynamical process mathematically, we introduce the probability $P(N, t)$ that the system is in state N at time t . Events increase N by one, so probability flows from state $N - 1$ into N . The time evolution of $P(N, t)$ is therefore governed by the equation

$$\frac{dP(N, t)}{dt} = k P(N - 1, t) - k P(N, t).$$

The first term represents transitions into state N , while the second term represents transitions out of state N . This equation is an example of a master equation.

In general, master equations are difficult to solve analytically. However, we can simulate stochastic trajectories numerically. Instead of computing $P(N, t)$ directly, we repeatedly generate sample paths by letting the computer draw random variables to determine when the next event occurs and updating N accordingly. To do so, we must know the probability distribution of the waiting time until the next event.

For a Poisson process, the waiting time τ between events follows an exponential distribution,

$$p(\tau) = k e^{-k\tau},$$

as derived in the box below. Drawing τ from this distribution and updating the system step by step yields one realization of the stochastic trajectory.

i Info: Waiting-time distribution of a Poisson process

We derive the probability distribution of the waiting time τ until the next event occurs in a Poisson process with constant rate k .

Let $P_{ne}(t)$ denote the probability that *no event* has occurred up to time t . During a short interval Δt , the probability that no event occurs is

$$1 - k\Delta t$$

Therefore,

$$P_{ne}(t + \Delta t) = P_{ne}(t) (1 - k\Delta t).$$

Rearranging gives

$$\frac{P_{ne}(t + \Delta t) - P_{ne}(t)}{\Delta t} = -k P_{ne}(t).$$

Taking the limit $\Delta t \rightarrow 0$ yields the differential equation

$$\frac{dP_{ne}}{dt} = -k P_{ne}(t).$$

This equation has the solution

$$P_{ne}(t) = e^{-kt},$$

using the initial condition $P_{ne}(0) = 1$ (no even occurred at time $t = 0$ by definition).

What is the probability that the next event occurs exactly at time τ ?

Intuitively, two things must happen:

1. No event occurs before time τ .
2. An event occurs immediately after time τ .

The probability that no event has occurred before τ is

$$P_{ne}(\tau) = e^{-k\tau}.$$

The probability that an event occurs in the next small interval $[\tau, \tau + \Delta t]$ is approximately $k\Delta t$.

Therefore, the probability that the event occurs between τ and $\tau + \Delta t$ is

$$P_{ne}(\tau) k\Delta t.$$

Dividing by Δt and taking the limit $\Delta t \rightarrow 0$ gives the probability density

$$p(\tau) = ke^{-k\tau}.$$

Thus, the waiting time between events in a Poisson process follows an exponential distribution.

An example of stochastic trajectories generated in this way is shown in Fig. 21.1A. Each trajectory (or realization of the stochastic process) consists of discrete jumps occurring at random times. Importantly, repeating the simulation produces different trajectories, reflecting the inherent stochasticity of the process.

By generating many independent realizations, we can explore the statistical properties of the system. For example, we can examine the distribution of $N(t)$ at a fixed time point, as shown in Fig. 21.1B.

Notably, this distribution follows a Poisson distribution,

$$P(N, t) = \frac{(kt)^N}{N!} e^{-kt},$$

(black line in Fig. 21.1B), which explains the name *Poisson process*: not only are the waiting times exponentially distributed, but the total number of events occurring in a fixed time interval follows a Poisson distribution.

Intuitively, this result can be understood by dividing time into many small intervals Δt . In each interval, an event occurs with probability $k\Delta t$, analogous to repeatedly flipping a highly

biased coin. The total number of events in time t then follows a binomial distribution, which converges to a Poisson distribution in the limit $\Delta t \rightarrow 0$.

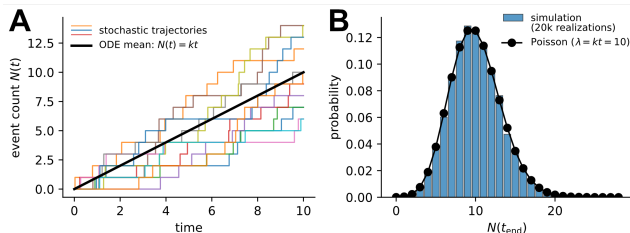


Figure 21.1: **Poisson process** (A) Example stochastic trajectories of a Poisson process with constant event rate $k = 1$. Each trajectory consists of discrete jumps occurring at random times. Repeating the simulation yields different realizations, reflecting intrinsic stochasticity. (B) Distribution of the number of events $N(t)$ at a fixed time point $t = 10$ across many independent trajectories. It follows a Poisson distribution (solid line) with mean $\langle N(t) \rangle = kt$.

21.2 The Gillespie Algorithm

To study biologically relevant situations, we must go beyond a single Poisson process. The Poisson process resembles, in the deterministic limit, the simple differential equation

$$\frac{dN}{dt} = k,$$

which describes a constant production rate. Real biological systems, however, typically involve multiple interacting processes whose rates depend on the current state of the system. For example, in exponential growth we have

$$\frac{dN}{dt} = \lambda N,$$

so the rate of events depends on the current population size. Similarly, in gene expression, transcription, translation, and

degradation occur simultaneously, and their rates depend on molecule numbers.

To formalize this idea, we introduce the concept of a *propensity*. The propensity $a_j(x)$ of reaction j is the probability per unit time that this reaction occurs, given the current state x of the system. Importantly, propensities depend on molecule numbers. For example, the degradation of mRNA occurs at a rate proportional to the number of mRNA molecules present. If twice as many molecules are present, twice as many degradation events are expected per unit time. In this sense, propensities represent the stochastic analogue of reaction rates in differential equations.

To simulate such systems stochastically, we use the Gillespie algorithm, introduced by Daniel Gillespie. This algorithm provides an exact simulation method for chemical master equations. The core idea is simple: given the current state of the system, we calculate the rates at which all possible reactions can occur, and then use two random numbers to determine (i) when the next reaction occurs and (ii) which reaction it is. See the box for more details.

i Info: Gillespie Algorithm

The Gillespie algorithm provides an exact simulation method for stochastic chemical reaction systems described by a master equation.

Consider a system with reactions $j = 1, \dots, R$. Each reaction has a propensity $a_j(x)$, defined as the probability per unit time that reaction j occurs given the current state x of the system.

Step 1: Compute propensities.

For the current state, compute all propensities a_j and their sum

$$a_0 = \sum_{j=1}^R a_j.$$

Step 2: Draw time to next reaction.

The waiting time τ until the next reaction is drawn from

an exponential distribution with rate a_0 :

$$\tau = \frac{1}{a_0} \ln\left(\frac{1}{u_1}\right),$$

where $u_1 \in (0, 1)$ is a uniform random number.

Step 3: Choose which reaction occurs.

Select reaction j such that

$$\sum_{i=1}^{j-1} a_i < u_2 a_0 \leq \sum_{i=1}^j a_i,$$

where $u_2 \in (0, 1)$ is a second independent uniform random number. This selects reaction j with probability a_j/a_0 .

Step 4: Update the system.

Update the system state according to the stoichiometry of reaction j and advance time by τ .

Step 5: Repeat.

Return to Step 1 and iterate.

This procedure generates statistically exact trajectories of the underlying chemical master equation and is the standard method for simulating low-copy-number stochastic systems such as gene expression.

21.3 Stochastic Gene Expression: The Central Dogma Revisited

To illustrate how stochastic simulation can be applied to a biologically relevant system, we now revisit the central dogma model previously introduced in deterministic form. Specifically, we described earlier the mRNA and protein dynamics using differential equations,

$$\frac{dm}{dt} = G\kappa - \delta_m m, \quad \frac{dp}{dt} = \gamma m - (\delta_p + \lambda)p,$$

where κ is the transcription rate, δ_m the mRNA degradation rate, γ the translation rate per mRNA, and δ_p and λ the protein degradation and dilution rate due to growth.

In the stochastic formulation, we no longer track continuous concentrations, but instead integer molecule numbers m and p . Changes occur through four discrete reaction events:

Transcription:	$m \rightarrow m + 1$ with propensity $G\kappa$
mRNA degradation:	$m \rightarrow m - 1$ with propensity $\delta_m m$
Translation:	$p \rightarrow p + 1$ with propensity γm
Protein removal:	$p \rightarrow p - 1$ with propensity $(\delta_p + \lambda)p$

These reactions define a chemical master equation, which we simulate using the Gillespie algorithm. That is, we determine the total rate at which any of the four reactions occurs by summing their propensities, then use one random number to determine the time of the next event and a second random number to determine which reaction occurs.

Stochastic trajectories.

Simulating this system produces stochastic trajectories for $m(t)$ and $p(t)$. As an example, we consider induction of gene expression starting from low copy numbers. The resulting trajectories are shown in Fig. 21.2A and B for mRNA and protein copy numbers, respectively. Each line represents one realization of the stochastic process. Unlike the smooth deterministic solution, these trajectories exhibit discrete jumps and fluctuations.

Importantly, the trajectories fluctuate around the deterministic steady states,

$$m^* = \frac{G\kappa}{\delta_m}, \quad p^* = \frac{\gamma m^*}{\delta_p + \lambda},$$

which were derived in the previous section. The deterministic differential equations therefore describe the *mean behavior* of the system, while the stochastic simulations reveal the variability around that mean.

Steady-state distributions and noise scaling.

To characterize this variability in more detail, we analyze the distribution of copy numbers once the system has reached steady state. Figures 21.2C and D show the steady-state distributions of mRNA and protein counts across many independent realizations.

We can further quantify noise by analyzing how the variance scales with the mean. To this end, we vary the transcription rate κ while keeping all other parameters fixed. For mRNA, the variance scales approximately linearly with the mean (Fig. 21.2E), as expected for a Poisson process where

$$\text{Var}(m) = \langle m \rangle.$$

In contrast, the variance of protein copy numbers increases faster than the mean (Fig. 21.2F), indicating *overdispersion* relative to a Poisson process. This enhanced variability arises because protein synthesis depends on the stochastic mRNA copy number. Fluctuations in mRNA propagate to proteins and effectively amplify noise. Protein dynamics therefore combine intrinsic stochasticity in translation with inherited fluctuations from upstream transcriptional events.

21.4 Transcriptional Bursting

Overall, these considerations suggest a considerable amount of noise in gene expression. However, in real cells gene expression noise can be even stronger than predicted by this minimal birth–death model. In both bacteria and eukaryotes, genes are often not transcribed at a constant rate but rather in bursts: periods of high transcriptional activity alternate with silent periods. This phenomenon, known as transcriptional bursting, arises from stochastic switching of promoters between active and inactive states.

This produces mRNA distributions that are broader than Poisson. Notably, this is consistent with the overdispersion often observed in RNA-seq data. The negative binomial distribution commonly used to model RNA-seq counts can be interpreted as the steady-state distribution expected from a bursting gene.

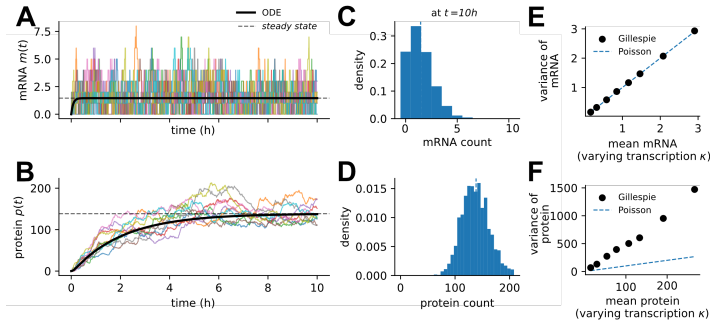


Figure 21.2: **Stochastic gene expression in the central dogma model.** (A,B) Stochastic trajectories of mRNA (A) and protein (B) copy numbers following the induction of gene expression at time 0. Each line represents one realization of the stochastic process solved by the Gillespie algorithm. (C,D) Steady-state distributions of mRNA (C) and protein (D) copy numbers obtained from many independent simulations. (E,F) Noise scaling as transcription rate κ and thus the mean number of mRNA and protein copy numbers is varied. All other parameters as for the deterministic version of the transition, Fig. 20.2.

In practice, however, negative binomial dispersion estimates in differential expression tools such as DESeq2 may also reflect additional sources of variability, including biological heterogeneity and measurement noise.

Part IX

Week 9: Nonlinear dynamics (optional)

22 Nonlinear Dynamics

i Optional topic. Weeks 8 and 9 (dynamical systems and nonlinear dynamics) are an optional part of the course.

So far, we have focused on dynamical systems that are *linear*. For example, in the central dogma model we described protein dynamics for a continuously transcribed gene (κ is constant) in the form

$$\frac{dp}{dt} = f(p),$$

with

$$f(p) = \gamma m - (\delta_p + \lambda)p.$$

with $m = m^* = \kappa/\delta_{mRNA}$ being constant if a steady state in mRNA levels have been reached.

This equation is called linear because the rate of change $f(p)$ depends linearly on the state variable p . Production terms were either constant or proportional to upstream variables, and degradation terms were proportional to molecule number.

1-dimensional linear systems possess a single steady state p^* , defined mathematically by

$$f(p^*) = 0.$$

Graphically, this steady state corresponds to the point where the curve $f(p)$ crosses zero (Fig. ??B). Stead states can be stable or unstable, that is a small deviation from the steady state either

decays back towards the steady state (stable) or grows away from the steady state. In the case of linear protein synthesis discussed above, the steady state is stable. Mathematically, this happens as the slope of f around p is negative and it can be illustrated by arrows (Fig. ??C).

Linear systems provide an important baseline for understanding biological dynamics. For example, the central dogma model we introduced established a quantitative relationship between steady-state mRNA and protein abundance, capturing the general trends observed experimentally (Fig. 20.1).

However, biological systems are often inherently nonlinear. Non-linearity enables qualitatively new behaviors that are biologically essential and impossible in purely linear systems. For example, proteins regulate their own production, transcription factors bind cooperatively, enzymes saturate, and promoters switch between active and inactive states. In all these cases, reaction rates depend on molecular concentrations in a nonlinear way generating features like threshold responses, switch-like behavior, multiple steady states, and memory of past signals.

i Info: Definition of Linear and Nonlinear Dynamical Systems

Consider a one-dimensional dynamical system

$$\frac{dx}{dt} = f(x).$$

The system is called *linear* if the right-hand side is a linear function of the state variable x , i.e.

$$f(x) = ax + b,$$

with constants a and b .

Equivalently, a system is linear if the state variable appears only to the first power and is not multiplied with itself or with other state variables.

If $f(x)$ contains nonlinear functions of x (for example powers x^n with $n \neq 1$, Hill functions, products of variables, exponentials, or other nonlinear dependencies), the system is called *nonlinear*.

In a linear system, the rate of change depends at most proportionally on the current state. In a nonlinear system, the dependence on the state variable is more complex.

22.1 Negative Feedback in Gene Expression

As a first example of how nonlinearity impacts gene expression, consider a protein that represses its own transcription. This is a classical example of a *negative feedback loop*.

Instead of a constant transcription rate κ , we model transcription using a Hill-type repression function (Fig. ??D):

$$\kappa(p) = \frac{\kappa_0}{1 + (p/K)^n}.$$

Here, κ_0 is the maximal transcription rate, K sets the characteristic protein level at which repression becomes significant, and n is the Hill coefficient describing cooperativity, that is how steeply transcription changes with protein concentration. Such functional forms arise naturally from protein–DNA binding equilibria as we discussed further below.

With this more complex transcription behavior, the full central dogma equations now become

$$\frac{dm}{dt} = G\kappa(p) - \delta_m m, \quad \frac{dp}{dt} = \gamma m - (\delta_p + \lambda)p.$$

Mathematically, that is a fully coupled system of differential equations. mRNA copy numbers affect protein synthesis (translation), and protein copy numbers affect mRNA synthesis (transcription).

To proceed, we here a small trick. Specifically, we assume that mRNA turnover is much faster than protein turnover. We use this separation of time scales to simplify the dynamics and focus on protein dynamics (see box):

$$\frac{dp}{dt} = f(p) = \frac{\gamma G}{\delta_m} \frac{\kappa_0}{1 + (p/K)^n} - (\delta_p + \lambda)p.$$

i Info: Separation of timescales

As we have seen from bacterial measurements, mRNA lifetimes are on the order of minutes, whereas protein turnover occurs on the order of hours. If this difference in time scales is sufficiently large, we can simplify the dynamics by assuming that mRNA rapidly adapts to its steady state while protein copy numbers p change more slowly.

Mathematically, we approximate mRNA by its quasi-steady value:

$$m = m(p) \approx \frac{G\kappa(p)}{\delta_m}.$$

Substituting this expression into the protein equation yields a single differential equation for the variable p , as shown in the main text.

This is a nice trick but notably it only works if timescales are clearly separated. For mRNA and protein dynamics this condition is well fulfilled in some scenario but it can also fail which is important to keep in mind.

With this reduction, we can analyze the dynamics graphically again in a 1d representation. The function $f(p)$ is shown in Fig. ??E. Steady states satisfy $f(p^*) = 0$. As in the linear case, there is a single stable steady state toward which the system relaxes (Fig. XYZ shows the flow diagram). However, $f(p)$ is now nonlinear. How does this differ from the linear case? Negative feedback increases the magnitude of the negative slope near p^* . In other words, the restoring force becomes stronger.

To illustrate this, we again consider gene expression ON-dynamics, comparing the linear and the negative-feedback scenarios ???. Starting from $p = 0$, the system with negative feedback approaches its steady state more rapidly. Thus, negative feedback enhances dynamic stability and accelerates relaxation toward the steady state.

Beyond deterministic stability, negative feedback also affects stochastic fluctuations. In the simple linear birth–death model, protein noise at steady state is approximately Poisson:

$$\text{Var}(p) \approx \langle p \rangle.$$

With negative feedback, fluctuations are actively corrected. If p fluctuates upward, transcription decreases; if p fluctuates downward, transcription increases. This suppresses variance relative to the unregulated case.

Stochastic simulations (see Fig. 22.1) show that negative autoregulation reduces the Fano factor

$$\frac{\text{Var}(p)}{\langle p \rangle}$$

compared to the linear model. For sufficiently strong feedback, protein fluctuations can become Poisson. When tweaking parameter levels, one can even get sub-Poissonian.

add  Figure

Figure 22.1: **Noise suppression by negative feedback.**

Comparison of steady-state protein distributions for constitutive expression (linear model) and negative autoregulation with matched mean protein level. Negative feedback reduces variance and lowers the Fano factor relative to the Poisson baseline.

22.2 Molecular Implementation of Feedback Loops

We have seen that negative feedback stabilizes both the mean expression level and the magnitude of stochastic fluctuations. Is this biologically relevant, and how can such feedback loops be implemented at the molecular level? The short answer is: feedback regulation is ubiquitous in biological systems, and it arises naturally from molecular interactions.

To illustrate this more concretely, let us briefly revisit the basics of transcriptional control. Genes are not transcribed continuously at a constant rate. Instead, cells tightly regulate transcription by controlling the activity and recruitment of RNA polymerase (RNAP) to specific promoters.

In bacteria such as *E. coli*, there are typically on the order of 10^3 – 10^4 RNA polymerase molecules per cell, while the chromosome contains several thousand genes. Not all promoters are actively engaged at the same time, and RNAP is therefore a limited resource that must be dynamically allocated.

In eukaryotic cells, the situation is more complex. Multiple RNA polymerases transcribe different classes of genes, and transcription initiation involves large multi-protein complexes and chromatin remodeling. Nonetheless, the same general principle also applies: transcription depends on regulated molecular interactions at promoters.

For a gene to be transcribed, RNA polymerase must bind to its promoter and initiate transcription. Cells control this recruitment through regulatory proteins, which either enhance or inhibit RNAP binding.

As example, a negative feedback loop can be implemented when a the protein encoded by a gene acts as a repressor of its own promoter. As protein concentration increases, promoter occupancy by the repressor increases, reducing transcription.

A textbook example is the *lac* operon in *E. coli*. The LacI repressor binds to operator sequences in the promoter region and blocks transcription. LacI can bind to multiple operator

sites simultaneously, forming DNA loops that enhance repression. Such cooperative binding introduces nonlinearity in the transcription rate and can be described phenomenologically by Hill-type functions.

Importantly, transcriptional control is only one class of nonlinear regulation in the cell, and the Hill-type functions we introduced represent just one mathematical manifestation. Allosteric regulation of metabolic enzymes controls metabolic fluxes in a nonlinear manner, phosphorylation–dephosphorylation cycles generate highly nonlinear responses in signaling pathways, and feedback between mechanical forces and molecular assembly produces nonlinear input–output relationships. Furthermore, spatial organization and compartmentalization introduce additional nonlinear effects, as reaction rates depend on local concentration, diffusion, and molecular crowding.

22.3 Positive Feedback and Bistability

Negative feedback stabilizes expression levels. A second fundamental regulatory motif is *positive feedback*, in which a protein enhances its own production. Such self-activation can qualitatively change system behavior.

To illustrate this within the central dogma framework, we here consider a scenario where transcription increases with protein abundance according to a Hill-type activation function:

$$\kappa(p) = \kappa_0 \frac{(p/K)^n}{1 + (p/K)^n}.$$

Here, κ_0 denotes the maximal transcription rate, K sets the activation threshold, and n controls the steepness (cooperativity) of the response. Unlike repression, this function increases sigmoidally with p , reflecting that protein promotes its own production.

Assuming time-scale separation between mRNA and protein dynamics, the protein dynamics reduce to

$$\frac{dp}{dt} = f(p) = \frac{\gamma G}{\delta_m} \kappa(p) - (\delta_p + \lambda)p.$$

The first term describes production via self-activation, the second term combines degradation and dilution.

As before, steady states satisfy

$$f(p^*) = 0.$$

Graphically, steady states correspond to intersections between:

- the nonlinear production term
- the linear loss term $(\delta_p + \lambda)p$

For certain parameter values, three intersections can occur. Two of these are stable and one is unstable. Stability follows from the slope condition:

$$\left. \frac{df}{dp} \right|_{p^*} < 0 \quad \text{stable}, \quad \left. \frac{df}{dp} \right|_{p^*} > 0 \quad \text{unstable}.$$

Intuitively, in the unstable state a small increase in p causes further growth away from that state.

22.4 From Steady States to Bifurcations

So far, we have analyzed steady states for one fixed set of parameters. However, biological systems operate under changing conditions.

Suppose we vary a control parameter, for example:

- the maximal activation strength κ_0 ,
- the cooperativity n ,
- or the growth rate λ (dilution).

Mathematically, steady states satisfy

$$f(p^*, \lambda) = 0,$$

so the steady-state protein level p^* becomes a function of the parameter.

For weak activation (small n or low κ_0), the curves intersect only once, producing a single stable steady state.

However, for sufficiently strong positive feedback, three intersections appear.

At specific critical parameter values, two steady states collide and annihilate each other. This qualitative change in the number of steady states is called a *bifurcation*.

A bifurcation occurs when a small change in a parameter causes a qualitative change in system behavior.

In this case, a saddle-node (fold) bifurcation creates or destroys a pair of steady states: one stable and one unstable.

To visualize this systematically, we plot all steady states p^* as a function of the control parameter.

Such a plot is called a *bifurcation diagram*.

Stable steady states are drawn as solid lines. Unstable steady states are drawn as dashed lines.

For strong positive feedback, the diagram has an S-shape:

- A lower stable branch (low expression),
- An upper stable branch (high expression),
- An intermediate unstable branch separating them.

The turning points correspond to saddle-node bifurcations.

22.5 Switch-like Behavior

The presence of two stable steady states implies bistability.

Small perturbations around either stable state decay back to that state. However, perturbations that cross the unstable threshold drive the system toward the alternative stable state.

The system therefore behaves as a switch:

- Below a critical protein level, expression collapses to the low state.
- Above that threshold, expression rises to the high state.

The unstable steady state acts as a separatrix: it divides the basins of attraction of the two stable states.

Such behavior is impossible in linear systems and arises purely from nonlinear positive feedback.

If we now slowly increase the control parameter (for example the activation strength), the system remains on the lower stable branch until the first saddle-node bifurcation is reached. At this point, the lower state disappears and the system jumps to the upper branch.

If we subsequently decrease the parameter again, the system does not switch back at the same point. Instead, it returns at the second saddle-node bifurcation.

Thus the transition thresholds depend on direction. This phenomenon is called hysteresis.

Hysteresis means the system retains memory of its past state.

22.6 Nonlinearity Changes System Structure

	Linear Model	Positive Feedback Model
Number of steady states	One	One or multiple
Response to input	Smooth	Threshold-like
Memory	None	Possible
Qualitative transitions	No	Yes

Nonlinearity alters not just parameter sensitivity but the qualitative structure of the dynamical system.

22.7 Parameter Dependence and Bifurcations

So far, we have analyzed the system for fixed parameter values. However, biological systems rarely operate at fixed parameters. External signals, inducer concentrations, nutrient availability, or signaling inputs effectively modify parameters of the system, for example the maximal transcription rate κ_0 .

We now ask a more general question:

How do steady states change when a parameter is varied continuously?

In the positive feedback system, the number and stability of steady states depend sensitively on κ_0 .

For small κ_0 , production is too weak to sustain a high-expression state. Only a single low-expression steady state exists.

As κ_0 increases, the sigmoidal production curve rises. At a critical value of κ_0 , two new steady states appear simultaneously: one stable and one unstable. The system now possesses three steady states.

If κ_0 is increased further, the unstable and low-expression steady states eventually collide and disappear, leaving only the high-expression state.

A qualitative change in the number or stability of steady states as a parameter is varied is called a *bifurcation*.

Figure 22.2 illustrates this behavior in a bifurcation diagram. The horizontal axis represents the control parameter κ_0 , while the vertical axis shows the corresponding steady-state protein levels p^* . Solid lines denote stable steady states, and dashed lines denote unstable steady states.

add Figure

Figure 22.2: **Bifurcation diagram for positive feedback.**

Steady-state protein levels p^* as a function of the maximal transcription rate κ_0 . For small κ_0 , a single stable steady state exists. At a critical value, two additional steady states appear (one stable, one unstable). Over an intermediate range of κ_0 , the system is bistable. Stable steady states are shown as solid lines, unstable steady states as dashed lines.

This diagram summarizes how nonlinearity changes the structure of the system. In contrast to linear systems, where steady states vary smoothly with parameters, nonlinear systems can undergo abrupt qualitative transitions.

22.8 Hysteresis and Memory

The existence of two stable steady states over a range of parameter values has an important dynamical consequence: the system's behavior depends on its history.

Suppose κ_0 is slowly increased from a small value. The system remains in the low-expression state until the upper bifurcation point is reached. At that point, the low state disappears and the system switches abruptly to the high-expression state.

Now suppose κ_0 is decreased again. The system does not immediately return to the low state. Instead, it remains in the high-expression state until a lower critical value of κ_0 is reached, where the high state disappears.

Because the switching points differ depending on the direction of parameter change, the system exhibits *hysteresis*.

Hysteresis implies memory. The current state of the system depends not only on the current parameter value, but also on the path by which that value was reached.

add

Figure

Figure 22.3: **Hysteresis in a bistable system.** When the control parameter κ_0 is increased (forward sweep), the system switches from low to high expression at the upper threshold. When κ_0 is decreased (backward sweep), the switch back occurs at a lower threshold. The system therefore displays path-dependent behavior.

Such hysteretic behavior is impossible in linear systems. It arises purely from nonlinear feedback and the coexistence of multiple stable steady states.

Biologically, hysteresis enables robust cell-fate decisions, irreversible developmental transitions, and stable differentiation states. Once a system has switched into a high-expression state, it can remain there even if the inducing signal is partially removed.

22.9 Nonlinearity Changes System Structure

We can now summarize the progression:

	Linear System	Positive Feedback System
Number of steady states	One	One or multiple

	Linear System	Positive Feedback System
Response to parameter change	Smooth	Abrupt (bifurcation)
Memory	None	Possible (hysteresis)
Qualitative transitions	No	Yes

Nonlinearity does not merely modify parameter sensitivity. It changes the qualitative structure of the dynamical system.

This structural change — the emergence of multiple stable states, bifurcations, and hysteresis — underlies many forms of biological decision-making.

References